

Computational Complexity

Lecture Notes

Snapshot as of 2026/02/01 16:03:53

Abstract

These are draft lecture notes for the computational complexity course in Winter Semester 2025/26 at Ariel University. They are written by students: Efrat Cohen, Dotan Danino, Talya Cohen, Dvir Opatowsky, Hillel Ohayon, Ayelet Katkov, Alok Maity, Yohann Pardes, and edited by Edward A. Hirsch.

Disclaimer: this draft definitely contain typos. If you spot one, please report it either to the author of the corresponding section or to the editor (Edward).

Contents

1	Lecture 1. Scribe: Efrat Cohen	4
1.1	Assumptions	4
1.2	Reminder: DTM	5
1.3	Deterministic time complexity and the class P	6
1.4	Nondeterministic machines, witnesses, and the class NP	6
1.5	Important examples of languages in NP	8
1.6	Classes of complements and co-NP	10
1.7	Important examples of languages in co-NP	11
1.8	Reductions, hardness, and completeness	12
1.9	Many-one polynomial-time reductions, NP - and co-NP -completeness . . .	13
1.10	Oracle machines and Turing reductions	14
2	Lecture 2. Scribe: Dotan Danino	15
2.1	Search Problems and Classes	15
2.2	Turing reductions between arbitrary problems (A formalistic definition) . .	16
2.3	Levin's Reductions	16
2.4	An important example of Levin's reduction: Circuit-SAT to 3-SAT	18
2.5	A search-to-decision reduction for SAT and other NP -complete languages .	19
2.6	Levin's optimal algorithm for any NP problem (Universal Search)	21

3	Lecture 3. Scribe: Talya Cohen, Dvir Opatowsky	21
3.1	Search-NP and Levin reductions — recap from the previous lecture	21
3.2	The Cook–Levin Theorem: a proof that transforms Turing machines into circuits	24
4	Lecture 4. Scribe: Hillel Ohayon	28
4.1	NP -Intermediate Languages	28
4.1.1	Defining the Padded Language SAT_H	29
4.1.2	Defining the Function H	29
4.2	Polynomial Hierarchy (PH)	32
5	Lecture 5. Scribe: Talya Cohen	36
5.1	The Polynomial Hierarchy, continued	36
5.2	Oracle Turing Machines: recap	36
5.3	Quantifier Characterization of the Polynomial Hierarchy	37
5.4	Complete Problems for Σ_k^P (and Π_k^P)	41
5.5	Easy Observations and the Collapse of the Polynomial Hierarchy	43
5.6	Polynomial-Size Circuits and the Karp–Lipton Theorem	44
5.6.1	Polynomial-Size Circuits	44
5.6.2	The Karp–Lipton Theorem (introduction)	45
6	Lecture 6. Scribe: Ayelet Katkov	45
6.1	Reminders from previous lectures	46
6.2	The Karp–Lipton Theorem (now with a proof)	46
6.3	Fixed-Polynomial Circuit-Size Lower Bounds	50
6.3.1	Fixed-Polynomial Circuit Lower Bounds for Σ_4^P : Kannan’s Theorem	51
6.3.2	Pushing it lower: Σ_2^P instead of Σ_4^P	53
6.4	Introduction to Space Complexity	54
6.4.1	Definitions: DSpace , PSPACE , NSpace	54
6.4.2	Intuition: how multiple quantifiers (two-player games!) can be computed in polynomial space	55
6.4.3	Examples of PSPACE problems	57
6.4.4	QBF-SAT : a PSPACE -Complete Problem	58
7	Lecture 7. Scribe: Dotan Danino	60
7.1	PSPACE , continued	60
7.1.1	QBF : A PSPACE -Complete Problem	60
7.1.2	Review of the relations between the classes we considered	63
7.2	Small Space Classes: L and NL	63
7.2.1	Counting Configurations in Small Space Classes	65
7.2.2	Examples of Logspace Problems	66

7.2.3	Logspace reductions and P -completeness	67
7.2.4	CIRCUIT.EVAL : A P -complete problem	68
8	Lecture 8. Scribe: Alok Maity	69
8.1	Small space complexity classes — reminders	69
8.2	The reachability problem STCON , with application to space complexity . . .	69
8.2.1	Solving STCON in quadratic deterministic space	69
8.2.2	Application of STCON : Savitch's Theorem	71
8.3	The Immerman–Szelepcsényi theorem	74
8.4	Circuits as a computational model and Introduction to parallel computations	77
8.4.1	Depth is Time	77
8.4.2	Examples of parallel algorithms	78
8.4.3	Polynomial-time uniform and logspace-uniform families of circuits .	79
8.4.4	Relations between parallel computation classes and other classes . .	80
8.5	The space hierarchy theorem (and other hierarchy theorems)	81
8.5.1	A few words about universal Turing machines	81
8.5.2	The space hierarchy theorem	82
8.5.3	The time hierarchy theorem	83
8.5.4	The nondeterministic time hierarchy theorem	85
8.5.5	Corollaries	85
8.6	Padding	85
9	Lecture 9. Scribe: Yohann Pardes	86
9.1	A postscriptum to the previous lecture: Arithmetic circuits	86
9.1.1	Randomized (probabilistic) computation	87
9.1.2	Introduction to randomized algorithms	88
9.2	One-sided bounded error	88
9.2.1	Example: Verifying matrix multiplication	89
9.2.2	Probability amplification: the case of one-sided error	90
9.2.3	Example: Polynomial Identity Testing	91
9.2.4	Two-sided bounded error	93
9.2.5	Amplification of success: the case of two-sided errors	94
9.2.6	RP and BPP via witnesses and computation trees	94
9.3	Errorless algorithms	95
9.4	Closure properties and relation of ZPP , RP , and BPP to other classes . .	96
9.4.1	Example: Errorless algorithm	97
10	Lecture 10. No scribe, essentially just reformatted slides	98
10.1	BPP , continued	98
10.1.1	Reminder: A PPTM, what is it?	98
10.1.2	Reminder: Two-sided bounded error	99

10.1.3	Non-uniform advice: another view of $\mathbf{P}/\mathbf{poly}$	100
10.1.4	$\mathbf{BPP} \subset \mathbf{P}/\mathbf{poly}$	101
10.1.5	$\mathbf{BPP} \subseteq \Sigma_2^p \cap \Pi_2^p$	102
10.1.6	Probabilistic reductions: Solution isolation by a randomized reduction	104
10.2	Relativization: How do oracles transform classes (and proofs)	107
10.2.1	Relativization	107
10.2.2	Exercises: How to work correctly with the oracle notation	107
10.3	Introduction to Interactive Protocols (Interactive Proofs)	108
10.3.1	Deterministic protocols	109
10.3.2	Protocols with private coin	109
11	Lecture 11. Scribes: Yohann Pardes & Hillel Ohayon	112
11.1	Interactive Protocols (continued)	112
11.1.1	Protocols with private coin	112
11.1.2	Yet simpler protocols: $\mathbf{MA}$ (Merlin speaks first)	117
11.1.3	Polynomially many rounds: $\mathbf{IP} = \mathbf{PSPACE}$	120
12	Lecture 12. No scribe, essentially just reformatted slides	127
12.1	More examples of interactive protocols ($\mathbf{IP}$)	127
12.1.1	Sumcheck protocol	127
12.1.2	Interactive protocol for computing the permanent of an integer matrix	128
12.1.3	An interactive protocol for $\mathbf{QBF}$	129
12.2	Probabilistically Checkable Proofs (PCP) and Hardness of Approximation	131
12.2.1	Probabilistically Checkable Proofs and the formulation of the PCP theorem	131
12.2.2	Reminder: Optimization problems, Approximation algorithms	132
12.2.3	PCP Theorem and Hardness of Approximation	133
12.2.4	The proof of $\mathbf{NP} \subseteq \mathbf{PCP}(\text{poly}(n), O(1))$.	138

1 Lecture 1. Scribe: Efrat Cohen

The course is mostly based on the book “Computational Complexity: A Modern Approach” by Sanjeev Arora and Boaz Barak [AB09]. Of course, it is not the whole book, but also there may be additions that are not in the book.

1.1 Assumptions

In this course we always speak about binary strings (because everything can be encoded by bits), thus all our languages (decision problems) are subsets of $\{0, 1\}^*$ (sometimes we identify a language L with its characteristic function $L(x)$ that says “yes” if $x \in L$ and “no” otherwise).

We do not mention this explicitly, but we assume that all objects we study are represented by their “reasonable” encodings that are not overblown (one does not need an exponential number of bits to represent a graph) and do not contain any “magic” information (the chromatic number is not a part of the representation of a graph). One can always assume that all encodings are syntactically correct: this allows us, for example, to say that $\text{SAT} = \overline{\text{UNSAT}}$: all binary strings that do not represent Boolean formulas are mapped by this encoding to some fixed small object, say, $x \wedge \bar{x}$.

1.2 Reminder: DTM

Let us begin with a reminder of our main computational model: [deterministic Turing machine](#). One can think of it as an algorithm (deterministic and with unbounded memory), and according to Church–Turing thesis (even its polynomial-time bounded version) we can indeed think so. However, sometimes we need a more formal definition.

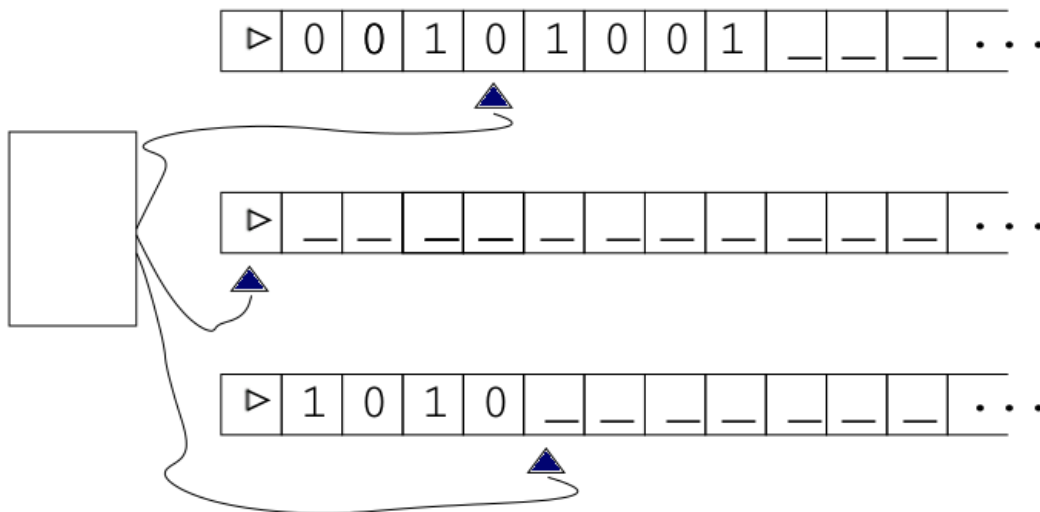


Figure 1: DTM

Definition 1.1 (Deterministic Turing machine, DTM). Formally, a k -tape DTM M is described by its finite set of states Q (with two special states: a starting state q_{start} and a halting state q_{halt}), its tape alphabet Γ (a finite set of symbols that includes a designated “blank” symbol $\sqcup$, a designated “start” symbol $\triangleright$, and the numbers 0 and 1 — one can assume that no other symbols are needed), and a function δ , called the transition function of M , or its program, that tells us what to do at each step:

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{\leftarrow, \rightarrow, \cdot\}^k$$

(see Figure 1).

DTMs can compute functions and predicates. For the latter purpose some sources prefer to define the accepting and the rejecting state q_{yes}, q_{no} instead of q_{halt} ; others simply let the machine write the answer bit on its [output] tape and go to q_{halt} . Sometimes it is useful to distinguish the input tape (read-only), the output tape (write-only), and other (working, read-write) tapes. In most cases it is not needed.

Every Turing machine has a number (write down its program as a binary string and prepend it with 1 to make it the binary representation of a positive natural number without leading zeroes), called **Turing number**. Thus one can list all the DTMs as $T_1, T_2, \dots$. One can assume that if such a description is followed by a garbage, then it represents the same machine, so each machine appears infinitely many times in this numbering.

There exists a universal Turing machine that given (on its input tape) a Turing number $n \in \mathbb{N}$ and input $x \in \{0, 1\}^*$ outputs the same answer as $T_n(x)$. Its running time can be made as small as $O(t \log t)$, where t is the number of steps that T_n does and the constant in $O(\dots)$ depends on n (but not on x : the asymptotics is over $|x|$). For most purposes it's enough to assume that the overhead is polynomial (and the constant before this polynomial is roughly the size of the program of T_n , say, $100 \log_2 n$).

1.3 Deterministic time complexity and the class **P**

We all know the class **P** of polynomial-time decidable problem. Let us redefine it using classes of specific polynomial complexity.

Classes **DTime** formalize the notion of computations of time-bounded complexity.

Definition 1.2. The class **DTime**. Let $T : \mathbb{N} \rightarrow \mathbb{N}$ be some function. We let **DTime** $[T(n)]$ be the set of all languages that can be accepted by a DTM in time $O(T(n))$. More formally,

$$L \in \mathbf{DTime}[T(n)] \iff \exists \text{DTM } M : L = L(M) \text{ and } \max_{x \in \{0,1\}^n} \text{time}_M(x) = O(T(n)).$$

Definition 1.3 (The class **P**).

$$\mathbf{P} = \bigcup_{k \in \mathbb{N}} \mathbf{DTime}[n^k].$$

1.4 Nondeterministic machines, witnesses, and the class **NP**

One can define the class **NP** in two ways: via a polynomial-time verifiable relation between questions and solutions (that will be used later also to define search problems) and via nondeterministic machines.

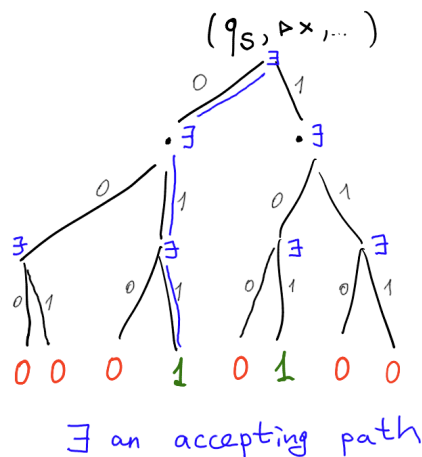
Polynomial-time verifiable solutions.

Definition 1.4 (The class **NP**). A language L is in **NP** if there exist a polynomial p and a binary relation¹ $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ decidable² by a polynomial-time DTM such that $\forall x \in \{0, 1\}^*$

$$x \in L \iff \exists w \in \{0, 1\}^{p(|x|)} \text{ such that } R(x, w) = 1.$$

If $R(x, w) = 1$ then we call w a **witness**, or a **certificate**, or a (correct) **solution**, for $x \in L$.

Nondeterministic Turing machines. The class **NP** can also be defined using nondeterministic Turing machines (abbreviated NDTM or NTM, where **N** stands for **n**ondeterministic). The only difference between an NTM and a DTM is that an NTM has more than one option of how to proceed. One can define its transition function as a multifunction (that is, a function that returns a (nonempty) set of possibilities). However, it is easy to see that two options are always enough (one can split a choice between 2^k options into k choices between just two options). Thus a computation of a NTM can be described as a binary tree



$x \in L \Rightarrow$ at least one
green leaf

$x \notin L \Rightarrow$ red leaves only

Figure 2: Computation tree of NTM

¹There are three equivalent options: to let witnesses have length $p(|x|)$, to let them have length $\leq p(|x|)$, and let them have arbitrary length but require that if there is a correct solution then there is also a short correct solution ($\exists w R(x, w)$, then $\exists w' \in \{0, 1\}^{\leq p(|x|)} R(x, w')$). The latter is sometimes called a **polynomially bounded relation**, but we will use all three approaches interchangeably.

²Note that we can think of R as a language of (reasonably encoded) pairs.

Leaves of this tree are marked by 1 or 0 depending on whether the machine accepts or rejects.

We say that a NTM M accepts its input x if and only if there is some “successful” sequence of these choices (which we call the nondeterministic choices of M) that would make M to accept on input x (thus there is a leaf marked with 1, we call the path from the root to this leaf an [accepting path](#)).

The running time of NTM is the maximum of the number of steps it makes before halting for every possible sequence of nondeterministic choices, that is, the time is the depth of its computation tree. (We can assume that it makes a choice at every step.)

If we know the running time upper bound of our machine, we can assume that the lengths of all the branches are equal (if the machine stops earlier, it can patiently wait until this its time elapses).

We can define **NTime** classes by analogy to **DTime**.

Definition 1.5. The class **NTime**. Let $T : \mathbb{N} \rightarrow \mathbb{N}$ be some function. We let **DTime** $[N(n)]$ be the set of all languages that can be accepted by a NTM in time $O(T(n))$. More formally,

$$L \in \mathbf{NTime}[T(n)] \iff \exists \text{NTM } M : L = L(M) \text{ and } \max_{x \in \{0,1\}^n} \text{time}_M(x) = O(T(n)).$$

Theorem 1.1. The following definition of **NP** is equivalent to Def. 1.4:

$$\mathbf{NP} = \bigcup_{k \in \mathbb{N}} \mathbf{NTime}[n^k]$$

Proof. The main idea is that the sequence of nondeterministic choices made by an accepting computation of an NTM N can be viewed as a certificate that the input is in the language, and vice versa.

Namely, given a relation R with witnesses of a polynomial length $p(n)$, where n is the input length, a nondeterministic machine will write nondeterministically a witness w (that is, choose 0 and 1 as the next bit nondeterministically until it writes $p(n)$ bits) and then execute the polynomial-time machine for R to check $R(x, w)$.

In the other direction, $R(x, w)$ is defined by a deterministic machine that reads w bit by bit and executes the program of N so that instead of its i -th nondeterministic choice it deterministically selects the option (0 or 1) designated by the bit w_i . $\square$

1.5 Important examples of languages in NP

Satisfiability of Boolean formulas. As an example of relation R we can look at a Boolean formula e.g.

$$F = ((x_1 \vee \overline{x_2} \vee x_3) \wedge (x_3 \vee \overline{x_4})).$$

Its solution (called a [satisfying assignment](#)) contains values for all its variables that make this formula true. For example,

$$A = \{x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 0\}$$

makes true the formula F above.

It's enough to provide n bits (where n is the number of variables in the input formula) to describe an assignment, but we will write it in more detail (like $x_1 = 1, x_3 = 0$) to point out which variables are we going to substitute in the case of a partial assignment.

Given A , we can check in polynomial time whether indeed $F(A) = 1$.

Thus the relation $R(F, A) = "F(A) = \text{True}"$ indeed defines a problem in **NP**: the certificates are short (even shorter than the questions!) and the verification is fast (polynomial-time). They call this problem **FormulaSAT**.

Recall that a Boolean formula over variables $x_1, \dots, x_n$ is in conjunctive normal form (CNF) if it is an AND of OR's of variables or their negations. We call these disjunctions **clauses** (for example, the formula in the example above contains two clauses $(x_1 \vee \bar{x}_2 \vee x_3)$ and $(x_3 \vee \bar{x}_4)$). If every clause in a formula has length at most k , we call it a **k -CNF**.

Let us remind two important languages

Definition 1.6.

$$\text{SAT} = \{F \text{ in CNF} : \exists A \ F(A) = 1\},$$

$$k\text{-SAT} = \{F \text{ in } k\text{-CNF} : \exists A \ F(A) = 1\}.$$

Later we will give (or remind) a proof of the Cook–Levin Theorem that says that $k = 3$ already makes this problem difficult: **3-SAT** is **NP-complete**³.

Satisfiability of Boolean circuits

Definition 1.7. A **Boolean circuit** is a directed acyclic graph that has sources known as input nodes $x_1, x_2, \dots, x_n$, unary and binary Boolean operations in its internal (non-input) nodes (called also **gates**), and one or more outputs (internal nodes designated as the outputs $y_1, y_2, \dots, y_n$). The indegree of an internal node is 1 or 2 depending on the arity of the operation in this node. The outdegree is arbitrary. When there is a single output node, it is convenient to assume that its outdegree is zero, that is, it is a sink.

For example, the circuit on Fig. 3 computes the majority (voting) function of three input bits (voters).

A circuit is a just a diagram showing how to compute the output from the input by applying Boolean operations (OR $\vee$, AND $\wedge$, XOR $\oplus$, NOT $\neg$, etc): one feeds the input bits into $x_1, x_2, \dots, x_n$ and computes the operations one by one; once the next operation gets all its arguments (flowing in through the incoming edges), it can be computed.

Alternatively, one can view a Boolean circuit as a straight-line Boolean program consisting of equations $y_t = y_j \odot y_k$, where $\odot$ is a binary operation, y_i 's are associated with the nodes of the circuit, the first n of them are x_i 's, and $t > j, k$ (that is, the order is topological, so that we never need a value that has not been computed yet).

³This notion will be reminded later in this lecture.

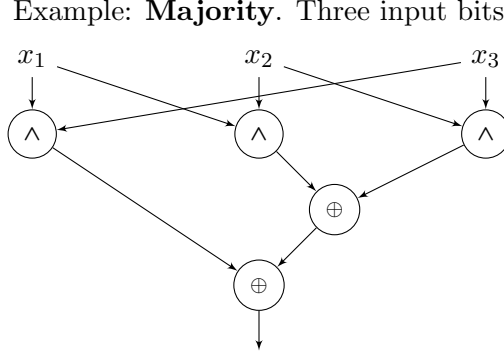


Figure 3: A circuit for the majority function of three input bits.

A Boolean circuit with n inputs and m outputs naturally computes a Boolean function $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ that we will denote by the same letter as the circuit.

Definition 1.8. **Circuit-SAT** The circuit satisfiability problem (language) **Circuit-SAT** (or **CKT-SAT**) consists of all (strings representing) circuits with a single output that have a satisfying assignment.

$$\text{Circuit-SAT} = \{C \text{ is a circuit} : \exists A \ C(A) = 1\},$$

where A is an assignment of Boolean variables to all variables of C .

Circuit-SAT is clearly in **NP** because the satisfying assignment can serve as the certificate. In what follows we will prove that

Theorem. **Circuit-SAT** is **NP**-complete.⁴

Then Cook–Levin theorem will be a corollary that we will prove in the next lecture. Clearly **3-SAT** is a particular case of **SAT**, which is a particular case of **FormulaSAT**, which is a particular case of **Circuit-SAT** (a formula is essentially a rooted tree, a particular case of a directed acyclic graph).

1.6 Classes of complements and co-NP

For a $L \subseteq \{0, 1\}^*$ is a language, we denote by $\bar{L}$ the complement of L , i.e., $\bar{L} = \{0, 1\}^* \setminus L$.

Definition 1.9 (co- $\mathcal{C}$). For every class of languages $\mathcal{C}$ we define the class of complements

$$\text{co-}\mathcal{C} = \{L : \bar{L} \in \mathcal{C}\}.$$

⁴Lemma 6.31 from the book [AB09]

Note that it is not the complement of the class itself!

Obviously, $\mathbf{co-P} = \mathbf{P}$ (when a DMT enters its halting state, it can accept instead of rejecting and reject instead of accepting). This is (as we think) not so for $\mathbf{co-NP}$.

While the computation of a nondeterministic machine that is the base for the class $\mathbf{NP}$ can be thought of as a tree with the existential quantifier $\exists$ in each node, for a **co-nondeterministic** Turing machine (**coNTM**) it is a tree with the existential quantifier $\forall$ in each node. Once again let us emphasize that this is the quantifier what is important, and not particular symbols marking the leaves. Still let us define a co-nondeterministic machine: it consists of the same ingredients as a NTM, but the notion of accepting is different: it accepts its input if and only if *every* sequence of nondeterministic choices leads to accepting (every path comes to a leaf marked with 1).



Figure 4: A computation tree of coNTM

Therefore, it rejects if there is a path that leads to a leaf outputting 0. See Figure 4.

Definition 1.10 ($\mathbf{co-NP}$, alternative definition).

$$L \in \mathbf{co-NP} \iff \exists \text{coNTM } M : L = L(M) \text{ and } \max_{x \in \{0,1\}^n} \text{time}_M(x) = O(T(n)).$$

One can define it also using witnesses (like Def. 1.4) replacing $\exists$ with $\forall$.

There are several important open questions involving the classes $\mathbf{P}$, $\mathbf{NP}$ and $\mathbf{co-NP}$:

$$\mathbf{NP} \stackrel{?}{=} \mathbf{co-NP}, \quad \mathbf{P} \stackrel{?}{=} \mathbf{NP}, \quad \mathbf{P} \stackrel{?}{=} \mathbf{NP} \cap \mathbf{co-NP}.$$

Of course, we believe in non-equalities here, but...

1.7 Important examples of languages in $\mathbf{co-NP}$

Example 1.1. $\overline{\text{SAT}} = \text{UNSAT} = \{\Phi : \Phi \text{ is not satisfiable}\} \in \mathbf{co-NP}$.

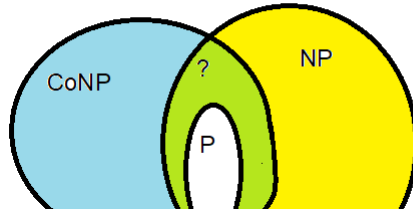


Figure 5: **P**, **NP**, **co-NP**

Example 1.2. Consider the complement to $\text{Circuit-SAT} \in \mathbf{NP}$:

$$\overline{\text{Circuit-SAT}} = \{C : \forall w \in \{0, 1\}^n C(w) = 0\} \in \mathbf{co-NP}.$$

(Note that what is important here for belonging to **co-NP** instead of **NP** is the quantifier ($\forall$ instead of $\exists$) and not the value (0 instead of 1).)

Example 1.3. The question whether in a processor is all good (there are **no** bugs) belongs to **co-NP**: given a bug (a specific input where it fails), one can check whether it is indeed a bug (but not prove there is no bug). In symbols: $\{(C, D) : \forall x C(x) = D(x)\} \in \mathbf{co-NP}$, where C, D are Boolean circuits. This is called the **equivalence checking problem**, which is quite important for the industry.

Remark 1.1. Note that the class **NP** formalizes the notion of a **proof system** for the propositional logic: a certificate is a “proof” that we can verify efficiently. Therefore, the question $\mathbf{NP} \stackrel{?}{=} \mathbf{co-NP}$ (the central question for proof complexity) can be reformulated as “is there a proof system that has a polynomial-size proof of unsatisfiability for every unsatisfiable formula”. While there are short proofs of satisfiability (satisfying assignments), it is unclear how to prove unsatisfiability concisely.

Note that if we had such “efficient” (polynomially bounded) proof systems, we could give short (and verifiable) proofs for equivalence checking (in particular, prove the correctness of new optimized parts of a microprocessor using the correctness of their old reliable versions).

1.8 Reductions, hardness, and completeness

A **reduction** reduces one computational problem W (here w is for weak) to another computational problem S (here s is for strong) so that given a device (black box) that solves S we can solve also W . We denote the existence of a reduction either as $W \leq S$ (because reducibility defines a partial order on problems) or as $W \rightarrow S$, with various subscripts and superscripts to emphasize the type of reductions that we use (yes, there are many different types of reductions!).

Once we have a class $\mathcal{C}$ of problems and we fixed a specific type of reductions, we can talk about

- **$\mathcal{C}$ -hard** problems: A problem H is hard for $\mathcal{C}$ under this type of reductions if for every $A \in \mathcal{C}$, it reduces to H (that is, $A \leq H$).
- **$\mathcal{C}$ -complete** problems: A problem is complete for $\mathcal{C}$ if it is hard for $\mathcal{C}$ and belongs to $\mathcal{C}$ itself.

All reductions that we consider in this course are polynomial-time.

1.9 Many-one polynomial-time reductions, NP- and co-NP-completeness

The simplest notion of reductions between decision problems (languages) is many-to-one.

Definition 1.11. A many-one polynomial-time reduction of W to S is a polynomial-time computable function⁵ $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ such that

$$\forall x \in \{0, 1\}^* \quad x \in W \iff f(x) \in S.$$

We denote the existence of such a reduction as $W \leq_p S$ or $W \leq_{m,p} S$.

Once we have a polynomial-time algorithm for S , a polynomial-time algorithm for W is immediate: map the input x using f to $f(x)$, then run the algorithm for S on $f(x)$ and return its answer. Therefore,

Lemma 1.1. If $S \in \mathbf{P}$ and $W \leq_p S$, then $W \in \mathbf{P}$.

In other words, $\mathbf{P}$ is closed under many-one polynomial-time reductions. It is easy to see that $\mathbf{NP}$ and $\mathbf{co-NP}$ are also closed under them (a NTM for W can run a reduction and then work as a NTM for S on its result).

Lemma 1.2. If $S \in \mathbf{NP}$ and $W \leq_p S$, then $W \in \mathbf{NP}$.
If $S \in \mathbf{co-NP}$ and $W \leq_p S$, then $W \in \mathbf{co-NP}$.

Very important! Many-one reductions do not allow to change the answer (for example, to flip the answer: yes instead of no, no instead of yes).

It is easy to see that many-one polynomial-time reductions are transitive and reflexive, so they indeed define a (non-strict) partial order.

⁵That is, computable on a polynomial-time DTM.

NP-completeness and co-NP-completeness. Now we can define **NP-hard**, **NP-complete**, **co-NP-hard**, **co-NP-complete** languages according to the general definition above and using many-one polynomial-time reductions.

Lemma 1.3. Let H be **NP-complete**. Then

1. $H \in \mathbf{P} \iff \mathbf{P} = \mathbf{NP}$.
2. $H \in \mathbf{co-NP} \iff \mathbf{NP} = \mathbf{co-NP}$.

The last item can be symmetrically formulated for **co-NP-complete** problems. The proofs of all items follow from the closeness properties of these classes and from the transitivity of reductions.

1.10 Oracle machines and Turing reductions

We now define a much more general type of reductions that allow to use their “black box” in an arbitrary way. (Many-one reductions use it only once and give the same answer.) In programming terms, an oracle machine uses its oracle as an external subroutine.

For this purpose, let us define a variant of Turing machines called oracle Turing machines. These are machines that are given access to an “oracle” that can magically (and instantly) solve a certain problem. Let us start with reductions between languages, then an oracle is a language, that is, a function $\{0, 1\}^* \rightarrow \{0, 1\}$.

Definition 1.12. (Oracle Turing machines) An oracle Turing machine is a DTM $M^\bullet$ that has a special tape (an “oracle tape”) and special oracle states q_{query} , q_{answer} . The bullet $\bullet$ over it says that it has a “hole” where we can insert any oracle (any function $\{0, 1\}^* \rightarrow \{0, 1\}$).

To execute $M^\bullet$, one needs to use a specific oracle language $O \subseteq \{0, 1\}^*$ (the resulting construction — an oracle machine using a specific oracle — is denoted M^O). Whenever during the execution M enters the state q_{query} , the oracle instantly solves the problem, i.e., the contents y of the oracle tape is replaced by $O(y)$ and the machine transits to the state q_{answer} . This transition costs 1 step only when we define the running time of M^O .

We can now define Turing reductions (also called oracle reductions).

Definition 1.13 (polynomial-time Turing reduction). We say that a language W is polynomial-time Turing-reducible to a language S (denoted $S \leq_{T,p} W$) if there is a polynomial-time⁶ oracle DTM $R^\bullet$ such that

$$\forall x \in \{0, 1\}^* \quad x \in W \iff R^S(x) = 1.$$

Classes **NP** and **co-NP** are **not** closed under polynomial-time Turing reductions! (At least, we think so — we cannot prove it.)

⁶That is, working in polynomial time irrespectively of what oracle is given to it.

2 Lecture 2. Scribe: Dotan Danino

A Turing reduction is a powerful reduction because we can use the oracle to solve a problem and return the opposite answer if we choose to. Therefore, **SAT** is polynomial-time Turing-reducible to **UNSAT**, which is unlikely to be true for many-one polynomial-time reductions.

More than that, we can naturally generalize Turing reductions from languages (decision problems) to other types of problems (for example, search problems): they formalize the notion of using an external subroutine through an abstract interface — every programmer in, say, C++ is familiar with that notion.

Before doing that, let us define search problems.

2.1 Search Problems and Classes

A **search problem** is associated with a binary relation between inputs and (candidate) solutions:

$$R(x, w) = \text{“}w \text{ is a correct solution to } x\text{”}$$

Algorithm A **solves the search problem** associated with R if

$$\forall x \quad (\exists w \ R(x, w) \implies R(x, A(x)))$$

The most interesting search problems are those where we can verify (the correctness of) solutions efficiently.

Defining some search problems classes.

Remark 2.1. In the literature, they commonly denote the following classes as **FNP** and **FP**. We will not use this notation to avoid ambiguity (sometimes it is used also for different classes. . .).

Definition 2.1. (The search problem associated with) a relation R belongs to **Search-NP** if R is polynomially bounded and polynomial-time computable, and

$$\forall x \quad R \subseteq \{0, 1\}^n \times \{0, 1\}^{\leq p(n)}$$

Remark 2.2. Note that we can consider a binary relation⁷ R as a language of pairs

$$\{\langle x, w \rangle : R(x, w)\}.$$

Thus we can even write “ $R \in \mathbf{P}$ ”, but it does **not** mean that R is corresponding to a polynomial-time solvable search problem: it means that we can verify the correctness of a solution in polynomial time and it does not mean that we can find that fast a correct solution or even say whether it exists or not!!!

⁷Recall that $R(x, w)$ has the same meaning as $R(x, w) = \mathbf{True}$, $R(x, w) = 1$, and $(x, w) \in R$.

Like efficiently solvable **NP** problems are called **P** problems, there are also efficiently solvable **Search-NP** problems.

Definition 2.2. $R \in \text{Search-P}$ if it is in **Search-NP** and there exists a polynomial-time algorithm A that solves the search problem associated with R .

Search-NP vs. NP. **Search-NP** is a natural generalization of **NP**: instead of just asking *whether* a solution exists, we require *finding* it. Thus, while **NP** problems (languages) have a yes/no answer, **Search-NP** problems require producing an actual solution (witness, certificate) w that proves that x is indeed in the language.

A more formal explanation: while in a decision problem we are asking about the membership in a language, in a search problem we are asking about finding (correct) solutions for a binary relation.

2.2 Turing reductions between arbitrary problems (A formalistic definition)

As mentioned above, one can use (a generalization of) Turing reductions to reduce search problems to search problems, search problems to decision problems (this will be our usecase below), etc. Oracle machines can compute functions and they can use all types of oracles (not just languages, but any functions, and even more).

Definition 2.3. A problem⁸ W is **Turing-reducible** to a problem S ($W \rightarrow_T S$, $W \leq_T S$) if there is an oracle deterministic Turing machine $M^\bullet$ such that for every black box⁹ σ solving S (that is, satisfying the condition $\forall y \quad \sigma(y) \in \text{Answers}_S(y)$),

- $\forall x \quad M^\sigma(x) \in \text{Answers}_W(x)$,
- $\forall x \quad \text{time}_{M^\bullet}(x) \leq \text{poly}(|x|)$ (this should happen for the work of M with every possible oracle).

Here $\text{Answers}_Q(z)$ denotes the set of correct answers to the problem Q for the input z (if Q is a decision problem, this set contains just a single bit; if Q is a search problem, this set $\text{Answers}_Q(z) = \{w : Q(x, w) = 1\}$ contains all correct solutions to z , etc).

If M runs in polynomial time, then we say that this is a **polynomial-time Turing reduction**, and $W \rightarrow_T^p S$ (this is the only usecase in this course).

2.3 Levin's Reductions

Leonid Levin (recall the Cook–Levin theorem) in his seminal paper considered search problems, thus he needed reductions between them. These reductions are the default choice for reductions between search problems (search-to-search).

⁸Any type of a problem.

⁹Think about a function for now.

Definition 2.4. Levin's reduction (see Fig. 6) from W to S (where W, S are search problems, i.e., relations) is defined by a triplet of polynomial-time computable functions (f, g, h) (where informally f transfers inputs from W to S , h transfers solutions from W to S , and g transfers solutions backwards, from S to W). Levin defined them as $f, g, h : \{0, 1\}^* \rightarrow \{0, 1\}^*$, we will allow g and h to access the input (and not just a solution) as well, i.e., $g, h : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$. The three functions must satisfy the following two conditions¹⁰: for every x, z, u ,

1. $W(x, u) \iff S(f(x), h(x, u))$.
(The input of S is $f(x)$ and the witness is $h(x, u)$). This condition guarantees that x has a solution if and only if $f(x)$ has a solution, that is, f is a many-one reduction.)
2. $S(f(x), z) \iff W(x, g(x, z))$.
(That is, $g(x, z)$ is a solution for W if and only if z is a solution for S on input $f(x)$. This condition transfers solutions of S to W , but the first condition is still needed to guarantee that there exists some solution we could transfer!)

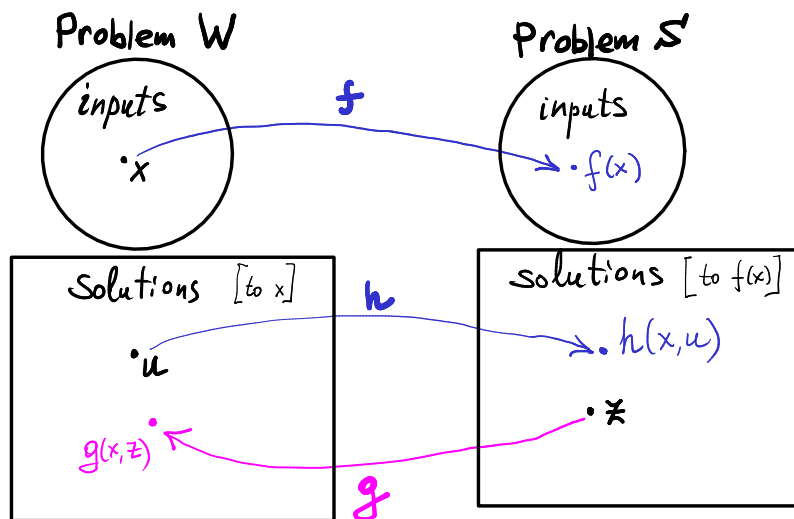


Figure 6: A diagram of Levin's reduction (input mapping f , solution mapping g, h)

Levin's reduction helps solving a search problem using a black box for another search problem just once and in a prespecified manner (apply f , solve S , then apply g to transfer the solution back to W), whereas Turing reductions allow to use this black box any number

¹⁰In the actual lecture there was a typo ($\Rightarrow$ instead of $\iff$). Such reductions would work as well, but **Search-NP** may not be closed under them without the first $\iff$.

of times and also can be used to solve a search problem using a black box for a decision problem.

2.4 An important example of Levin's reduction: Circuit-SAT to 3-SAT

This reduction will show us that once we prove **Search-NP**-completeness of a search version of **Circuit-SAT**, the same will hold for finding satisfying assignments of **3-SAT**¹¹. (Think of it as a step towards the goal of establishing **Search-NP**-completeness of the latter, and also as a step in a proof of the Cook–Levin theorem about **NP**-completeness of **3-SAT** as a language.)

In the **Circuit-SAT** problem (its search version), we are given a Boolean circuit C with n variables (input bits) $x_1, \dots, x_n$ and a single output. We need to find such bits $b_1, \dots, b_n$ that $C(b_1, \dots, b_n) = 1$. We formally define the corresponding relation

$$R_{\text{Circuit-SAT}}(C, b) = 1 \iff C(b) = 1,$$

where b represents the assignment $(x_1 = b_1, \dots, x_n = b_n)$.

To perform the reduction, let us denote the result of every internal gate of C as $y_{n+1}, y_{n+2}, \dots, y_m$ (identifying $y_1, \dots, y_n$ with $x_1, \dots, x_n$).

It is easy to write a small 3-CNF formula checking that the result of the Boolean operation in the gate is correct. For example, consider the following gate computing OR ($\vee$):

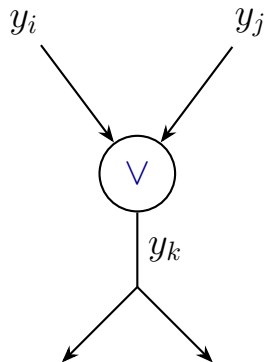


Figure 7: Example for a gate in the circuit

We want to ensure $y_k = y_i \vee y_j$. We add clauses to ensure that if this is not true, the 3-CNF formula will not be satisfied if the assignment has such unfortunate values for

¹¹**3-SAT** is the satisfiability problem for a formula in **3-CNF**, that is, a conjunction of disjunctions (**clauses**) containing at most three **literals** (i.e., variables or their negations).

y_i, y_j, y_k . For the OR operation these are

$$\begin{aligned} &(\overline{y_i} \vee y_k), \\ &(\overline{y_j} \vee y_k), \\ &(y_i \vee y_j \vee \overline{y_k}). \end{aligned}$$

It is easy to write such a set of clauses for every binary (and unary) operation, even a not so efficient truth table method (see the end of Lecture 3 for a reminder) would give at most 4 clauses (just list all the wrong assignments for y_i, y_j, y_k and write a separate 3-clause for each of them; the example above is a bit more efficient equivalent).

We do this for all gates thus writing the conjunction of all these clauses. Finally, we add a clause containing only y_m (the output gate) to enforce the circuit evaluate to 1.

Our reduction functions:

- $f(C)$ constructs the formula described above from the circuit C .
- $g(C, ab) = a$, where $a \in \{0, 1\}^n$ contains values for the x variables, and $b \in \{0, 1\}^{m-n}$ contains¹² the values computed in the internal gates y .
- $h(C, a)$ computes the values of y given x and outputs a bit string of size m given a bit string of size n .

The function f is polynomial-time because we add a constant number of (easily computable) clauses for each gate, h computes additional values for the assignment using circuit C , and g just cuts the assignment.

Lemma 2.1. Note that we can formulate even more than needed for the reduction of **Circuit-SAT** to **3-SAT**: given a circuit C with inputs $x_1, \dots, x_n$, we constructed in polynomial time a formula F in 3-CNF with variables $x_1, \dots, x_n, w_1, \dots, w_m$ such that

$$\forall x \in \{0, 1\}^n (C(x_1, \dots, x_n) = 1 \iff \exists w \in \{0, 1\}^m F(x_1, \dots, x_n, w_1, \dots, w_m) = 1).$$

2.5 A search-to-decision reduction for SAT and other NP-complete languages

Theorem 2.1. Consider the search problem associated with a relation $R \in \mathbf{Search-NP}$. If the language $L = \{x : \exists w (x, w) \in R\}$ is **NP**-complete, then $R \rightarrow_T^p L$.

¹²To follow Levin's definition strictly, the function g must ensure also that it does not "fix" the solution if ab is unsatisfying while a is satisfying. In order to do this, one needs to represent **Circuit-SAT** in a bit different (though "morally equivalent") way, we will not do it — left as an exercise for those interested in this kind of formalism.

Proof. We begin with showing such a search-to-decision reduction for **SAT**. The idea is to try setting a value for the first variable x_1 by querying the yes/no oracle and thus make our choice — follow the path (in one of the two possible directions $x_1 = 0$, $x_1 = 1$) that leads to a satisfiable formula (if there are two such paths, it does not matter which one to follow). Then do it with the second variable, etc.

Algorithm 2.1 (Solving **Search-SAT** using a (decision) **SAT** oracle).

Input: F .

```

1: if  $\text{SAT}(F) == 0$  then
2:   return False
3: else
4:   for  $i = 1$  to  $n$  do
5:     if  $\text{SAT}(F|_{x_i=1}) = 1$  then (see footnote13)
6:       Set  $x_i = 1$ 
7:       (Remove the clauses containing  $x_i$  without negation from  $F$ , remove  $\overline{x_i}$  from the
        others)
8:     else
9:       Set  $x_i = 0$ 
10:      (Remove the clauses containing  $\overline{x_i}$  from  $F$ , remove  $x_i$  from the others)
11:    end if
12:  end for
13:  return the values we set for  $(x_1, \dots, x_n)$ 
14: end if

```

This algorithm makes at most $2n + 1$ queries to the oracle, it follows a single branch in the brute-force search tree composed of all possible assignments to the n variables of F .

Now as R Levin-reduces to **Search-SAT** (we will prove its **Search-NP**-completeness later), **Search-SAT** Turing-reduces to **SAT** as we proved above, and **SAT** reduces to L using a many-one reduction (due to the assumption that L is **NP**-complete), and all these reductions are polynomial-time, we can compose them together to obtain a polynomial-time Turing reduction of the search problem R to the decision problem L . $\square$

¹³ $F|_{x_i=1}$ denotes F with x_i replaced (substituted) by 1 (**True**). Line 7 explains how to make this substitution and obtain again a formula in CNF.

Example 2.1.

$$\begin{aligned}
F &= (x_2 \vee \overline{x_3}) \wedge (x_1 \vee x_2) \wedge (\overline{x_1} \vee x_2 \vee x_3) \\
F|_{x_1=1} &= (x_2 \vee \overline{x_3}) \wedge (x_2 \vee x_3) \\
F|_{x_1=0} &= (x_2 \vee \overline{x_3}) \wedge (x_2)
\end{aligned}$$

2.6 Levin’s optimal algorithm for any NP problem (Universal Search)

Let us enumerate all Turing Machines as $M_1, M_2, \dots$ and build a table where columns are machines and rows are time steps. We run the machines in a “dovetailing” manner:

1. the 1st step of M_1
2. the 1st step of M_2
3. the 2nd step of M_1
4. the 2nd step of M_2
5. the 1st step of $M_3 \dots$

(This is not yet a specific enumeration — see its details below.)

If M_i stops before the specified step number and outputs a valid witness (we can check it in polynomial time as $R(x, M_i(x))$), we output $M_i(x)$ and stop.

One can use the following enumeration: Given the machine number i and the step number ℓ ,

$$T(i, \ell) = 2^i \cdot (2\ell + 1).$$

Note that i and ℓ are easily (and uniquely) recoverable from the value of T . Thus at “stage” $t = T(i, \ell)$ we recover i and ℓ and run M_i for ℓ steps.

Consider a specific machine M_k . Since k (the index of the correct machine) is a constant, our algorithm (Levin’s optimal algorithm) devotes a constant portion $1/2^k$ of its stages to running M_k (with different time bounds), and the overall time overhead of our algorithm compared to the time spent by M_k is polynomial.

(Fine details are left as an exercise.)

In particular, if there is an “interesting” set of inputs S that have solutions, and there is some (unknown to us!) machine M that solves R in polynomial time on this set of inputs, then Levin’s algorithm is also polynomial-time on this set S (in particular, if $\mathbf{P} = \mathbf{NP}$, then Levin’s algorithm is polynomial-time).

3 Lecture 3. Scribe: Talya Cohen, Dvir Opatowsky

3.1 Search-NP and Levin reductions — recap from the previous lecture

Search-NP. **Search-NP** is a natural generalization of **NP**: instead of just asking whether a solution exists, we require finding it. It is frequently denoted “**FNP**”, but this notation is also used for a different class, so we will not use it to avoid ambiguity. Thus, while **NP** problems (languages) have a yes/no answer, **Search-NP** problems require producing the actual solution (witness, certificate) y that proves that x is indeed in the language.

A more formal explanation: while in a *decision problem* we are asking about membership in a *language*, in a *search problem* we are asking about finding solutions for a *binary relation*.

We define the class **Search-NP** as the class of all polynomially bounded polynomial-time verifiable relations.

That is, a (search) problem in **Search-NP** is associated with a binary relation between inputs and (candidate) solutions such that

- for every input x of length n , any its solution y has length at most $p(n)$ for some polynomial p ,
- given x and y , it is possible to verify in polynomial time whether y is a valid solution for x .

To solve a search problem, given y , one has to output a valid solution for it. A couple of remarks are due before we remind a formal definition:

Remark 3.1. One can additionally require that if x has no solution, then the algorithm solving the problem must say it explicitly. However, given an algorithm that solves positive instances (those having a solution) correctly, it is easy to update it to handle this case: since R is verifiable in polynomial time, we can re-check whether our output is a valid solution, and say “no” if it’s not.

Remark 3.2. In the literature there are three formulations for “polynomially bounded”:

1. solutions of length at most $p(n)$,
2. solutions of length exactly $p(n)$,
3. solutions of any length, but if x has a solution, then one of its solutions must be of length at most $p(n)$ (note that in this case the algorithm verifying R must work in time polynomial in both the input length and the candidate solution length).

It is easy to see that they all lead to the same definition of search problems in **Search-NP** and **Search-P** (that is, to switch between 1–2–3, one needs to change R and any algorithm solving it only a little bit).

Let us remind now a formal definition:

Definition 3.1 (Search-NP). In a search problem, we are given a binary relation $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$; an algorithm solves this search problem if for every $x \in \{0, 1\}^*$ it outputs $y \in \{0, 1\}^*$ such that $R(x, y) = 1$.

A problem associated with R is in **Search-NP** if there is a polynomial p and a polynomial-time algorithm V such that

- $R \subseteq \{0, 1\}^* \times \{0, 1\}^{\leq p(n)}$ (solutions are polynomially bounded),

- $\forall x, y \in \{0, 1\}^* V(x, y) = R(x, y)$ (solutions are polynomial-time verifiable).

Remark 3.3. Don't be misguided by notation $R \in \mathbf{P}$. Yes, R is a relation (and a search problem is associated with it), but it can be also viewed as a language $\mathcal{R}$ of pairs

$$\langle x, y \rangle \in \mathcal{R} \iff R(x, y) = 1$$

While R is easy as a language (we can check in polynomial time whether y is a correct solution), it may be difficult as a search problem (given x , find y).

We can associate a *different* language with R , for which, as we learned a week ago, sometimes the decision problem may be polynomial-time equivalent to the search problem R , namely,

$$L(R) = \{x \in \{0, 1\}^* : \exists y \in \{0, 1\}^* R(x, y) = 1\}$$

(the language of those inputs that have correct solutions).

Levin's reductions. Just as decision problems can be related to each other via many-one reductions, we can also relate search problems using a similar concept. Let W_1 and W_2 be two search problems. To say that W_1 reduces to W_2 , we use Levin's reductions.

A Levin reduction from W_1 to W_2 consists of three polynomial-time computable functions f, g, h that satisfy the following conditions:

1. For every input x and every (candidate) solution y to x for the problem W_1 , y is a correct solution if and only if the value $h(x, y)$ is a valid solution for $f(x)$ for the problem W_2 :

$$W_1(x, y) \iff W_2(f(x), h(x, y)).$$

2. For every input x and every solution z to $f(x)$ for the problem W_2 , z is a valid solution if and only if the value $g(x, z)$ is a valid solution for x in problem W_1 :

$$W_1(x, g(x, z)) \iff W_2(f(x), z).$$

If such functions f, g, h exist, we say that W_1 **Levin-reduces** to W_2 , and we write:

$$W_1 \leq_L W_2.$$

Levin's reductions of search problems extend the more popular notion of many-one polynomial-time reductions of decision problems (in fact, f is a many-one reduction of the corresponding languages showing $L(W_1) \leq^P W_2$).

Remark 3.4. Note that Levin's reduction not only transforms inputs but also maps solutions back to the original domain using the function g , ensuring that solving W_2 efficiently implies that W_1 can also be solved efficiently. The third function h is needed for formulating the first correctness condition: without this condition, we could map x that has solutions into $f(x)$ that has no solutions, and then g (and the second condition) would not help us as g needs a correct W_2 -solution to give us a correct W_1 -solution.

3.2 The Cook–Levin Theorem: a proof that transforms Turing machines into circuits

In this lecture, we study an alternative proof of the **NP**-completeness of 3-SAT (the *Cook–Levin Theorem*).

Theorem 3.1 (S.A.Cook, L.A.Levin). 3-SAT is **NP**-complete.

In the previous lecture we learned one ingredient of it: a polynomial-time reduction from **Circuit-SAT** to 3-SAT (both as search problems and a decision problems). It remains to prove the **NP**-completeness of **Circuit-SAT**.

Theorem 3.2. **Circuit-SAT** is **NP**-complete (and the corresponding search problem is **Search-NP**-complete).

Proof. To prove **NP**-completeness, we need to show two statements:

1. **Circuit-SAT** $\in$ **NP**,
2. **Circuit-SAT** is **NP**-hard.

The first statement is easy: Given a Boolean circuit C with variables $x_1, \dots, x_n$, if a candidate assignment of Boolean values to these variables is given, we can evaluate the circuit gate by gate — thus in polynomial time. If the output is 1, we accept, otherwise we reject. It shows that **Circuit-SAT** $\in$ **NP** as a language and its search problem (for the relation $\{(C, b) : C(b) = 1\}$, where C is a circuit, $b \in \{0, 1\}^n$, and n is the number of variables in C) is in **Search-NP**.

The second statement is harder. To prove that **Circuit-SAT** is **NP**-hard, we now show how to reduce *any* **NP**-language L to **Circuit-SAT** in polynomial time, that is, there exists a polynomial-time computable function f such that:

$$x \in L \iff f(x) \in \text{Circuit-SAT}.$$

Let $L \in$ **NP**. By the definition of **NP** (we have seen the equivalence of two definitions in the first lecture), there exists a polynomial $p(n)$ and a polynomial-time **deterministic** Turing machine M such that

$$x \in L \iff \exists y \ M(\langle x, y \rangle) = 1.$$

Our goal is to build a polynomial-time function f that given x outputs a circuit C_x such that

$$\exists y \ M(\langle x, y \rangle) = 1 \iff \exists z \ C_x(z) = 1 \text{ (i.e., } C_x \in \text{Circuit-SAT)}.$$

We will do even more:

$$\forall y \ M(\langle x, y \rangle) = 1 \iff C_x(y) = 1.$$

Note that since f is polynomial-time, the circuit C_x will be inevitable of polynomial size (in $|x|$).

It suffices to prove the following lemma (see its proof below):

Lemma 3.1. Given a DTM M and a number $t \in \mathbb{N}$, we can construct in time polynomial in $|\langle M \rangle|$ and t a Boolean circuit C such that

$$\forall u \ (C(u) = 1 \iff T(u) \text{ accepts in at most } t \text{ steps}).$$

Assuming this lemma (to be proved shortly), we can complete the proof of our theorem. Indeed, let M be our machine and q be a polynomial upper-bounding its running time. Given x , let f do the following:

- Construct the circuit C as in Lemma 3.1 applied to the machine M and $t = q(|x|)$ (assume w.l.o.g. that the total length of $\langle x, y \rangle$ is upper-bounded by the running time — otherwise M has no time even to read them). Note that this circuit C depends on variables x and y , because they are parts of the input to M .
- Plug in to C the values for the bits of x that f is given as its argument. Output the resulting circuit — we denote it C_x , and it depends on y only.

By this lemma, for every y

$$C_x(y) = 1 \iff M(\langle x, y \rangle) = 1. \quad (1)$$

It follows that

$$x \in L \iff \exists y \in \{0, 1\}^{p(|x|)} M(x, y) = 1 \iff \exists y C_x(y) = 1 \iff C_x \in \text{Circuit-SAT} \iff f(x) \in \text{Circuit-SAT},$$

therefore f is a many-one reduction (obviously running in polynomial time) from L to **Circuit-SAT**. This shows that $L \leq^p \text{Circuit-SAT}$ and complete our proof of the **NP**-hardness part.

Note that Levin's reduction of search problems is straightforward: f is the same function, and g and h are simply identity functions, the conditions are satisfied due to (1). $\square$

Proof of Lemma 3.1. To convert a Turing machine computation that takes a specified number of steps (in our application this number is polynomial in the input size), note that it suffices to consider only t cells: the machine has no time to come to further cells.

The main nodes of our circuit. Let us build a table $t[i, j]$ representing the machine's tape at all steps of the run of this machine. Its rows (i) will correspond to time (the step number), its columns (j) will correspond to specific cells of the tape. That is, the cell (i, j) represents what is in the cell number j at step i . This is $O(1)$ bits (if the tape alphabet is $\{0, 1, \triangleright, \sqcup\}$, it's just two bits per (i, j)).

In addition, we will need a similar table of bits for the head position: $h[i, j] = 1$ if and only if at step i the head is observing the cell j .

Eventually, we need the state of the machine at step i denoted by $s[i]$. Since the number of state is finite, for each i this $s[i]$ contains $O(1)$ bits.

We associate a node of our circuit with each of these bits.

The inputs of our circuit. The inputs of our circuit correspond to what is written on the tape at step 0 (at the start). Some of them are replaced by constants (for $\triangleright$ and spaces), the remaining ones correspond to the input of the machine.

Simulating the machine's computation using our circuit. For each (non-input) node mentioned above we build a subcircuit that computes it from the nodes of the previous level (row). This is because the configuration of a DTM at step i is fully determined by its configuration at step $i - 1$.

For specific i, j , the following data is sufficient to compute $t[i, j]$ and $h[i, j]$:

- the state $s[i - 1]$ (which is $O(1)$ bits),
- whether the head is nearby $h[i - 1, j - 1], h[i, j], h[i, j + 1]$ (which is 3 bits),
- what is written in nearby cells $t[i - 1, j - 1], t[i, j], t[i, j + 1]$ (which is $O(1)$ bits).

This is enough due to the [locality](#) of computation of DTM: if the head is not in the cell j or a neighboring cell, then the machine has no chance to change $h[i, j]$ or $t[i, j]$ at this particular step: they will stay exactly the same as they were at step $i - 1$ (the head moves only one cell at a time and all the changes are localized at its current location).

In order to build a subcircuit $\text{CellCircuit}_{i,j}$ computing $t[i, j]$ and $h[i, j]$ from this data, we need just $O(1)$ nodes, because it has $O(1)$ inputs and $O(1)$ outputs! We can build it constructively from the transition function $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow, \cdot\}$ of our machine M . The easiest (though not the most efficient) way of doing it is to use the truth-table technique (see a reminder below). Even though this method is exponential in the number of inputs, its complexity is a constant, because the number of its inputs is a constant!

It remains to compute the next state $s[i]$. We can attempt to do it for each cell j by a similarly constructed subcircuit $\text{StateCircuit}_{i,j}$ in the hope that the head is there (i.e., $h[i, j] = 1$). It is also $O(1)$ size and can be produced in time $O(1)$. Now to compute $s[i]$ we use the result of $\text{StateCircuit}_{i,j}$ for the value of j where $h[i, j] = 1$ and ignore it for all other cells, that is, we make a conjunction (AND, $\wedge$) of each its output k (the k -th bit of the state) with $h[i, j]$ to get an intermediate bit $v[i, j, k] = \text{StateCircuit}_{i,j}[k] \wedge h[i, j]$ and then connect by disjunctions (OR, $\vee$) all the resulting bits to compute the k -th bit of $s[i]$ as $\vee_j h[i, j, k]$.

It is easy to see that we can compute in polynomial time the circuit connecting all the subcircuits we constructed so far for all $0 \leq i, j \leq q(|u|)$. Its size is $O(q(|u|)^2)$ and in any case it is polynomial size because it is computed in polynomial time.

The output. So far our circuit lacks the output. What are we interested in? Whether our machine accepts! That is, the (one-bit) output should indicate whether it came to the accepting state at the last step $j = q(|u|)$.

Remark 3.5. A remark on fine details of the complexity this construction is due. While it is obvious that it is polynomial (quadratic) in the number of steps, the dependence on the machine size is less obvious.

One can prove (and in the latest version of the Automata course we did) that every DTM can be converted into one that uses the four-letter tape alphabet $\{0, 1, \triangleright, \sqcup\}$, it is not hard to see.

However, the number of states can be arbitrary. To argue that the size of our subcircuits are also polynomial also *in the size of the description of our DTM* (which we do *not* use for our **NP**-completeness result), one should either stop using the truth-table technique and convert each instruction of our machine into the corresponding part of our subcircuits $\text{CellCircuit}_{i,j}$ and $\text{StateCircuit}_{i,j}$, or one can encode the state in binary (thus $\text{StateCircuit}_{i,j}$ will have a logarithmic number of variables, so even the truth-table method will produce a polynomial-size circuit), in which case one needs compute the (global) output bit using a small subcircuit checking that these bits indeed represent the accepting state.

Summary: Intuitive explanation of the whole proof.

1. Every **NP** language can be recognized by some polynomial-time Turing machine M working on an input x and a certificate y .
2. The computation of $M(x, y)$ can be simulated by a Boolean circuit C_x with variables for y .
3. Therefore, the problem of checking whether there exists y such that $M(x, y) = 1$ is equivalent to checking whether C_x has a satisfying assignment (**Circuit-SAT**).
4. Finally, since any circuit can be translated into an equivalent 3-CNF formula (**CIRCUIT-SAT** $\leq^p$ **3-SAT**), we arrive at **3-SAT** problem.

Reminder: A truth-table technique. Let f be a Boolean function over variables $x_1, \dots, x_k$. We can construct a Boolean circuit for f (and even a Boolean formula) based on its truth table. For each assignment $(x_1, \dots, x_k)$ such that $f(x_1, \dots, x_k) = 1$, we create a conjunction (AND) term that represents this assignment: if a variable x_i takes the value 0, we use $\neg x_i$, and if it takes the value 1, we use x_i as is. Finally, we connect all these conjunctions with disjunctions (ORs).

Consider an example:

x_1	$\dots$	x_{k-1}	x_k	$f(x_1, \dots, x_k)$
0	$\dots$	0	0	1
0	$\dots$	0	1	1
0	$\dots$	1	1	0
$\vdots$			$\vdots$	$\vdots$
1	$\dots$	1	1	1

From the rows, where $f(x_1, \dots, x_k) = 1$, we build the formula:

$(\neg x_1 \wedge \neg x_2 \wedge \dots \wedge \neg x_k) \vee$

$(\neg x_1 \wedge \neg x_2 \wedge \dots \wedge x_k) \vee$

$\dots$

$(x_1 \wedge x_2 \wedge \dots \wedge x_k).$

This is a formula in DNF computed by the corresponding circuit containing $O(2^k)$

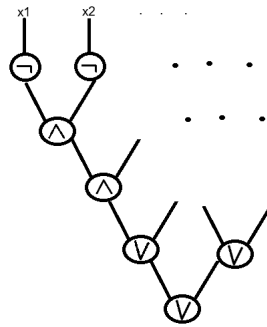


Figure 8: A circuit for DNF

nodes. If our function outputs not just a single bit, but a string of ℓ bits, we repeat this construction for each output bit and get $O(2^k \cdot \ell)$ nodes.

4 Lecture 4. Scribe: Hillel Ohayon

4.1 NP-Intermediate Languages

In the previous lectures we studied **NP**, its subset **P**, and the notion of **NP**-completeness, thus another subset containing **NP**-complete languages. A natural question arises:

Must every language in $\mathbf{NP} \setminus \mathbf{P}$ be **NP**-complete?

(Un)surprisingly, the answer is likely to be **no**, that is, such languages are likely to exist.

Obviously, we cannot (currently) prove unconditionally that they exist: if $\mathbf{P} = \mathbf{NP}$, they cannot exist. We will prove their existence under the assumption $\mathbf{P} \neq \mathbf{NP}$ (thus in fact they exist *if and only if* $\mathbf{P} \neq \mathbf{NP}$).

Definition 4.1 (**NP**-intermediate). A language L is **NP-intermediate** if:

$$L \in \mathbf{NP}, \quad L \notin \mathbf{P}, \quad L \text{ is not } \mathbf{NP}\text{-complete}.$$

This result is due to Ladner:

Theorem 4.1 (Ladner, 1975). If $\mathbf{P} \neq \mathbf{NP}$ then there exists a language $L \in \mathbf{NP} \setminus \mathbf{P}$ such that

L is not $\mathbf{NP}$ -complete (under many-one polynomial-time reductions).

The next sections contain the proof of this result. Informally speaking, we construct such a language by padding $\mathbf{SAT}$ (that is, adding some garbage to our formulas, thus extending the input length) in a very carefully controlled way. The key idea is to choose a padding function that grows fast enough so that the new language is not $\mathbf{NP}$ -complete anymore, but slow enough so that it is still not in $\mathbf{P}$.

4.1.1 Defining the Padded Language $\mathbf{SAT}_H$

Let ψ be a Boolean formula (in CNF, but it does not matter here) and let

$$n = |\psi|$$

be its length. Define the padded language:

$$\mathbf{SAT}_H = \left\{ \psi 0 1^{n^{H(n)}} \mid \psi \in \mathbf{SAT} \right\},$$

where the function $H : \mathbb{N} \rightarrow \mathbb{N}$ (to be defined later) controls the padding length.

It is very easy to extract ψ from its padded version: read the input right-to-left until the first zero occurs. Likewise, given $H(n)$, it is very easy to write the padded version of ψ .

Remark 4.1. Note that if one takes $H(n) = n$, that is, the padding length is n^n , then $\mathbf{SAT}_H \in \mathbf{P}$, because the input length becomes so large compared to its essential part (ψ) that it gives us a lot of time to solve it even by brute force while still remaining within the time bound $O(|\text{input}|^2)$.

4.1.2 Defining the Function H

(*Turing numbers: reminder.*) Recall that there is a numbering of all deterministic Turing machines

$$M_1, M_2, M_3, \dots$$

such that each machine appears infinitely many time in this sequence.

For each $n \in \mathbb{N}$, we define:

$$H(n) = \begin{cases} \min_{i < \log \log n} \{i \mid \forall x \in \{0, 1\}^{\leq \log n} (M_i(x) = \mathbf{SAT}_H(x) \wedge \text{time}_{M_i}(x) \leq i \cdot |x|^i)\}, & \text{if this set is nonempty,} \\ \log \log n, & \text{otherwise.} \end{cases}$$

This makes H grow just fast enough to prevent its $\mathbf{NP}$ -completeness while also preventing *any* polynomial-time algorithm from deciding $\mathbf{SAT}_H$ on all inputs.

Remark 4.2. We defined SAT_H using H and H using SAT_H , so one needs to check that this doubly-recursive definition is correct.

It is so because the definition of $H(n)$ uses SAT_H on inputs of length at most $\log n$ and the definition of SAT_H uses H for its argument not larger than the length of the input to SAT_H . Thus the recursion indeed converges.

Exercise 4.1. Show that H is computable in $O(n^4)$ time (any other polynomial will also do).

Exercise 4.1 guarantees that $\text{SAT}_H \in \mathbf{NP}$.

The function H can demonstrate two possible behaviors:

$$(1) H(n) = O(1), \quad (2) H(n) \text{ is unbounded.}$$

Exercise 4.2. Prove that if $H(n)$ is unbounded, then $H(n) \xrightarrow{n \rightarrow \infty} \infty$.

We do not know yet which case really takes place, so we will analyze both.

Case 1: $H(n)$ is bounded. Assume $H(n) \leq C$ for all n .

Then the padding is always of length at most n^{C+1} , that is, polynomial length, so

$$\text{SAT} \leq^p \text{SAT}_H$$

(the reduction simply computes $H(n)$ and adds the required padding to its input). In this case SAT_H remains $\mathbf{NP}$ -complete. Therefore, $\text{SAT}_H \notin \mathbf{P}$, because $\text{SAT}_H \in \mathbf{P}$ would contradict the assumption $\mathbf{P} \neq \mathbf{NP}$.

However, it means that $H(n)$ is unbounded: if it is bounded by a constant C for every n — and thus for every input length, then a single machine M_C would solve SAT_H in polynomial time (namely, $C|x|^C$) on all input lengths¹⁴. It contradicts the condition of Case (1)!

Therefore, Case (1) is impossible.

Case 2: $H(n)$ is unbounded. Assume now that $H(n)$ is unbounded.

Similarly to Case (1), it already gives us that $\text{SAT}_H \notin \mathbf{P}$, since $\text{SAT}_H \in \mathbf{P}$ would imply that there exists some machine M_C deciding SAT_H in time $C|x|^C$ on all the inputs, and then $H(n)$ would be bounded by this constant C , contradicting now the condition of our current case.

It remains to prove that SAT_H is not $\mathbf{NP}$ -complete.

¹⁴Think why H cannot alternate between two constants C and D , it will help you also to solve one of the exercises.

Assume towards contradiction that it is **NP**-complete. Thus, in particular,

$$\text{SAT} \leq^p \text{SAT}_H$$

via some polynomial-time reduction R running in time

$$i^* \cdot n^{i^*}$$

for some constant i^* .

Let ψ be an input formula of length $m = |\psi|$. The output of the reduction can be split as

$$R(\psi) = \varphi 0 1^{H(n)}, \quad \text{where } n = |\varphi|$$

for some formula φ (of course, depending on ψ), otherwise (if the output is syntactically incorrect) ψ is trivially unsatisfiable by the definition of a many-one reduction.

Since R runs in time $i^* \cdot n^{i^*}$, it cannot write more than this many bits, that is,

$$|R(\psi)| \leq i^* \cdot m^{i^*}.$$

Thus:

$$|\varphi| + 1 + n^{H(n)} \leq i^* \cdot m^{i^*}.$$

The length of the padding part $1^{n^{H(n)}}$ is, of course, bounded by the same polynomial bound. Therefore, as H grows (it is unbounded!), the padding will leave only a small share of the string for the “interesting” part (φ). Therefore, for sufficiently large m the value of H will grow to more than $10i^*$, and we will have

$$i^* \cdot m^{i^*} > n^{H(n)} > n^{10i^*},$$

that is,

$$|\varphi| = n \leq \text{const} \cdot m^{i^*/10} = \text{const} \cdot |\psi|^{i^*/10},$$

(The part of the language with smaller values of m is a constant-size language, so it can be solved in $O(1)$ time by brute force, and we can ignore it.)

Recursive shrinking. The formula part φ is the most interesting component of the output. Here is why: since padding is determined by the length of φ , we can now apply R recursively on φ :

$$\psi_0 = \psi, \quad \psi_1 = \varphi, \quad \psi_2 = R(\psi_1), \quad \psi_3 = R(\psi_2), \dots$$

After (much fewer than) $O(\log m)$ iterations, the length of our formula becomes a constant. At this point the **SAT** instance is tiny and can be evaluated directly by brute force.

Therefore, coming back from this recursion we can determine the satisfiability of the original input ψ in polynomial time. Thus we obtain:

$$\text{SAT} \in \mathbf{P},$$

contradicting $\mathbf{P} \neq \mathbf{NP}$. Thus our assumption was false and we proved that

$$\boxed{\text{SAT}_H \text{ is not } \mathbf{NP}\text{-complete.}}$$

Conclusion. To summarize what has happened,

- We defined a function H computable in polynomial time and a language $\text{SAT}_H \in \mathbf{NP}$.
- Case (1): If $H(n)$ is bounded, then we get a contradiction, so we ruled out this case.
- Case (2): If $H(n)$ is unbounded (and growing), then
 - $\text{SAT}_H \notin \mathbf{P}$ (as we promised);
 - moreover, assuming it is $\mathbf{NP}$ -complete leads to a shrinking recursion that gives a polynomial-time algorithm for SAT , again contradicting $\mathbf{P} \neq \mathbf{NP}$ (thus SAT_H is not $\mathbf{NP}$ -complete, as we promised).

Thus we proved Theorem 4.1.

4.2 Polynomial Hierarchy (PH)

The polynomial hierarchy generalizes the classes $\mathbf{NP}$ and $\mathbf{co-NP}$ by allowing *multiple layers* of nondeterminism and oracle access. Informally, one may imagine increasingly powerful “rings” of complexity classes:

$$\mathbf{P} \subseteq \mathbf{NP}, \mathbf{co-NP} \subseteq \Sigma_2^P, \Pi_2^P \subseteq \dots$$

We will give two definitions of it:

- through oracles (like $\mathbf{NP}^{\mathbf{co-NP}^{\mathbf{NP}^{\dots}}}$) and
- through quantifiers (like $x \in L \iff \exists w_1 \forall w_2 \exists w_3 \dots R(x, w_1, \dots)$).

Oracle-based classes. For two classes $\mathcal{C}$ and $\mathcal{D}$, one can try to form the class $\mathcal{C}^{\mathcal{D}}$ as the class of languages accepted by machines like those defining the class $\mathcal{C}$ (but with an oracle!) given access to languages from class $\mathcal{D}$.

It does not give a formal definition of $\mathcal{C}^{\mathcal{D}}$, because the same class of languages $\mathcal{C}$ can be defined using different types of computational devices with different types of oracle access. (Even $\mathbf{NP}$ can be defined both using DTMs and NTMs, luckily it does not make a problem, but there are hundreds of other complexity classes!)

Therefore, it is better to define $\mathbf{P}^{\mathcal{D}}$ and $\mathbf{NP}^{\mathcal{D}}$ for a class $\mathcal{D}$ explicitly (even that may require additional comments regarding the class $\mathcal{D}$, see Remark 4.4 below).

Oracle-based definition of PH. Given any language B , we denote by $\mathbf{NP}^B$ the class of languages decidable by a **nondeterministic polynomial-time oracle Turing machine** equipped with the oracle B . That is,

$$\mathbf{NP}^B = \{ L(M^B) \mid M^\bullet \text{ is a nondeterministic polynomial-time oracle machine} \}.$$

(A nondeterministic oracle TM is defined similarly to deterministic oracle TM, just with nondeterministic choice.)

Remark 4.3. If M^B is such a machine with an oracle, then we can easily convert $M^\bullet$ into $M_1^\bullet$ that will accept the same language given the oracle $\bar{B}$: it is the same machine but with **all oracle answers inverted** (it works as M , asks the oracle as M , negates its answer by its own instruction, and continues as M). Thus access to an oracle and to its complement does not change the power of $\mathbf{NP}^B$ and similar classes, and $\mathbf{NP}^{\mathcal{D}} = \mathbf{NP}^{\mathbf{co-D}}$ for every class $\mathcal{D}$.

Using the oracle access recursively for a constant number of steps yields the (first definition) of the polynomial hierarchy.

Definition 4.2 (Polynomial Hierarchy). Define

$$\begin{aligned} \Sigma_0^P &= \Pi_0^P = \mathbf{P}, \\ \Sigma_1^P &= \mathbf{NP}, \quad \Pi_1^P = \mathbf{co-NP}, \end{aligned}$$

and inductively:

$$\Sigma_{k+1}^P = \mathbf{NP}^{\Sigma_k^P}, \quad \Pi_{k+1}^P = \mathbf{co-NP}^{\Sigma_{k+1}^P} (= \mathbf{co-NP}^{\Sigma_k^P}).$$

One can define also additional classes $\Delta_{k+1}^P = \mathbf{P}^{\Sigma_k^P}$.

The union of all the classes of this hierarchy¹⁵ is denoted

$$\mathbf{PH} = \bigcup_{k \geq 0} \Sigma_k^P = \bigcup_{k \geq 0} \Delta_k^P.$$

It can be easily seen that the whole i -th level of the hierarchy is included in each class of the higher levels (to see this fact it suffices to call the oracle just once).

Thus

$$\Sigma_1^P = \mathbf{NP}, \quad \Pi_1^P = \mathbf{co-NP}, \quad \Sigma_2^P = \mathbf{NP}^{\mathbf{co-NP}} (= \mathbf{NP}^{\mathbf{NP}}), \quad \Pi_2^P = \mathbf{co-(NP}^{\mathbf{co-NP}}), \text{ etc.}$$

Remark 4.4. What if we want to use two or more different oracles $B_1, B_2, \dots$ of the same class $\mathcal{D}$ in the same machine? If $\mathcal{D}$ has complete problems (under many-one polynomial-time reductions), it is not a problem: we can use a single $\mathcal{D}$ -complete oracle D , and to query B_i for $q \in ? B_i$ we simply reduce this question to some $f(q) \in ? D$ and then ask D .

However, if $\mathcal{D}$ is not known to contain $\mathcal{D}$ -complete problems, further clarifications would be needed. Fortunately, Σ_i^P and Π_i^P do have complete problems, as we will see later.

¹⁵Not defined in the actual lecture.

An example language in $\mathbf{NP}^{\mathbf{NP}}$. Consider the language

$$T = \{C \mid \exists x \forall y \neq x C(x, y) = 1\},$$

where C is a Boolean circuit with $2n$ input bits, and $x, y \in \{0, 1\}^n$.

(Imagine a tournament where $C(x, y) = 1$ means that player x wins its home game against player y . The results are thus implicitly — and quite *succinctly* — described by this circuit. Then $C \in T$ if and only if there is a “universal winner” that wins its home game against everyone else.)

A nondeterministic oracle machine for T works as follows:

- On input C , nondeterministically guess $x \in \{0, 1\}^n$.
- Query an oracle for the language

$$B = \{\langle C, x \rangle : \exists y (y \neq x \wedge C(x, y) = 0)\}.$$

- If the oracle says “yes” (meaning a bad y exists), reject.
- Otherwise accept.

Since $B \in \mathbf{NP}$ (evaluating a circuit on a specific input (x, y) is an easy task), we conclude that

$$T \in \mathbf{NP}^{\mathbf{NP}} = \Sigma_2^P.$$

Quantifier characterization. Every Σ and Π class of this hierarchy can be described equivalently via a bounded number of alternating quantifiers¹⁶.

Theorem. A language L is in Σ_k^P if and only if there exists a polynomially bounded and polynomial-time computable¹⁷ relation R such that

$$x \in L \iff \exists y_1 \forall y_2 \exists y_3 \cdots Q_k y_k R(x, y_1, \dots, y_k),$$

where $Q_k = \exists$ if k is odd and $Q_k = \forall$ if k is even. Similarly, $L \in \Pi_k^P$ if the first quantifier is $\forall$ (then Q_k corresponds to the opposite parity).

¹⁶This theorem has not been formulated in this actual lecture, it will be a corollary of what we started to prove in the lecture.

¹⁷Recall that an n -ary relation is polynomial-time computable if the corresponding language of n -tuples is in $\mathbf{P}$.

Computation-tree intuition for the proof of equivalence of the two definitions.

We are going to prove the equivalence step by step, a building brick will be

$$L \in \Sigma_k^p \iff \text{there is } R \in \Pi_{k-1}^p \text{ such that}$$

$$x \in L \iff \exists y R(x, y).$$

Recall that the run (all the possibilities of a run!) of a nondeterministic machine (even with an oracle!) on a specific input x can be described as a computation tree where paths correspond to sequences of nondeterministic choices (or simply bit strings, which correspond to witnesses in the alternative definition of **NP**). An input is accepted if and only if at least one root-to-leaf path ends in acceptance. For polynomial-time machines we can assume that all the paths have the same length and the deterministic part of the computation (so-to-say, checking the witness) is done in the leaf.

Given a path in this tree, one can run deterministically in polynomial time the machine using the choices made on this path. However, how could we simulate querying the oracle if we do not have this oracle at hand?

If $B \in \Sigma_{k-1}^p$, then each such oracle query corresponds to another nondeterministic machine with an oracle from the $(k-2)$ -th level (one level lower). Making a step of mathematical induction¹⁸ $k \mapsto k-1$ we can even characterize it with a quantifier: $b \in B \iff \exists a (a, b) \in S$ for some $S \in \Pi_{k-1}^p$.

Thus if someone wants to persuade us that $q \in B$, they can give us a witness to S , then we could verify it in Π_{k-2}^p .

If someone wants to persuade us that $q \notin B$, they can simply say that the answer is as stated, then we can check it ourselves in Π_{k-1}^p . The point is that in both cases we can do it using an oracle from a Π -class (we will give our OK if some query *does* belong to a language in a Π -class). In the next lecture we will show that we will merge all such queries in a *single* Π -style computation (recall the notion of a co-nondeterministic machine) computing their conjunction (“all checks went OK”), therefore proving the following theorem:

Theorem. $L \in \Sigma_k^p \iff$ there is a polynomially bounded $R \in \Pi_{k-1}^p$ such that

$$x \in L \iff \exists y R(x, y).$$

We defer the details of its formal proof to the next lecture. Note that by considering the complement one can obtain a similarly looking statement for Π_k^p and apply these statements inductively (alternating between the two) to get a k -quantifier characterization of Σ_k^p . We will talk about this in the next lecture.

¹⁸The induction base is **NP**, and it was proved in earlier lectures.

5 Lecture 5. Scribe: Talya Cohen

5.1 The Polynomial Hierarchy, continued

In the previous lecture we started to study the polynomial hierarchy.

The Polynomial Hierarchy generalizes the classes **NP** and **co-NP** (and **P**) using oracle machines. It is defined inductively based on the availability of oracles from lower levels.

Definition 5.1 (Classes of the Polynomial Hierarchy). The classes are defined inductively as follows:

Level 0 (Base Case):

$$\Sigma_0^p = \Pi_0^p = \Delta_0^p = \mathbf{P}$$

Level 1:

$$\begin{aligned}\Sigma_1^p &= \mathbf{NP} \\ \Pi_1^p &= \mathbf{co-NP} \\ \Delta_1^p &= \mathbf{P}\end{aligned}$$

Level k (for $k \geq 2$):

$$\begin{aligned}\Sigma_k^p &= \mathbf{NP}^{\Pi_{k-1}^p} = \mathbf{NP}^{\Sigma_{k-1}^p} \\ \Pi_k^p &= \mathbf{co-}\Sigma_k^p \\ \Delta_k^p &= \mathbf{P}^{\Sigma_{k-1}^p}\end{aligned}$$

The union of the entire hierarchy is denoted by

$$\mathbf{PH} = \bigcup_{k \geq 0} \Sigma_k^p (= \bigcup_{k \geq 0} \Pi_k^p).$$

Note that the class **PH** defined above is a class of languages, and the **Polynomial Hierarchy** is a set of classes. It may sound a bit ambiguous, but these are standard terms in computational complexity.

5.2 Oracle Turing Machines: recap

Before proving the main characterization theorem, we formally define the computational model used in the hierarchy.

Definition 5.2 (Oracle Turing Machine). An **Oracle Turing Machine (OTM)** $M^\bullet$ is a Turing machine equipped with a special **query tape** and three special states: q_{query} , q_{yes} , and q_{no} . To execute $M^\bullet$ with a specific oracle $O \subseteq \{0, 1\}^*$ (denoted as M^O):

- Whenever M writes a string y on the query tape and enters the state q_{query} , the oracle instantly checks membership in O .

- After this step, the machine transitions to q_{yes} if $y \in O$, or to q_{no} if $y \notin O$.
- This query step counts as a single computational step.

The above definition can be stated for a deterministic Turing machine, but not only.

Definition 5.3. A **Nondeterministic Oracle Turing Machine (NOTM)** is defined similarly, but the base machine M is nondeterministic. The computation of an NOTM N^O on input x can be visualized as a tree, where each branch represents a nondeterministic choice (the existential quantifier $\exists$).

A **co-nondeterministic Oracle Turing Machine (coNOTM)** is defined similarly, but the base machine M is nondeterministic. The computation of a coNOTM N^O on input x can be visualized as a tree, where each branch represents a co-nondeterministic choice (the universal quantifier $\forall$).

Therefore, an input is accepted by a NOTM if and only if this machine comes to the accepting state *at least in one* leaf (there *exists* an accepting path).

Also, an input is accepted by a coNOTM if and only if this machine comes to the accepting state *in every* leaf (*all* paths are accepting).

5.3 Quantifier Characterization of the Polynomial Hierarchy

A central result in this lecture establishes the equivalence between the oracle-based definition (computational) and the alternating quantifier definition (logical).

Theorem 5.1 (Quantifier Characterization). For every integer $k \geq 1$, a language L is in Σ_k^P if and only if there exists a polynomially bounded relation $R \in \mathbf{P}$ such that for every x :

$$x \in L \iff \exists w_1 \forall w_2 \exists w_3 \cdots Q_k w_k \langle x, w_1, w_2, \dots, w_k \rangle \in R,$$

where $Q_k = \exists$ if k is odd, and $Q_k = \forall$ if k is even. Since R is polynomially bounded, there is a polynomial p such that the length of each w_i is bounded by $p(|x|)$.

Proof. We prove the equivalence by induction on k .

Base case ($k = 1$). The base case corresponds to the standard definitions of **NP** and **co-NP**.

$$\Sigma_1^P = \mathbf{NP} = \{L : \exists T \in \mathbf{P} \text{ s.t. } \forall x (x \in L \iff \exists w \langle x, w \rangle \in T)\}$$

$$\Pi_1^P = \mathbf{co-NP} = \{L : \exists S \in \mathbf{P} \text{ s.t. } \forall x (x \in L \iff \forall w \langle x, w \rangle \in S)\}$$

This matches the theorem statement for $k = 1$.

Induction Step ($k - 1 \mapsto k$). Let $L \in \Sigma_k^p = \mathbf{NP}^{\Sigma_{k-1}^p}$. We must construct a language $T \in \Pi_{k-1}^p$ such that

$$x \in L \iff \exists w_1 (x, w_1) \in T. \quad (2)$$

Since $L \in \mathbf{NP}^{\Sigma_{k-1}^p}$, by definition there exist a nondeterministic polynomial-time oracle TM $N^\bullet$ and an oracle language $O \in \Sigma_{k-1}^p$ such that $x \in L \iff N^O(x)$ accepts.

The Induction Hypothesis says that the equivalence holds for level $k - 1$. In particular,

- for our $O \in \Sigma_{k-1}^p$, its membership $q \in O$ has a quantifier-based characterization starting with $\exists$.
- for its complement $\overline{O} \in \Pi_{k-1}^p$, the membership $q \in \overline{O}$ has a quantifier-based characterization starting with $\forall$.

In particular, the induction hypothesis says that since the language O is in Σ_{k-1}^p , there is a language $C \in \Pi_{k-2}^p$ such that

$$\forall q \ q \in O \iff \exists w \ \langle q, w \rangle \in C. \quad (3)$$

Also recall that since the language $\overline{O}$ is in Π_{k-1}^p , it is decided by a polynomial-time **co-nondeterministic** Turing machine $M_{\overline{O}}^\bullet$ with access to some oracle $B \in \Sigma_{k-2}^p$. Such a machine $(M_{\overline{O}}^B)$ accepts an input iff *every* computation branch ends in acceptance.

1. Construction of the Outer Witness w_1

For each input x , define w_1 to encode full evidence for an accepting path (we will have to verify it using $T \in \Pi_{k-1}^p$). Our witness w_1 will include:

1. **The accepting path itself**, that is, a sequence of nondeterministic choices of $N^O(x)$ leading this machine to the accepting state. (Recall that we can limit the number of choices at each step to two (0/1) and thus consider this sequence a bit string.)
2. **Oracle queries and answers**: For each step where a query q_i is made, we record the pair (q_i, a_i) where $a_i \in \{0, 1\}$. (*We actually do not need to store the q_i 's, because we can compute them ourselves while simulating $N^O(x)$ on its accepting path, but let them be there.*)
3. **Local witnesses for “YES” answers**: For each i with $a_i = 1$, we include a witness u_i proving that $q_i \in O$, that is, $M_O(q_i, u_i) = 1$. (If $a_i = 0$, we do not need any u_i , let it be empty.)

2. Definition of the Verification Language T

We define T such that $(x, w_1) \in T$ iff all the following conditions hold:

- (1) **Path correctness:** The path in w_1 indeed drives $N^O(x)$ to the accepting state provided the oracle's answers are indeed a_i 's included in w_1 . (This can be checked deterministically in $\mathbf{P}$ using a universal Turing machine. At the same time the queries q_i can be extracted — or, if they are already included in w_1 , we can verify that $N^O(x)$ indeed makes this queries on this path.)
- (2) **Verification of “YES” answers:** For each i with $a_i = 1$, verify that u_i is a valid proof for $q_i \in O$. We have already observed in (3) that by induction hypothesis, this verification is a decision problem for some language $C \in \Pi_{k-1}^p$, that is, $q_i \in O \iff \langle q_i, u_i \rangle \in C$.
- (3) **Verification of “NO” answers:** For each j with $a_j = 0$, we require $q_j \notin O$. Therefore, we need to check $q_j \in \overline{O}$, and, as observed earlier, $\overline{O} \in \Pi_{k-1}^p$.

Denoting the polynomial-time path verification procedure by V and putting it all together:

$$\langle x, w_1 \rangle \in T \iff V(x, w_1) \wedge \bigwedge_{i:a_i=1} \langle q_i, u_i \rangle \in C \wedge \bigwedge_{i:a_i=0} q_i \in \overline{O}, \quad (4)$$

where all a_i, q_i, u_i are extracted from w_1 .

Note that the condition in this description of T is the conjunction of a polynomial number of conditions computable in Π_{k-1}^p . It remains to show that this conjunction is computable in Π_{k-1}^p itself.

3. Merging the Computations

We give two proofs for this fact: one based on mathematical logic, another one based on the computation-tree intuition.

Logic-style proof. Applying the induction hypothesis repeatedly to the two languages mentioned in (4), we arrive at

$$\begin{aligned} C &= \{q : \forall y_1 \exists y_2 \dots Q' y_{k-2} \langle q, u, y_1, \dots, y_{k-1} \rangle \in R_1\}, \\ \overline{O} &= \{q : \forall y_1 \exists y_2 \dots Q y_{k-1} \langle q, y_1, \dots, y_k \rangle \in R_2\}, \end{aligned}$$

where $R_1, R_2 \in \mathbf{P}$ and the quantifiers Q, Q' are defined by parity. Therefore, the conditions in our conjunction are of the form

$$\forall y_1 \exists y_2 \dots Q y_{k-1} \langle q, y_1, \dots, y_k \rangle \in R,$$

where y_i 's are not shared between these conditions. Recall from mathematical logic that in this case the quantifiers can be moved to the front and the conjunction of all these conditions (let us say that there are t conditions, where t is polynomial in the input size — it is bounded by the running time of the outer machine) can be written as

$$\forall y_1^{(1)} \forall y_1^{(2)} \dots \forall y_1^{(t)} \exists y_2^{(1)} \exists y_2^{(2)} \dots \exists y_2^{(t)} \dots Q y_{k-1}^{(1)} Q y_{k-1}^{(2)} \dots Q y_{k-1}^{(t)} \\ \left(V(x, w_1) \wedge \bigwedge_{i:a_i=1} \left(\langle q_i, u_i, y_1^{(i)}, \dots, y_{k-2}^{(i)} \rangle \in R_1 \right) \wedge \bigwedge_{i:a_i=0} \left(\langle q_i, y_1^{(i)}, \dots, y_{k-1}^{(i)} \rangle \in R_2 \right) \right),$$

which corresponds to the definition of a Π_{k-1}^p language according to the quantifier characterization (thus induction hypothesis). Note that the condition stated after the quantifiers is indeed polynomial-time computable (as R_1, R_2 are) and all a_i, q_i, u_i in it are parts of w_1 , so it is indeed a condition on x and w_1 only.

Computation-tree-style proof. This version of the proof is more intuitive (more natural), however, it must be done accurately in order to avoid some pitfalls.

Note that since all a_i, q_i, u_i in our condition are parts of w_1 , one can formally assume that the input to all these computations is $\langle x, w_1, i \rangle$, thus we are talking about the intersection of a polynomial number of languages in Π_{k-1}^p . *(This transformation of conjunction into intersection is not important: we can combine computations on different arguments the same way, in particular, if the arguments of the next computation are computed in the process of the previous computation.)*

Let $A, B \in \Pi_{k-1}^p$. By induction hypothesis, they are decided by polynomial-time co-nondeterministic oracle machines $M_A^{D_1}$ and $M_B^{D_2}$, where $D_1, D_2 \in \Sigma_{k-2}^p$. Construct a co-nondeterministic oracle machine $M_\cap$ deciding $A \cap B$ as follows:

1. On input z , behave as $M_A(z)$. If M_A arrives in its rejecting state, then $M_\cap$ rejects immediately.
2. Otherwise, after M_A comes to its accepting state, continue as $M_B(z)$ till it comes to its accepting or rejecting state, and accept or reject accordingly.

Analysis:

- If $z \in A \cap B$, then every branch of M_A accepts and thus the computation is continued. From every such leaf, every branch of M_B accepts. Thus, all branches of $M_\cap$ accept.
- If $z \notin A \cap B$, then either $z \notin A$ (so M_A has a rejecting branch that remains a rejecting branch for $M_\cap$) or $z \in A \wedge z \notin B$ (so while every M_A branch accepts, the subsequent M_B trees contain rejecting branches). In both cases, $M_\cap$ has a rejecting branch.

By iterating this polynomially many times, we get $T \in \Pi_{k-1}^p$.

The only problem is that $M_A^{D_1}$ and $M_B^{D_2}$ access two different oracles (fortunately, this situation does not replicate in our case, because all the checks use the same two oracles as they are done just by two machines: one for C and another one for $\overline{O}$). To conform to the definition of our oracle class, we need a single oracle.

If Σ_{k-2}^p had a complete language C_{k-2} , we could simply replace both D_1 and D_2 by this language: before accessing the oracle, perform the reduction of our query to C_{k-2} and then query C_{k-2} on the result of this reduction. However, we have not yet established the existence of complete problems for the classes of polynomial hierarchy (we will do very soon), thus let us proceed differently. Queries to two oracles can be replaced by queries to the oracle

$$D = \{(b, s) : b \in \{1, 2\}, \text{if } b = 1 \text{ then } s \in D_1, \text{ otherwise } s \in D_2\}$$

If we want to query $s \in D_i$, we simply query $(i, s) \in D$. Note that $D \in \Pi_{k-1}^p$: the machine accepting it looks at b and then proceeds either as the machine $M_1^{E_1}$ for D_1 or as the machine $M_2^{E_2}$ for D_2 , thus the problem of having two oracles is transferred to the next level $E_1, E_2 \in \Sigma_{k-2}^p$, etc, until we arrive at the 0-th level without any oracle. $\square$

5.4 Complete Problems for Σ_k^p (and Π_k^p)

We define a canonical complete problem for each level of the hierarchy, generalizing **SAT** (and **̄SAT**).

Theorem 5.2 (Completeness of **QBF_k**). The language **QBF_k** (Quantified Boolean Formulas with k alternations starting from $\exists$) is Σ_k^p -complete, where

$$\mathbf{QBF}_k = \{ \text{true formulas } \exists w_1 \forall w_2 \dots Q w_k \Phi(w_1, \dots, w_k) \},$$

where Φ is in CNF if $Q = \exists$ and in DNF if $Q = \forall$.

(Note that since all the variables of Φ are quantified and there are no free variables, this quantified formula either evaluates to **True** or to **False**, its value is a constant.)

NB!!! Each quantifier, such as $\exists w_1$, corresponds to many Boolean variables and not just a single bit!

Example 5.1. $\mathbf{QBF}_1 = \{ \text{true formulas } \exists w \Phi(w) \}$, i.e., the language of satisfiable formulas in CNF with the existential quantifier for the satisfying assignment formally written in front of the formula.

Since in the case of a single quantifier it is obvious which Boolean variables belong to which set w_i , we omit it in the definition of **SAT**, but otherwise it is essentially the same language.

Proof of Theorem 5.2.

Reducing every $L \in \Sigma_k^p$ to a quantified circuit (not yet CNF or DNF).

Consider an arbitrary language $L \in \Sigma_k^p$ and reduce it to a quantified circuit (like QBF_k , but with a circuit instead of a formula). Indeed, $L = \{x : \exists w_1 \forall w_2 \exists w_3 \dots Q w_k \langle x, w_1, \dots, w_k \rangle \in R\}$, where $R \in \mathbf{P}$. By Lemma 3.1 we can build in polynomial time a circuit C_n that is equivalent to R for $|x| = n$ (this circuit will have more than n input bits, because the input to R is $\langle x, w_1, \dots, w_k \rangle$, however, the number of bits in all these w_i 's is bounded by a polynomial in $|x|$). Therefore, our reduction can output the formula $\exists w_1 \forall w_2 \exists w_3 \dots Q w_k C(w_1, \dots, w_k) = 1$, where C is the circuit C_n with a substituted value for x , but we wanted to build a CNF or a DNF rather than a circuit.

To do this we will use the reduction from **Circuit-SAT** to **SAT** (See 2.4 and note that Levin's reduction (f, g, h) also provides a many-one reduction f .) How we employ it, depends on the quantifier $Q w_k$, that is, on the parity of k .

The Structure of Φ (CNF vs. DNF).

- **If k is odd ($Q_k = \exists$):** The reduction from **Circuit-SAT** to **SAT** produces a 3-CNF formula. The reduction introduces auxiliary existential variables ($\exists b$), which merge with the final $\exists w_k$ quantifier.

$$\dots \underbrace{\exists w_k \exists b}_{\exists \langle w_k, b \rangle} \Phi(x, w_1, \dots, w_k, b).$$

Thus, for odd k , Φ is in **3-CNF**.

- **If k is even ($Q_k = \forall$):** The reduction introduces $\exists b$, which would cause an extra alternation ($\dots \forall w_k \exists b$). To avoid this, start with the *complement* language ($\bar{L}$), then we get

$$\forall w_1 \exists w_2 \forall w_3 \dots Q w_k C'(w_1, \dots, w_k) = 1,$$

but this time $Q = \exists$ and we can convert C' into a 3-CNF and merge the existential quantifiers as before. Negate this formula, because we initially wanted to build a reduction from L ; the negation will start with the existential quantifier and the 3-CNF will turn into 3-DNF by de Morgan laws.

Correctness and Complexity of the Reduction

- **Correctness:** The reduction preserves satisfiability. C_n outputs 1 iff R holds. The formula Φ is satisfiable iff C_n outputs 1. The quantifiers for auxiliary variables merge with the final block, preserving the k -alternation structure.
- **Complexity:** Constructing the circuit C_n from the polynomial-time machine for R takes polynomial time. Converting the circuit to a formula takes essentially linear time in the size of the circuit. Thus, the overall running time of the reduction is polynomial. $\square$

5.5 Easy Observations and the Collapse of the Polynomial Hierarchy

A collapse of a hierarchy is a situation where it becomes finite, that is, almost all its classes coincide. Before talking about it, let us summarize several easy but important observations concerning these classes.

Observation 0. All classes Σ_k^p and Π_k^p are closed under many-one polynomial-time reductions. Thus, if $L \leq_m^p L'$ and $L' \in \Sigma_k^p$, then $L \in \Sigma_k^p$.

Observation 1. If $\mathbf{P} = \mathbf{NP}$, then all levels of the polynomial hierarchy coincide:

$$\mathbf{P} = \mathbf{NP} \Rightarrow \Sigma_k^p = \Pi_k^p = \mathbf{P} \quad \forall k.$$

The reason is that under this assumption all oracles $\mathbf{NP}$ become $\mathbf{P}$ and thus can be incorporated into lower machines as subprocedures (that is, in any case $\mathbf{NP}^{\mathbf{P}} = \mathbf{NP}$ and $\mathbf{P}^{\mathbf{P}} = \mathbf{P}$). Thus we can gradually collapse the towers of oracles starting from the highest oracle in this tower $\mathbf{NP}^{\mathbf{NP}^{\dots^{\mathbf{NP}}}}$ to the basic class $\mathbf{P}$, $\mathbf{NP}$, or $\mathbf{co-NP}$. These classes of the first level also equal $\mathbf{P}$ under our assumption.

Now let us formulate the main collapse result.

Observation 2. If for some k we have $\Sigma_k^p = \Pi_k^p$, then the entire hierarchy collapses to the k -th level:

$$\Sigma_k^p = \Pi_k^p \implies \Sigma_k^p = \Sigma_{k+1}^p = \dots = \mathbf{PH}.$$

This follows from the fact that our assumption allows to negate the quantified formula defining a language in Π_k^p (or Σ_k^p), thus the first quantifier changes and two quantifiers merge: by Theorem 5.1, every $L \in \Sigma_{k+1}^p$ can be described as

$$L = \{x : \exists w \langle x, w \rangle \in T\}$$

for some $T \in \Pi_k^p$. By our assumption $T \in \Sigma_k^p$, that is by the same theorem it can be described as

$$T = \{y : \exists w' \langle y, w' \rangle \in S\},$$

where $S \in \Pi_k^p$. Substituting T into the description for L , we get

$$L = \{x : \exists w \exists w' \langle \langle x, w \rangle, w' \rangle \in S\},$$

and the two existential quantifiers merge into one.

Observation 3. The class

$$\mathbf{PH} = \bigcup_{k \geq 0} \Sigma_k^p$$

has no complete language unless the polynomial hierarchy collapses.

To see it, assume that a language L' is complete for $\mathbf{PH}$. Then $L' \in \mathbf{PH}$, and thus $L' \in \Sigma_k^p$ for some specific k (because $\mathbf{PH}$ is the union of these classes). We assumed that every language in $\mathbf{PH}$ reduces to L' ; since the class Σ_k^p is closed under polynomial-time reductions, it follows that:

$$\mathbf{PH} \subseteq \Sigma_k^p,$$

and hence the hierarchy collapses to level k .

It is unknown whether the polynomial hierarchy is infinite. However, we do not really believe in the collapse of $\mathbf{PH}$. Thus, it is unlikely to contain a complete language.

5.6 Polynomial-Size Circuits and the Karp–Lipton Theorem

5.6.1 Polynomial-Size Circuits

We now consider a non-uniform model of computation, where instead of a single Turing machine solving all input lengths, each input length n has its own circuit C_n .

Definition 5.4 (Size classes). Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function. A language L is in **Size** $[f(n)]$ iff there exists a family of Boolean circuits $\{C_n\}_{n \in \mathbb{N}}$, each circuit being of size $O(f(n))$ (containing $O(f(n))$ gates), such that for every input $x \in \{0, 1\}^*$:

$$x \in L \iff C_{|x|}(x) = 1.$$

Definition 5.5 ($\mathbf{P/poly}$).

$$\mathbf{P/poly} = \bigcup_{n \in \mathbb{N}} \mathbf{Size}[n^k],$$

that is, a language L is in $\mathbf{P/poly}$ iff there exists a family of polynomial-size Boolean circuits for it.

Theorem 5.3. $\mathbf{P} \subseteq \mathbf{P/poly}$.

This follows because a run of a polynomial-time deterministic Turing machine can be unfolded into a Boolean circuit if the input length and the time bound are known (Lemma 3.1).

On the other hand, $\mathbf{P/poly}$ is much larger than $\mathbf{P}$ (the reason for its strength is that a Turing machine's program contains a fixed number of bits, and a family of circuits is infinite):

Proposition 5.1. $\mathbf{P/poly} \not\subseteq \mathbf{P}$.

Proof. The class $\mathbf{P/poly}$ contains undecidable languages. For example, consider the language $\{x : T_{|x|} \text{ stops on input } \varepsilon\}$. This language is a version of the halting problem (you know this problem from your undergraduate courses such as Computability), it cannot be decided by any Turing machine stopping on every its input (even in exponential time!). However, there are very easy circuits solving this problem: they even do not need to look at their input, because the answer is the same for all strings of the same length. $\square$

5.6.2 The Karp–Lipton Theorem (introduction)

Polynomial-size circuits are much more powerful than polynomial-time computation. A natural question arises: Can they solve **SAT**? If it happened, that is, if $\mathbf{NP}$ was contained in $\mathbf{P/poly}$, this would imply a collapse of the polynomial hierarchy.

Theorem 5.4. [Karp–Lipton (R. Karp, R. Lipton, M. Sipser)] If $\mathbf{NP} \subseteq \mathbf{P/poly}$, then $\mathbf{PH} \subseteq \Sigma_2^p$.

Remark 5.1. Theorems of this style are a hot topic in the field of computational complexity: deeper and deeper collapses are shown under this assumption ($\mathbf{NP} \subseteq \mathbf{P/poly}$), that is, collapses to classes much smaller than Σ_2^p (however, for Δ_p^2 it is still unknown).

The proof of this theorem will be given in the next lecture.

6 Lecture 6. Scribe: Ayelet Katkov

The main part of this lecture includes two tightly connected topics:

- The Karp-Lipton Collapse Theorem (with proof).
- Fixed-polynomial circuit complexity lower bounds for classes of the polynomial hierarchy:
 - Kannan’s theorem: $\Sigma_4^p \not\subseteq \text{Size}[n^k]$.
 - Consequences for Σ_2^p .

We will then go from $\mathbf{PH}$ to a larger class (**PSPACE**). This lecture features Introduction to Space Complexity (covered in depth in the next lecture): definitions, **PSPACE**, the **QBF** language and other examples.

6.1 Reminders from previous lectures

Reminder: P/poly. A language L is in **P/poly** if and only if there exists a family $\{C_n\}_{n \in \mathbb{N}}$ of Boolean circuits (with n input bits), each of polynomial size in n , such that for all $x \in \{0, 1\}^*$,

$$x \in L \iff C_{|x|}(x) = 1.$$

This is called a (non-uniform) **polynomial-size circuit family** for L .

(Note: “non-uniform” here means that these circuits in the family have no single efficient algorithm that could print them all, that is, circuits on different input sizes n and n' can have absolutely different structure.)

Reminder: a Σ_k^p -complete language (proved last lecture)

$$\text{QBF}_k = \{ \text{true formulas } \exists w_1 \forall w_2 \exists \dots Q w_k \Phi \},$$

where Φ is a CNF (if k is odd) or DNF (if k is even) in variables appearing in the lists $w_1, \dots, w_k$.

(NB!!!) each of these strings w_i contains many Boolean variables!

$$\exists w_1 \sim \exists w_{1,1} \in \{0, 1\} \exists w_{1,2} \in \{0, 1\} \dots \exists w_{1,t} \in \{0, 1\}.$$

(Note: This problem is also known as Σ_k -QBF (Σ , because the first quantifier is $\exists$) and **TQBF** (“T” for “True”). Remember that k is the number of quantifiers, that is, the number of points where the quantifier changes from $\exists$ to $\forall$ or vice versa (one can consider $\exists u_1 \exists u_2$ a single quantifier $\exists u$, where u can be interpreted as a pair of two strings, $u = \langle u_1, u_2 \rangle$.)

6.2 The Karp–Lipton Theorem (now with a proof)

Let us prove the Karp–Lipton Theorem formulated in the previous lecture:

Theorem (Theorem 5.4 (R. Karp, R. Lipton, M. Sipser)). If $\mathbf{NP} \subseteq \mathbf{P/poly}$, then $\mathbf{PH} \subseteq \Sigma_2^p$.

Proof. We start with giving some intuition and then will proceed to a formal proof.

Intuition. We are proving the Karp–Lipton theorem, that is, we will assume $\mathbf{NP} \subseteq \mathbf{P/poly}$ and prove $\mathbf{PH} \subseteq \Sigma_2^p$. To prove the collapse of the Polynomial Hierarchy to the second level ($\mathbf{PH} = \Sigma_2^p$), it is enough to prove $\Sigma_3^p = \Sigma_2^p$ (or even $\Pi_2^p = \Sigma_2^p$).

Why so? If $\Sigma_3^p = \Sigma_2^p$, then we can replace the oracle in the definition of $\Sigma_4^p = \mathbf{NP}^{\Sigma_3^p}$ and it can be rewritten as $\Sigma_4^p = \mathbf{NP}^{\Sigma_2^p}$, that is, $= \Sigma_3^p$ by definition, and so on for $\Sigma_5^p, \dots$ resulting in $\mathbf{PH} = \bigcup_{k \in \mathbb{N}} \Sigma_k^p \subseteq \Sigma_2^p$.

Actually, $\Sigma_2^p = \Pi_2^p$ does the same job instead of $\Sigma_3^p = \Sigma_2^p$. Indeed, due to the quantifier-characterization of Σ_k^p, Π_k^p from the previous lecture, a language L in Σ_3^p can be represented as

$\{x : \exists w_1 \langle x, w_1 \rangle \in R\}$, where $R \in \Pi_2^p$. However, if $\Sigma_2^p = \Pi_2^p$, then R can be represented also as $\{\langle x, w_1 \rangle : \exists w_2 \forall w_3 \langle \langle x, w_1 \rangle, w_2, w_3 \rangle \in S\}$ for $S \in \mathbf{P}$, and in total $L = \{\underbrace{\exists w_1 \exists w_2}_{\exists \langle w_1, w_2 \rangle} \forall w_3 \langle x, w_1, w_2, w_3 \rangle \in S'\}$, where $S' \in \mathbf{P}$ is essentially S .

Proof plan ($\mathbf{NP} \subseteq \mathbf{P/poly} \Rightarrow \Sigma_3^p = \Sigma_2^p$):

- We want to build a many-one polynomial-time reduction, that is, to transform (in polynomial time) a $\mathbf{QBF}_k$ (a Σ_3^p -complete problem) instance $\exists x \forall y \exists z F(x, y, z)$ into a Σ_2^p -problem instance $\exists q \forall s \langle q, s \rangle \in R$ for some $R \in \mathbf{P}$.
- How to remove a quantifier? If we could solve “for free” (that is, using the same two quantifiers that find x and check y) the $\mathbf{SAT}$ instance $\exists z F(z)|_{x=x_*, y=y_*}$ (that is, the satisfiability of the formula obtained from F after replacing the variables in x and y by 0/1 constants¹⁹), then we could rid of the last quantifier.
- Since we assume $\mathbf{NP} \subseteq \mathbf{P/poly}$, then $\mathbf{SAT} \in \mathbf{NP}$ has small (polynomial-size) circuits family $\{S_n\}_{n \in \mathbb{N}}$ that solve it. We could use these circuits — if someone would give them to us.
- We can find (and use) them using only 2 quantifiers: $\exists$ to choose a circuit and $\forall$ to verify its correctness. (Note that we need to verify the correctness: $\exists$ only gives us a “witness” that we need to verify.)
- Then we will use these $\mathbf{SAT}$ circuits to remove the last quantifier $\Rightarrow$ to prove $\Sigma_3^p = \Sigma_2^p$.

Formal proof. We assume $\mathbf{NP} \subseteq \mathbf{P/poly}$, so $\mathbf{SAT} \in \mathbf{NP}$ has some polynomial-size circuit family $\{S_n^*\}_{n \in \mathbb{N}}$ deciding it. Now assume we are given some candidate circuit S_n (either correct or not — we do not know it yet). We need to verify it decides $\mathbf{SAT}$ for formulas of the corresponding length.

W.l.o.g. we can assume that the encoding of formulas in CNF satisfies the following natural condition: If a formula F is encoded using n bits, then n bits are also sufficient to encode smaller formulas (at least, those obtained from F by substituting some of its variables by 0/1 constants).

If their description is shorter, we can add some “garbage” to it to match size n .

We could live without this assumption, but then we would need to check $\{S_k\}_{k \leq n}$ instead of just a single S_n (because S_n needs exactly n bits of the input) — no problem, but then our notation would be cumbersome.

¹⁹Recall how we substituted a Boolean variable by its value in a formula in CNF in Algorithm 2.1: we removed the clauses that contained the satisfied literal (say, the literal $\bar{x}_i$ if we put $x_i = \mathbf{False}$) and removed the opposite literal from the other clauses. The simplification we have made there is nice and natural, while not absolutely necessary — alternatively, we could simply generalize $\mathbf{SAT}$ to allow the use of Boolean constants along with the variables, like in $(z_1 \vee z_2 \vee \mathbf{False}) \wedge (\bar{z}_1 \vee \mathbf{True})$.

Verification: By definition,

$$S_n \text{ is a correct SAT circuit } \iff \forall \Phi \in \text{SAT of size } \leq n, S_n(\Phi) = \text{SAT}(\Phi).$$

We can compute $S_n(\Phi)$ fast (in polynomial time) directly by evaluation the circuit S_n on input Φ , but what about $\text{SAT}(\Phi)$? How to compute SAT — it is a hard problem! However, SAT is downward-self-reducible (i.e., reducible to its smaller instances, of course, not for free). We have already used this property in the search-to-decision reduction, Algorithm 2.1 (in particular, if $S_n(F) = 1$, i.e., S_n claims that F is satisfiable, we can use S_n to extract a satisfying assignment and verify this assignment in polynomial time; but if $S_n(F) = 0$, it is unclear yet how to verify the correctness of this claim).

We can verify a candidate SAT algorithm (or a SAT circuit) “recursively”, at each step checking what happens for both possible values of the current variable (0,1).

$$\Phi \in \text{SAT} \iff \Phi|_{x_1=0} \in \text{SAT} \text{ or } \Phi|_{x_1=1} \in \text{SAT}$$

See visualized tree in Figure 9.

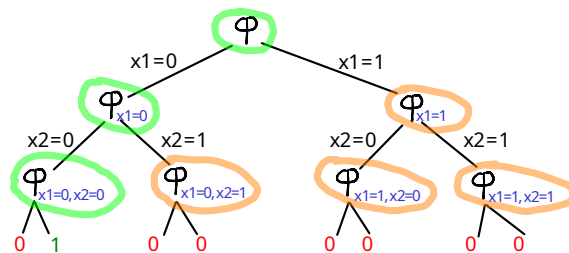


Figure 9: SAT formula reduction.

Unlike the search-to-decision reduction, to verify the answer “unsatisfiable” we need to check both branches. However, it would take exponential time. Fortunately, we have a universal quantifier that would help us: for each Φ described by n bits²⁰ (that is, as agreed above, for smaller formulas as well), let us verify

$$S_n(\Phi) = S_n(\Phi|_{x_1:=0}) \vee S_n(\Phi|_{x_1:=1})$$

(Here x_1 denotes the first variable appearing in Φ .)

This is similar to recursion, so what is missing? The base cases (the leaves of the tree). So we need to require that the behaviour of S_n on simple formulas is correct: $S_n(\text{True}) = 1$, $S_n(\text{False}) = 0$ for the two basic formulas without variables²¹

Summary: S_n solves SAT correctly for size- n formulas iff:

²⁰**NB!!!** n is not the number of variables in Φ , it is the number of input bits of S_n , and the encoding of Φ is fed to S_n bit by bit.

²¹Formally, **False** is a formula containing an already falsified, that is, empty disjunction; **True** is an already satisfied formula, that is, the empty conjunction containing no disjunctions at all.

$$\forall \Phi \text{ encoded by } n \text{ bits } \underbrace{S_n(\Phi) = (S_n(\Phi|_{x_1:=0}) \vee S_n(\Phi|_{x_1:=1})) \wedge S_n(\text{True}) = 1 \wedge S_n(\text{False}) = 0}_{\text{VERIFY}(S_n, \Phi)},$$

where the formulas $\Phi|_{x_i:=b}$ are encoded also by n bits (due to our agreement).

Let us denote the expression under the quantifier as $\text{VERIFY}(S_n, \Phi)$. It can be evaluated in polynomial time given S_n and Φ as the input.

Let us use this verification procedure to show the collapse. Assuming $\mathbf{NP} \subseteq \mathbf{P/poly}$ there exist correct **SAT** circuits, thus “ $\exists$ ” can give such a circuit to us (for length n) and “ $\forall$ ” can verify them, and then we can use them to solve **SAT** by ourselves thus getting rid of the last existential quantifier in a three-quantifier formula $\exists u \forall v \exists w F(u, v, w)$: for every formula F encoded by n bits and for every splitting its variables into u, v, w ,

$$\exists \textcolor{blue}{u} \forall \textcolor{red}{v} \exists \textcolor{green}{w} F \iff \exists S_n \exists \textcolor{blue}{u} \quad \forall \textcolor{red}{v} \forall \Phi \quad \text{VERIFY}(S_n, \Phi) \wedge S_n(\underbrace{F|_{\textcolor{blue}{u}, \textcolor{red}{v}}}_{\text{variables } \textcolor{green}{w}}) = 1,$$

where $F|_{\textcolor{blue}{u}, \textcolor{red}{v}}$ denotes the result of substituting in F the values given to the variables in $\textcolor{blue}{u}$ and $\textcolor{red}{v}$ by the quantifiers $\exists \textcolor{blue}{u} \forall \textcolor{red}{v}$.

Reminder: Each quantifier for example $\exists x$ can represent a sequence of similarly quantified Boolean variables, like $\exists x_1 \in \{0, 1\} \exists x_2 \in \{0, 1\} \exists x_3 \in \{0, 1\} \dots$.

Explanation: Formula $\exists u \forall v \exists w F(u, v, w)$ with three quantifiers means that after you are given the values of u, v by the first two quantifiers, then there exists some w that satisfies the formula that now has only variables in w . This is exactly the Boolean satisfiability problem, thus we can use the circuit S_n for **SAT** to solve it: S_n will tell us whether there exists a satisfying assignments for the variables (w) still remaining in $F|_{\textcolor{blue}{u}, \textcolor{red}{v}}$.

Therefore, to check whether the original formula $\exists u \forall v \exists w F$ is true (that is, belongs to the Σ_3^P -complete language $\mathbf{QBF}_3$), we can use an expression from the alternative definition (quantifier characterization, Theorem 5.1) of Σ_2^P : the existential quantifier, the universal quantifier, and then a polynomial-time computable condition $\text{VERIFY}(S_n, \Phi) \wedge S_n(\underbrace{F|_{\textcolor{blue}{u}, \textcolor{red}{v}}}_{\text{variables } \textcolor{green}{w}}) = 1$ (we already noticed that VERIFY is polynomial-time, substituting values to write down $F|_{\textcolor{blue}{u}, \textcolor{red}{v}}$ and compute the value of the circuit S_n on it can also be done in polynomial time).

Therefore, we proved that under the theorem’s assumption, $\Sigma_3^P \subseteq \Sigma_2^P$. $\square$

Corollary 6.1. If $\mathbf{NP} \subseteq \mathbf{P/poly}$, then $\mathbf{PH} \subseteq \Sigma_2^P \cap \Pi_2^P$.

Proof. The added part “ $\cap \Pi_2^P$ ” is due to the symmetry of **PH**: $\mathbf{PH} = \mathbf{co-PH}$ (as it contains all Σ - and **co**- Σ -, i.e., Π -classes). By the Karp-Lipton Theorem $\mathbf{PH} \subseteq \Sigma_2^P$, thus $\mathbf{co-PH} \subseteq \Pi_2^P$, but $\mathbf{PH} = \mathbf{co-PH}$ and thus $\mathbf{PH} \subseteq \Pi_2^P$. $\square$

Remark 6.1. Much deeper collapses under this assumption ($\mathbf{NP} \subseteq \mathbf{P/poly}$) are known, yet nobody is currently able to show a collapse to $\Delta_2^P = \mathbf{P}^{\mathbf{NP}}$.

6.3 Fixed-Polynomial Circuit-Size Lower Bounds

Circuit lower bounds are the “Holy Grail” of the complexity theory and TCS (think about $\mathbf{NP} \not\subseteq \mathbf{P}$ or $\mathbf{NP} \not\subseteq \mathbf{P}/\mathbf{poly}$). While there are no functions without equivalent circuit families (recall that one can always construct an exponentially large circuit out of the truth table), still there exist functions without equivalent small (polynomial-size) circuits!

A “circuit-size lower bound” for a function means the minimum size that Boolean circuits must have to compute this function: the function is “too hard” for smaller circuits.

An easy counting argument shows that such hard functions do exist:

Theorem 6.1. There is $f: \{0, 1\}^* \rightarrow \{0, 1\}$ such that $\nexists \{C_n\}_{n \in \mathbb{N}}$ of polynomial size in n computing f .

(Note that a function is defined on arbitrary-length inputs, that is, it is an infinite object. A circuit family is also an infinite object, but a single circuit C_n is able to read only a specific number n of bits, and its size $\text{poly}(n)$ is discussed in the theorem — of course, asymptotically in n .)

Proof. Let us split f into “slices” $f_n: \{0, 1\}^n \rightarrow \{0, 1\}$ according to the length of the input.

For a specific n , there are 2^{2^n} different functions $g: \{0, 1\}^n \rightarrow \{0, 1\}$ (think about a truth table for g : there are 2^n inputs, for each input we can choose g ’s value 0 or 1).

How many polynomial-size circuits are there? A circuit containing m gates can be described by $O(m \log n)$ bits (see an exercise and a remark below the proof). Thus for a specific polynomial p , there are at most $2^{O(p(n) \log n)}$ circuits of size $\leq p(n)$. Asymptotically (that is, starting from some sufficiently large n) $2^{O(p(n) \log n)} < 2^{2^n}$, i.e., there are much many functions on $\{0, 1\}^n$ compared to circuits, thus for almost every n there exists a function g without $p(n)$ -size circuits, so let $f_n := g$. $\square$

Exercise 6.1. A circuit containing m gates computing binary (or unary) operations can be described by $O(m \log n)$ bits.

Remark 6.2. A more efficient encoding is left as an exercise, however, even if you describe a circuit using $O(m^2)$ bits (inefficiently: using the adjacency matrix of the directed graph), our proof will also work; if you change the numbers in the proof of the next theorem accordingly, it will work as well.

Hard functions have extensive use in cryptography, in derandomization (replacing a random source by a pseudorandom generator), etc. We have proved their existence non-constructively, but now please give us such a hard function! Theorem 6.1 does not give any particular function, and we are interested in “explicit” f — ideally, computed in $\mathbf{NP}$ (recall that a Boolean function corresponds to a language, $x \in L_f \iff f(x) = 1$).

Currently no one can prove it exists, but we can prove something in this direction:

1. We will not prove an exponential lower bound: we will prove it just for an arbitrarily large polynomial.

2. We will not prove it for **NP** (nobody can yet): only for a somewhat larger class (Σ_4^p , then Σ_2^p , further improvements are possible).

6.3.1 Fixed-Polynomial Circuit Lower Bounds for Σ_4^p : Kannan's Theorem

Theorem 6.2 (Kannan). $\forall k \in \mathbb{N} \quad \Sigma_4^p \not\subseteq \text{Size}[n^k]$.

In other words: let us prove that, for every $k \in \mathbb{N}$, the class Σ_4^p has a language $L^{(k)}$ (=a function) that has circuit complexity larger than $\Omega(n^k)$.

Remark 6.3. The statement of this theorem does not mean $\Sigma_4^p \not\subseteq \mathbf{P}/\mathbf{poly}$. For the latter we would need to construct a *single* language L that would work against all circuits of polynomial size in n for *all polynomials* (i.e., all values of k in n^k). In other words, the difference is between $\forall k \exists$ hard L versus $\exists L$ that is hard $\forall k$ — it is in the order of quantifiers (our quantifiers for the mathematical statement of the theorem, not QBF's).

Proof.

- Recall (Theorem 6.1) that for a specific value of n , there are 2^{2^n} different functions $g : \{0, 1\}^n \rightarrow \{0, 1\}$. This is even far too many for us.
- Look at the first $(k+1) \log n$ variables (input bits) of g : there are $2^{2^{(k+1) \log n}} = 2^{n^{k+1}}$ different functions depending on them. (That is, after the values of these variables are fixed, the values of the other variables do not affect the values of g . For example, $g(x_1, x_2, x_3, x_4) = x_1 \oplus x_2$ is formally a function of four variables, but it depends only on its two first variables.)
- There are only $2^{O(n^k \log n)}$ circuits of size $O(n^k)$, as observed in the proof of Theorem 6.1.
- $2^{n^{k+1}} > 2^{O(n^k \log n)}$ asymptotically, thus there exists a function that cannot be computed any such circuit.
- We need to find (and compute) just one such function! With 4 quantifiers (to fit Σ_4^p) we will be able to do it.

Let us write what does it mean that g is hard for circuits of size $O(n^k)$. (To be absolutely strict and account for all constants in this $O(\dots)$, let us work against circuits of size $n^{k+0.1}$ — there are still fewer such circuits than functions depending on their $(k+1) \log n$ first variables.)

Literally, it means that every circuit C_n with fewer than $n^{k+0.1}$ gates does not compute g correctly, that is, there is a “buggy” input x where g and C_n produce different outputs:

$$\exists g \text{ depending on variables } x_1, \dots, x_{(k+1) \log n}$$

$$\forall C_n \text{ of size } \leq n^{k+0.1}$$

$$\exists x \in \{0, 1\}^n$$

$$g(x_1, \dots, x_n) \neq C_n(x_1, \dots, x_n)$$

Here g is described by its truth table: Note that $\exists g$ ranges over truth tables for functions depending on $(k+1) \log n$ variables, thus the size of this truth table is polynomial in n , namely, $2^{\lceil (k+1) \log n \rceil} \leq (2n)^{k+1}$.

The quantifier $\forall C_n$ ranges over circuits of size $\leq n^{k+0.1}$ — such a circuit can also be described by a polynomial number of bits, namely, by at most $cn^{k+0.1} \log n$ bits for some universal constant c .

The definition above describes the property of g being a hard function. However, there are many functions satisfying it, and we need to choose and compute a single such function. Let us order all functions (that is, truth tables) lexicographically (and write $g < g'$ if g is smaller than g' in this order), and let us choose the first (the smallest such function). To do this, we need to write that every smaller function *can* be computed by small circuits:

$$\forall g' < g \quad // \text{lexicographically smaller truth table}$$

$$\exists C'_n \text{ of size } \leq n^k$$

$$\forall x' \in \{0, 1\}^n g'(x'_1, \dots, x'_n) = C'_n(x'_1, \dots, x'_n).$$

Let us merge these two properties into one:

$$\exists g \text{ depending on variables } x_1, \dots, x_{(k+1) \log n} \quad // \text{truth table of polynomial size}$$

$$\forall g' < g \quad // \text{lexicographically smaller truth table}$$

$$\forall C_n \text{ of size } \leq n^k$$

$$\exists x \in \{0, 1\}^n$$

$$\exists C'_n \text{ of size } \leq n^k$$

$$\forall x' \in \{0, 1\}^n \quad (g(x_1, \dots, x_n) \neq C_n(x_1, \dots, x_n)) \wedge (g'(x'_1, \dots, x'_n) = C'_n(x'_1, \dots, x'_n))$$

It remains to output the value of this function. Let us come back to our initial goal: we need to prove that there is a hard function h (put another way, a hard language $H \in \Sigma_4^p$ such that we can compute it on z with four quantifiers. Here it is: for every input z of size n ,

$$z \in H \iff h(z) = 1 \iff$$

$$\exists g \text{ depending on variables } x_1, \dots, x_{(k+1) \log n} \quad // \text{truth table of polynomial size}$$

$$\forall g' < g \quad // \text{lexicographically smaller truth table}$$

$\forall C_n$ of size $\leq n^k$

$\exists x \in \{0, 1\}^n$

$\exists C'_n$ of size $\leq n^k$

$\forall x' \in \{0, 1\}^n :$

$(g(x_1, \dots, x_n) \neq C_n(x_1, \dots, x_n)) \wedge (g'(x'_1, \dots, x'_n) = C'_n(x'_1, \dots, x'_n)) \wedge \underline{g(z) = 1}.$

Let us pronounce again how it works:

- The smallest hard function is a specific function (mathematically! even if we do not compute it), call it h .
- If $h(z) = 1$, then the existential quantifier $\exists g$ selects $g = h$, the equality checks at the end of the description go successfully, and the check $g(z) = 1$ also succeeds, because $g = h$ and $h(x) = 1$.
- If $h(z) = 0$, then even if the existential quantifier $\exists g$ selects $g = h$ and thus the equality checks at the end of the description go successfully, the final check $g(z) = 1$ fails, because $g = h$ and $h(x) = 0$.
- Note that if the existential quantifier $\exists g$ selects some different function (a function that is not hard or that is not the smallest one), it does not influence the result (the truthfulness of the whole statement), because the equality checks at the end of this statement fail: if does not matter in the case of $h(z) = 0$ (the result is 0 anyway), and in the case of $h(z) = 1$ we are saved by the case where the existential quantifier $\exists g$ selects $g = h$.

We described it using four quantifiers $\exists\forall\exists\forall$ followed by a polynomial-time computable predicate (what we need to do to compute this predicate is evaluating given circuits C_n , C'_n on given inputs x , x' and looking for the entries corresponding to x , x' in the truth tables of g and g'). $\square$

6.3.2 Pushing it lower: Σ_2^p instead of Σ_4^p

The two theorems we prove in this lesson together imply a stronger result:

Corollary 6.2. $\forall k \in \mathbb{N} \quad \Sigma_2^p \not\subseteq \mathbf{Size}[n^k].$

Proof. Assume to the contrary that for some k , $\Sigma_2^p \subseteq \mathbf{Size}[n^k]$. However, trivially $\mathbf{NP} \subseteq \Sigma_2^p$ and $\mathbf{Size}[n^k] \subseteq \mathbf{P/poly}$, thus our assumption means, in particular, that $\mathbf{NP} \subseteq \mathbf{P/poly}$. The Karp-Lipton Theorem (Th. 5.4) then implies the collapse $\mathbf{PH} = \Sigma_2^p$. However, due to this collapse $\Sigma_2^p = \Sigma_4^p$, but we already know for this class that $\Sigma_4^p \not\subseteq \mathbf{Size}[n^k]$ by Kannan's Theorem (Th. 6.2), a contradiction! $\square$

Remark 6.4. Thus a better version of the Karp–Lipton theorem (with a stronger collapse) would automatically give us a better corollary providing a hard function in a smaller class — *as long as we are talking about collapses to classes containing $\mathbf{NP}$, because we used $\mathbf{NP} \subseteq \Sigma_2^P$ in the proof of the corollary.*

Such lower bounds are also known for some classes not containing $\mathbf{NP}$ (of course, it does *not* mean these classes are contained inside $\mathbf{NP}$). However, the techniques for them are very different.

6.4 Introduction to Space Complexity

In the next lesson we will speak about *space complexity*. Let us prepare for it.

Space complexity is the measurement of space, memory, tape cells, counted as the amount of bits used by an algorithm/program.

In terms of Turing machines, recall that **time** is the number of steps before a Turing machine stops.

Now **space** is the number of cells it uses, that is, the sum of the rightmost positions of its heads: $\sum_{i=1}^k m_i$, where k is the number of tapes and m_i is the rightmost position of the i -th head through the run of the machine.

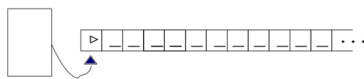


Figure 10: A 1-tape Turing machine.

Surprisingly, classes that could be defined by restricting the space (and not time) can also be defined using an unlimited amount of quantifiers $\exists \forall \exists \forall \exists \dots$. Therefore, in particular, polynomial-space bounded computations can be thought of as a generalization of $\mathbf{PH}$. More about this later.

6.4.1 Definitions: DSpace, PSPACE, NSpace

Caution! We now define space-bounded complexity classes without paying much attention to a specific version of the definition of a Turing machine. It does not matter as long as we are discussing polynomial-bounded space complexity, but when we later come to smaller-space complexity classes (such as logarithmic space) we will need to make this definition more precise w.r.t. our model of computation.

Definition 6.1. For a function $f: \mathbb{N} \rightarrow \mathbb{N}$, define

$$\mathbf{DSpace}[f(n)] = \{L \mid L \text{ is decided by a DTM using space } O(f(n))\}.$$

The union of this classes for all polynomials f is the class

$$\mathbf{PSPACE} = \bigcup_{k \geq 0} \mathbf{DSpace}[n^k].$$

Therefore,

PSPACE = all languages decided by DTMs using a polynomial in the input size amount of tape cells.

Soon we will see that **PSPACE** corresponds to two-player games like chess — this is not obvious from this definition.

For a NTM, the space is measured as the maximum space used on all nondeterministic paths. This means the space used by the most space-consuming possible non-deterministic path (*not* the sum of space used by all paths together!).

Definition 6.2. $\mathbf{NSpace}[f(n)] = \{L \mid L \text{ is decided by a NTM using space } O(f(n))\}.$

6.4.2 Intuition: how multiple quantifiers (two-player games!) can be computed in polynomial space

In what follows we will see how to characterize **PSPACE** using quantifiers. For now let us bring some intuition: As you might remember, we have previously described **NP** (a run of a nondeterministic machine) as a tree similar to Fig. 11, but where all the quantifiers are $\exists$, and for **co-NP** (co-nondeterministic machines) all quantifiers are $\forall$. One can generalize it and put different quantifiers in “layers” (so the tree in Fig. 11 corresponds to a simple instance of Σ_3^P , i.e., $\exists\forall\exists$; in fact, each layer can be more “thick”: recall that it is only the number of quantifiers changes that counts).

One could consider a tree with an arbitrary number of different quantifiers, with a natural recursive meaning:

- Each leaf contains either 0 or 1 (and is evaluated to this value).
- Let T consist of two subtrees T_1 and T_2 evaluated to $b_1 \in \{0,1\}$ and $b_2 \in \{0,1\}$. Then
 - If the quantifier in the root of T is $\exists$, it is evaluated as $\max(b_1, b_2)$.
 - If the quantifier in the root of T is $\forall$, it is evaluated as $\min(b_1, b_2)$.

Several examples of evaluating (sub)trees are shown in Fig. 11–14.

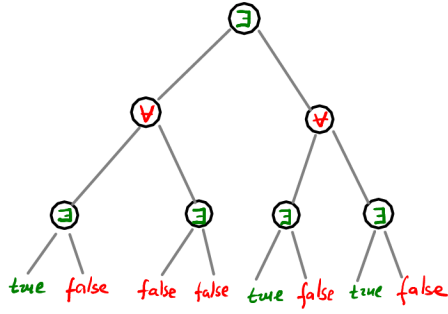


Figure 11: An $\exists\forall$ tree.

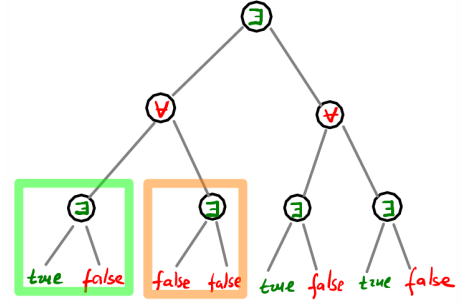


Figure 12: Two subtrees are evaluated.

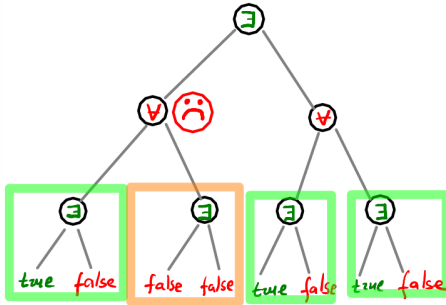


Figure 13: Two left subtrees and their parent are evaluated, going further...

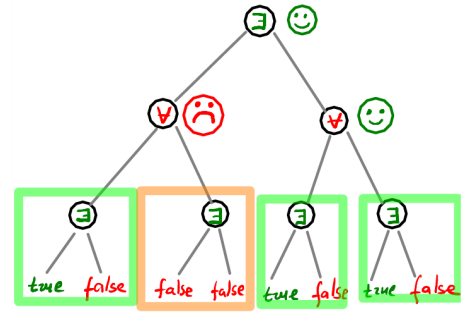


Figure 14: The whole tree is evaluated

One can think of such a tree as a two-player game: the root ($\exists$) — Player 1's move, the second level ($\forall$) — Player 2's decision, the third level is again Player 1's move, etc, the leaves are the game results for Player 1. Let us play for Player 1: then if we write a sentence $\exists$ our move $\forall$ our adversary move $\exists$ our move ... (etc) ... we win, it exactly means that we (Player 1) have a winning strategy in this game against any Player 2.

Such a tree can be evaluated using a depth-first procedure that requires only a polynomial amount of memory — we will talk about it in detail later. Moreover, we will see not only that this type of game is a **PSPACE** problem, but also that all **PSPACE** problems can be described this way.

6.4.3 Examples of PSPACE problems

Example 6.1 (SPACE-BH, SPACE TMSAT in the book). This example does not look like a game, but it is a “standard” problem for the class:

$$\text{SPACE-BH} = \{\langle M, z, 1^t \rangle : \text{DTM } M \text{ accepts } z \text{ in space } t\}.$$

Why is it in **PSPACE**? One can easily see from the construction of a universal Turing machine (we will discuss it later again) that if $M(x)$ uses space $s(|x|)$, then $\text{UTM}(\langle M, x \rangle)$ can simulate M on x using space $O(s(|x|))$.

In fact, SPACE-BH is **PSPACE**-complete (It follows from the definitions and is left as an exercise) under polynomial-time many-one reductions.

Example 6.2 (The “geography” game). A directed **graph** G (think of it as a map of flights), a distinguished **node** $v \in G$ (think of nodes as cities), **two players** A and B building a **path**.

A selects an edge (v, u_1) , then

B selects an edge (u_1, u_2) , then

A selects an edge $(u_2, u_3) \dots$ etc

It is **prohibited** to visit the same node twice: the player who has to do it, loses.

(NB! It is not about Eulerian or Hamiltonian paths! No need to visit everything.)

Let us formulate our decision problem formally:

Problem: given G, v , tell us whether A has a winning strategy.

Why this problem is in **PSPACE**? As for each such game,

- At each step, the range of moves is limited (neighboring unvisited vertices).
- The length of this game is limited (visit at most n vertices).
- Both the set of “valid moves” and “who wins in the final position” are poly-time computable ($O(n^2)$ for this game).

Therefore, we can build a tree of this game and evaluate it.

Exercise: complete the proof (after you read the rest of this lecture).

Hint: DFS can help.

Exercise (more difficult, but doable): prove it is **PSPACE**-complete. (For this, you will probably need to reduce another **PSPACE**-complete problem to it — we will talk about an important one soon.)

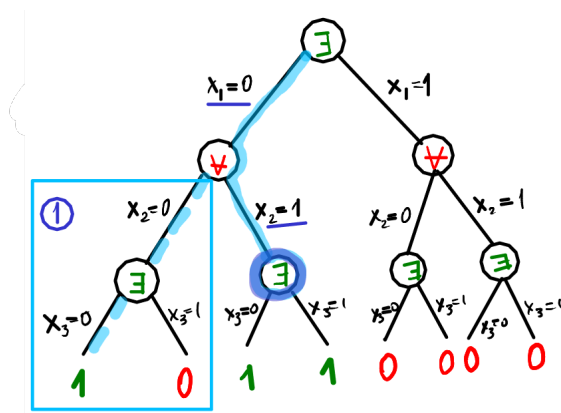
In what follows, we will show that the language QBF (aka QBF-SAT, aka TQBF) that consists of all true quantified Boolean formulas is a **PSPACE**-complete problem.

All x_i 's here are single bits, and Φ has no free variables (i.e., all its variables are quantified — these are x_i 's)!

Theorem 6.3. QBF is **PSPACE**-complete under polynomial-time many-one reductions.

To prove this theorem, we need to show that

1. $\text{QBF} \in \mathbf{PSPACE}$ (in this lecture we prove only this part).
2. QBF is $\mathbf{PSPACE}$ -hard.



²²**Exercise:** it does not matter much: even if this formula is not in CNF, and even if it is given as a circuit, we can easily convert it to CNF using additional quantifiers as in the reduction of **Circuit-SAT** to **SAT**.

Lemma 6.1. $\text{QBF} \in \mathbf{PSPACE}$.

Proof. Let Φ be a quantified Boolean formula, and let us consider its brute-force search tree considering the two values of each variable (x_1, x_2 , etc — see Figure 15). For example, the leaf to which the path 0, 1, 0 leads gets the value of $\Phi(0, 1, 0)$, that is, $\Phi|_{x_1=0, x_2=1, x_3=0}$.

Let us compute $Q_1x_1 Q_2x_2 \dots Q_nx_n \Phi(x_1, x_2, \dots, x_n)$, using depth-first search:

Recursive procedure Eval (input: node v).

1. If v is a leaf, return the value of Φ in it.
2. Let v_1, v_2 be the two children of v . Then
 - (a) Make a recursive call $e_1 := \text{Eval}(v_1)$.
 - (b) If $e_1 = 1$ and the quantifier in v is $\exists$, then return 1.
 - (c) If $e_1 = 0$ and the quantifier in v is $\forall$, then return 0.
 - (d) Make a recursive call $\text{Eval}(v_2)$ and return its result.

Let us estimate the space: How much data do we need to keep during this search? The crucial observation is that when *we come back from a recursive call, all the data that we used inside is forgotten* (it can be overwritten, and the space can be reused), and the depth of our recursion is polynomial (it is the number of Boolean variables).

We need to remember the current path (so that our procedure knows where to return to) and the last-returned value. (Reminder: DFS uses a stack, so if the vertices are ordered, knowing the current path (maximum n) and last two values (children sub-trees results: 2 bits) is sufficient, the above procedure is a bit optimized so a single value is sufficient). So just polynomial (even linear) space is used.

Therefore, indeed $\text{QBF} \in \mathbf{PSPACE}$. □

Corollary 6.3. $\Sigma_k, \Pi_k \subseteq \mathbf{PH} \subseteq \mathbf{PSPACE}$.

In the next lecture we will prove that QBF is $\mathbf{PSPACE}$ -hard (that is, consider an arbitrary $L \in \mathbf{PSPACE}$ and build a reduction from $L \in \mathbf{PSPACE}$ to QBF). The configuration graph of a Turing machine will be used for that.

7 Lecture 7. Scribe: Dotan Danino

7.1 PSPACE, continued

Recall this definition from the previous lecture:

Definition 7.1. For a function $f: \mathbb{N} \rightarrow \mathbb{N}$, define

$$\mathbf{DSpace}[f(n)] = \{L \mid L \text{ is decided by a DTM using space } O(f(n))\}.$$

The union of these classes for all polynomials f is the class

$$\mathbf{PSPACE} = \bigcup_{k \geq 0} \mathbf{DSpace}[n^k].$$

In what follows we will make the definition of **DSpace** more precise, but in the context of polynomial space it is already strict enough.

The nondeterministic version of these classes has been defined by analogy.

7.1.1 QBF: A PSPACE-Complete Problem

Recall that

Definition 7.2. QBF (Quantified Boolean Formula) is the set of true closed quantified CNF formulas:

$$\mathbf{QBF} = \{\text{true formulas of the form } Q_1x_1Q_2x_2\ldots Q_nx_n\Phi(x_1,\ldots,x_n)\}$$

where $Q_i \in \{\forall, \exists\}$.

(Here x_i 's are Boolean variables — exactly those variables that appear in Φ . For example, $\forall x_1 \exists x_2 (x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2})$.)

We continue proving the following theorem:

Theorem (Theorem 6.3). QBF is **PSPACE**-complete (under many-one polynomial-time reductions).

In the previous lecture we proved Lemma 6.1 putting QBF inside **PSPACE**: We considered Φ 's brute-force search tree (where a leaf like $0, 1, 0$ gets the value $\Phi(0, 1, 0)$) and computed the value of the quantified formula using a Depth-First Search (DFS) algorithm. The space required is proportional to the depth of the recursion (the number of variables), which is polynomial.

Thus to prove the above theorem it remains to prove the hardness:

Lemma 7.1. QBF is **PSPACE**-hard. *In fact, we will prove even more: it is NPSpace-hard.*

Proof. Let $L \in \mathbf{PSPACE}$. We will build a reduction from L to QBF. We construct the graph of configurations of the Turing Machine M accepting L . Since M uses $p(n)$ space, there are $O(2^{p(n)})$ different configurations. *Note that if we consider $L \in \mathbf{NPSPACE}$, we still can consider the graph of configurations, now of a NTM — the size bound is the same.*

Recall that in this graph there is a directed edge between two configurations $c_1 \rightarrow c_2$ iff the machine can go from c_1 to c_2 in a single step. This predicate $C(c_1, c_2)$ is easily computable (we need to check that c_1 differs from c_2 only in its state and in the neighborhood of the head position; in the proof of Lemma 3.1 we not only checked it but even computed c_2 from c_1 using a Boolean circuit that we constructed in polynomial time, it was a “single level” of our construction simulating the work of a Turing machine for many steps). Therefore, $C(c_1, c_2)$ can be described as a Boolean circuit of polynomial size. Moreover, it is easily seen from the reduction of **Circuit-SAT** to **3-SAT** in Section 2.4 that it can be converted into a Boolean formula F in CNF with additional variables with existential quantifiers, $\exists w F(c_1, c_2, w)$, the Boolean variables of this formula are distinct bits of c_1 , c_2 , and w (but the latter ones are quantified). This formula can be constructed in polynomial time: the only two things we need to construct it are the space bound (the size of configurations) and the program of M .

Goal: In order to decide $x \in^? L$, it suffices to solve the reachability problem in the configuration graph: to check whether there is a directed path from the starting configuration c_{start} (that includes x on the input tape) to c_{accept} (without loss of generality, **we can assume there is a unique accepting configuration** (specifically, with the head at the first cell and an empty tape)); this assumption is valid because any machine can be modified to erase its tape and reset the head position before entering the accepting state).

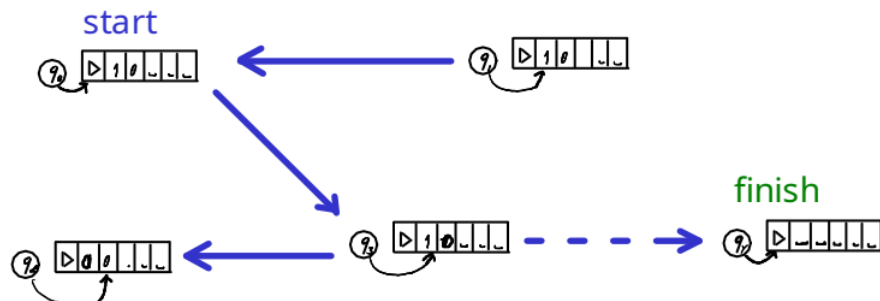


Figure 16: Configuration graph: aiming for reachability.

We will build our quantified Boolean formula expressing this reachability condition recursively. Namely, we will express a bounded version Φ_i of this condition such that

$$\Phi_i(c_1, c_2) = 1 \iff \text{there is a path from } c_1 \text{ to } c_2 \text{ containing at most } 2^i \text{ edges.} \quad (5)$$

Then $\Phi_{p(n)}$ will be our desired formula that our reduction will output.

Recursion Base: We mentioned above how to construct a Boolean formula describing a single step of a Turing machine. This will be our formula $\Phi_0(c_1, c_2)$.

Recursion Step:

Naive Approach: We first describe a naive approach that won't work; then we will repair it.

If we write literally the condition (5), it would give us a formula

$$\Phi_i(c_1, c_2) = \exists d (\Phi_{i-1}(c_1, d) \wedge \Phi_{i-1}(d, c_2)).$$

Indeed, c_2 is reachable from c_1 in $\leq 2^i$ steps iff there is a “middle node” d on this path that is reachable from c_1 in at most $2^i/2 = 2^{i-1}$ steps and such that c_2 is reachable from d in another at most 2^{i-1} steps.

The problem is that Φ_{i-1} appears twice, so if we “unfold” recursively this formula, the formula size will double at each step i . Since the recursion depth is $p(n)$, the final formula size would be $2^{p(n)}$ (exponential), which won't give us a polynomial-time reduction.

Corrected Approach (using additional universal quantifiers): To keep the formula size polynomial, we use universal quantifiers ($\forall$) to collapse the two recursive calls into a single expression. We define:

$$\Phi_i(c_1, c_2) = \exists d \forall x \forall y [((x = c_1 \wedge y = d) \vee (x = d \wedge y = c_2)) \Rightarrow \Phi_{i-1}(x, y)]$$

By using $\forall x \forall y$, we write Φ_{i-1} only once in the definition. This ensures the total size of the formula remains polynomial ($O(p(n)^2)$) if we substitute the definition of $\Phi_{p(n)-1}$ into $\Phi_{p(n)}$, etc (until we arrive at Φ_0).

Conclusion: Our reduction, given x , outputs the formula $\Phi_{p(n)}(c_{start}, c_{accept})$ as described above. All the quantifiers must be moved to the front and the condition must be converted to CNF using additional existential quantifiers. Recall that this condition includes

- a circuit C verifying a single step between two configurations,
- conditions of the form $z = t$ for two configurations z and t that can be expressed bit-by-bit by small circuits (even formulas: $z = t \iff \bigwedge_k z_k \oplus t_k \oplus 1$),
- Boolean operations $\vee, \neg$ (the latter one coming from the implication $\Rightarrow$).

Therefore, like in the reduction of **Circuit-SAT** to **3-SAT** (Section 2.4, Lemma 2.1) one can write an equivalent formula with existential quantifiers.

Note that in this formula only the final part Φ_0 indeed depends on the program of M . The input x appears in the starting configuration that has x on its tape (thus x — whose bits are 0/1-constants and not variables in this formula, these values are given to us on the input! — appears only in the beginning of this recursion, in $\Phi_{p(n)}$). The remaining part of the construction depends on the input *size* only, namely, on the value of $p(n)$, that is, of the polynomial upper-bounding the space usage of M evaluated for $n = |x|$. $\square$

Corollary 7.1 (of this proof). $\mathbf{PSPACE} = \mathbf{NPSPACE}$, and consequently $\mathbf{NPSPACE} = \mathbf{co-NPSPACE}$.

Proof. This is because our reduction did not assume M was deterministic; it works for any $L \in \mathbf{NPSPACE}$. Also $\mathbf{PSPACE} = \mathbf{co-PSPACE}$, because we one can invert the result of a deterministic polynomial-space machine by exchanging its accepting and rejecting states. $\square$

7.1.2 Review of the relations between the classes we considered

See Fig. 17 that illustrates the classes that we have considered so far.

Hypotheses and Facts:

- We believe $\mathbf{PH} \neq \mathbf{PSPACE}$, otherwise the Polynomial Hierarchy ($\mathbf{PH}$) would collapse (since $\mathbf{PSPACE}$ has complete problems, namely QBF, and for $\mathbf{PH}$ that would imply a collapse (Observation 3 in Section 5.5)).
- $\mathbf{PSPACE} \subseteq \mathbf{EXP}$. This is because the configuration graph has an exponential number of nodes, so we can solve reachability using BFS/DFS in exponential time.

Time vs. Space Relations:

- $\mathbf{DSpace}[s(n)] \subseteq \mathbf{DTime}[2^{O(s(n))}]$ (due to the number of configurations).
- $\mathbf{DTime}[f(n)] \subseteq \mathbf{DSpace}[f(n)]$ (Trivial).
- $\mathbf{DTime}[f(n)] \subseteq \mathbf{DSpace}[\sqrt{f(n) \log f(n)}]$ (Ryan Williams, 2025).

7.2 Small Space Classes: L and NL

Definition 7.3 (space complexity — small classes). In order to define small space complexity classes we need to make the definition of space complexity more precise. Namely, we need to consider Turing machines with a **separate read-only input tape** and count the space (the rightmost position of heads) on **working tapes only** (those that can be read and write freely). Otherwise a machine won't be able to read its input!

Remark 7.1. The same note applies to the witness tape if anyone wants to consider (we won't!) a nondeterministic space-bounded machine as a deterministic machine with a witness tape. Moreover, the witness tape must be read left-to-right only, otherwise the head position can be implicitly used for storing additional information thus effectively violating the space restriction.

Let us define logspace classes:

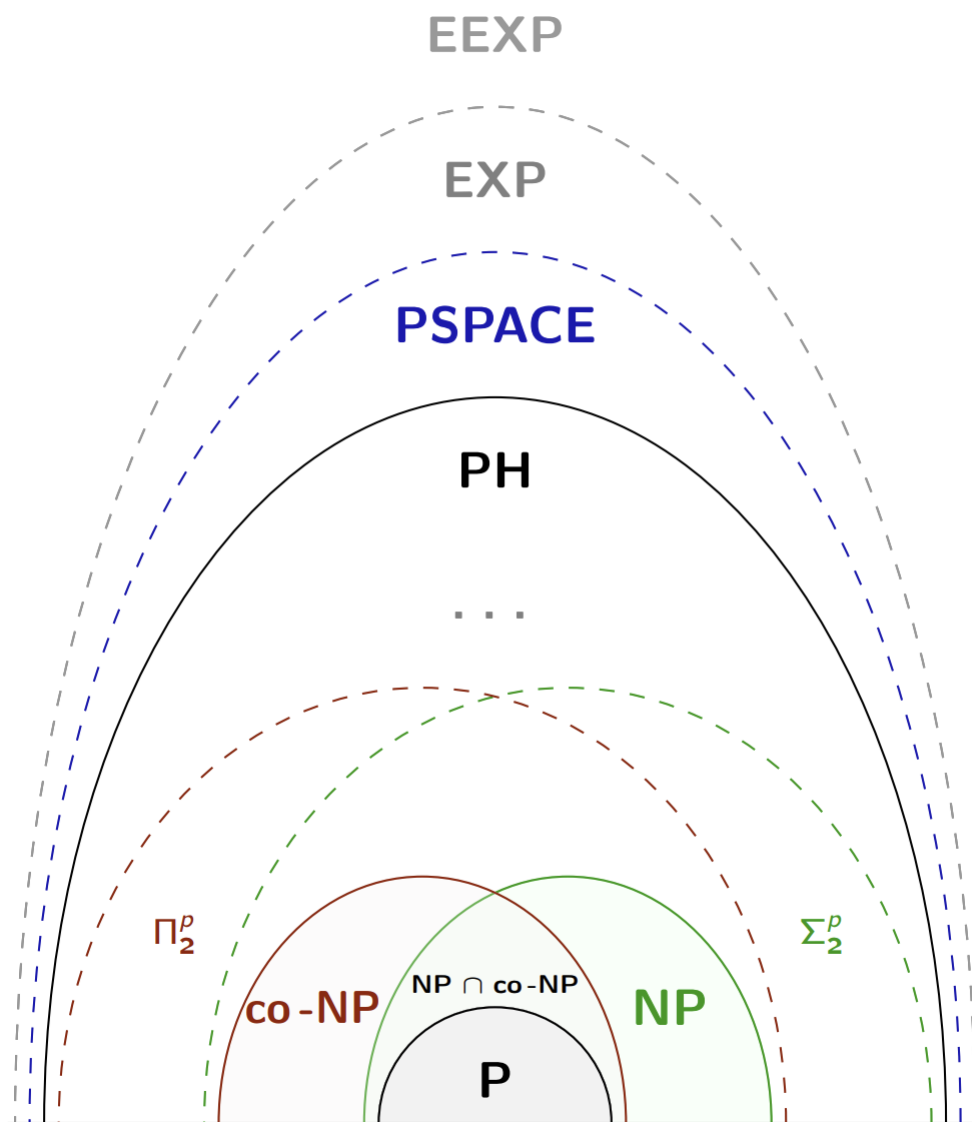


Figure 17: Our current knowledge

- $\mathbf{L} = \mathbf{DSpace}[\log n]$
- $\mathbf{NL} = \mathbf{NSpace}[\log n]$

One can observe that $\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P}$ (the latter inclusion follows from the inspection of the configuration graph of a nondeterministic logspace machine — see below for the details).

Open Questions:

$$\mathbf{L} \stackrel{?}{=} \mathbf{NL} \stackrel{?}{=} \mathbf{P}$$

7.2.1 Counting Configurations in Small Space Classes

Consider a NTM (or a DTM) using space $s(n) \geq \log n$. Note that once we know the input, we do not need to repeat it in each configuration in order to understand where our machine can go at the next step (the input tape is readonly and won't change). Thus it suffices to include into a configuration the following data:

- The state (constant number): $O(1)$ bits.
- Contents of work tapes: $O(s(n))$ bits.
- Head positions on work tapes: $O(\log s(n))$ bits.
- Head position on input tape: $O(\log n)$ bits.

Total: $\leq 2^{O(s(n)+\log n)} = 2^{O(s(n))}$ configurations (assuming $s(n) \geq \log n$).

Corollary 7.2. $\mathbf{NL} \subseteq \mathbf{P}$.

Proof. This is because the number of configurations of a $\mathbf{NL}$ machine is polynomial, thus we can solve the reachability problem in the graph of configurations in polynomial time. $\square$

In fact, in the next lecture we will see that the reachability problem in directed graphs is $\mathbf{NL}$ -complete.

Remark 7.2. Machines using less than $\log n$ space are a bizarre phenomenon.

$$\mathbf{DSpace}[\log \log n] \neq \mathbf{DSpace}[(\log \log n)^{1-\varepsilon}] = \mathbf{DSpace}[O(1)] = \mathbf{REG}.$$

That is, very low space machines are essentially finite automata.

7.2.2 Examples of Logspace Problems

Problems based on a counter. Recall that a DTM can support a binary counter and modify it (add or subtract 1) efficiently without using any additional space. Such a counter takes $\log n$ bits. It helps to compute within logarithmic space function like the input length, the number of 1's in the input, and other functions that do not depend on the order of bits — only on the number of 1's (such as the majority function).

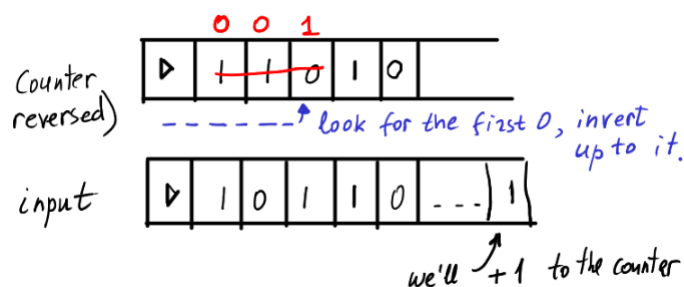


Figure 18: Logspace counter illustration.

BALANCED_FORMULA_EVAL. A formula is balanced if its depth is logarithmic in the number of operations. (Cf. balanced search trees.)

$$\{ \langle \Phi, x \rangle \mid \Phi \text{ is a balanced formula, } \Phi(x) = 1 \}$$

A formula (if we look at its syntactic structure) is a particular case of a circuit — just based on a tree instead of a dag (the only nodes that have outdegree more than one are variables — they can appear many times in a formula).

This tree can be evaluated using DFS, where each recursive step returns a single **output bit** (the value of the subformula) to the parent node. This process requires space proportional to the tree's depth.

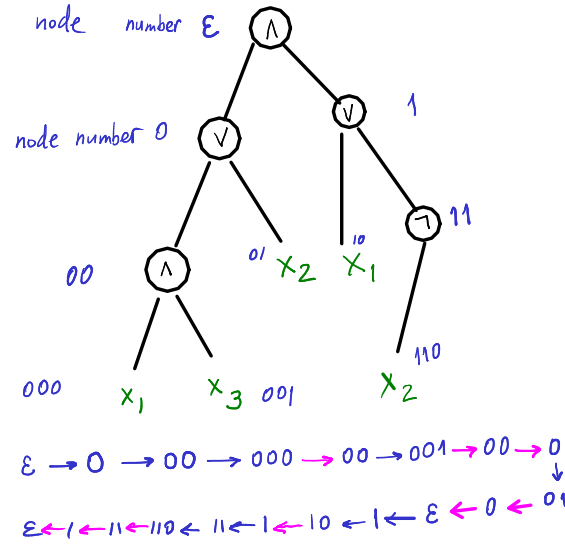


Figure 19: Formula evaluation as a tree traversal.

Note: While this trivially places balanced formulas in **L**, a theorem by **Sam Buss (1987)** shows that even **unbalanced** formulas can be evaluated in Logspace.

In fact, with a certain effort, each formula can be balanced.

7.2.3 Logspace reductions and P-completeness

Definition 7.4. W is **logspace-reducible** to S if there is a logspace-computable many-one reduction from W to S . When counting space used by a DTM implementing such a reduction, we count the space on its working tapes only, not including the readonly input tape and **the write-only write-once unidirectional output tape**.

Definition 7.5 (P-Completeness). A language S is **P-complete** if every $W \in \mathbf{P}$ is many-one logspace-reducible to S .

Theorem 7.1.

1. Logspace reductions are transitive.
2. **L** and **NL** are closed under logspace reductions.
3. If C is **P-complete**, then $C \in \mathbf{L} \iff \mathbf{P} = \mathbf{L}$.

Proof. Item 3 follows from items 1, 2. Item 2 is essentially a particular case of item 1 as we will see from the proof of item 1.

Thus let us prove item 1 (transitivity). We want to compute the composition $f(g(x))$ of two logspace-computable functions f and g .

Problem! The output of $g(x)$ is polynomial in size, so we cannot store the entire string $g(x)$ on a working tape (which is limited to $O(\log n)$), so we cannot just simulate g and then simulate f on g 's output.

Actual algorithm: Instead of storing $g(x)$, we **recompute it** whenever needed:

1. Start simulating the machine for f .
2. Whenever f needs to read the i -th bit of its input (which is the output of g), we **pause f** .
3. We run the machine for $g(x)$ **from scratch!** on another tape.
4. We verify which bit g is currently producing using a counter. We discard all bits except the i -th bit.
5. We pass the i -th bit to f and continue the execution of f .

Conclusion: We only need to store two **counters** for the index i and for the currently computed bit of output of g , which takes logarithmic space. We do not care about the increased running time. $\square$

7.2.4 CIRCUIT_EVAL: A P-complete problem

Theorem 7.2.

$$\text{CIRCUIT_EVAL} = \{\langle C, x \rangle \mid C(x) = 1\} \text{ is } \mathbf{P}\text{-complete.}$$

Proof (Sketch referring to a previous proof).

Membership: $\text{CIRCUIT_EVAL} \in \mathbf{P}$ is obvious.

Hardness: Let $L \in \mathbf{P}$. We know there is a polynomial-time DTM M that decides L . We construct a reduction $x \in L \iff f(x) \in \text{CIRCUIT_EVAL}$. We construct a circuit C that simulates the running of M on input x for $p(n)$ steps.

We constructed this reduction in Lemma 3.1. Let us review it and make sure that this construction is in fact a logspace reduction. Even though the resulting circuit is large (polynomial size), we do not need to store the entire circuit in memory. We can generate the description of the gates and wires on the fly using a few counters (to track the current time step and tape position — these are two “for” cycles). Since counters only require $O(\log n)$ bits, this reduction uses logarithmic space only. $\square$

8 Lecture 8. Scribe: Alok Maity

8.1 Small space complexity classes — reminders

Reminders from the previous lecture:

$\mathbf{DSpace}[f(n)] = \{L \mid L \text{ is decided by a DTM using space } O(f(n)); \text{ where the DTM has a dedicated read-only input tape}\}$,

$\mathbf{NSpace}[f(n)] = \{L \mid L \text{ is decided by a NTM using space } O(f(n)); \text{ where the NTM has a dedicated read-only input tape}\}$,

$\mathbf{L} = \mathbf{DL} = \mathbf{DSpace}[\log n]$,

$\mathbf{NL} = \mathbf{NSpace}[\log n]$.

A **logspace-computable function** is a function computable by a polynomial-time DTM that uses logarithmic space on its working tapes, while its input tape is read-only and its output tape is write-only, write-once and unidirectional.

A **logspace many-one reduction** (or just logspace reduction) is a many-one reduction that is a logspace-computable function.

Last week we learned how to combine several logspace-computable functions in one, in particular, it means that classes $\mathbf{L}$, $\mathbf{NL}$ and other space-bounded classes with at least $\log n$ bound are closed under logspace reductions.

Lecture plan:

- $\mathbf{NSpace}$ classes are closed under complement (this is non-trivial!).
- Small $\mathbf{DSpace}$ and $\mathbf{NSpace}$ classes are interleaved ($\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{DSpace}[\log^2 n]$).
- Parallel computation classes (called $\mathbf{NC}^i$ — we will define them in this lecture) are also related to small space complexity classes.
- More space brings strictly more power (the space hierarchy theorem, and we will learn the time hierarchy theorem as well).

8.2 The reachability problem STCON, with application to space complexity

8.2.1 Solving STCON in quadratic deterministic space

Recall the directed s-t-connectivity, or s-t-reachability, or “directed PATH” problem:

$\mathbf{STCON} = \{(G, s, t) : G = (V, E) \text{ is a directed graph, there is a directed path } s \rightsquigarrow t \text{ in } G\}$.

NB!!! *One can use it also for implicitly defined graphs, such as graphs of configurations of TMs, the only thing we need is a procedure for checking whether a specific edge $(x, y) \in E$.*

Lemma 8.1. $\mathbf{STCON} \in \mathbf{DSpace}[\log^2 n]$.

Proof. We want to define the following predicate:

$$\text{PATH}(x, y, i) = \text{“ there is a path } x \rightsquigarrow y \text{ of length } \leq 2^i \text{”}.$$

Then if we compute $\text{PATH}(s, t, \lceil \log_2 |V| \rceil)$, it will give us the answer to the STCON problem, because if t is reachable from s , it is reachable by a path without (directed) loops, i.e., consisting of at most $n - 1$ edges.

Let us define this predicate inductively (and then write the corresponding recursive procedure):

$$\text{PATH}(x, y, i) = \bigvee_{d \in V} (\text{PATH}(x, d, i - 1) \wedge \text{PATH}(d, y, i - 1))$$

The induction base is for paths of length 0 or 1:

$$\text{PATH}(x, y, 0) = \text{“ } x = y \text{ or there is an edge } (x, y) \in E \text{”}.$$

The induction base can be checked directly by accessing the graph, we do not need to keep anything else than the two nodes x, y (thus $O(\log n)$ bits) in order to find the edge in the graph description.

Now let us compute PATH using a recursive procedure:

Algorithm 1 $\text{PATH}(x, y, i)$, uses global data or a procedure to access the graph

```

1: procedure  $\text{PATH}(x, y, i)$ 
2:   if  $x = y$  then
3:     return True
4:   end if
5:   if  $i = 0$  then
6:     return  $(x, y) \in E$ 
7:   end if
8:   for each vertex  $d \in V$  do
9:     if  $\text{PATH}(x, d, i - 1)$  then
10:      if  $\text{PATH}(d, y, i - 1)$  then
11:        return True
12:      end if
13:    end if
14:  end for
15:  return False
16: end procedure

```

Observe that this algorithm uses only $O(\log^2 n)$ space: each copy of this recursive procedure uses a counter for the vertex d (this is $O(\log |V|)$ bits) and sends $3 \log_2 |V|$ bits

to its recursive calls. When the first recursive call is finished, the corresponding data is thrown away, thus the space used is $O(\log |V|)$ times the recursive depth, which is also $\log |V|$ (we start with $\lceil \log |V| \rceil$ and decrease i by 1 for recursive calls) — total of $O(\log^2 n)$.

(Recall that recursive calls are implemented using a stack: when a call is made, the parameters are put on the stack and space for local variables is allocated there; upon return this information is popped out and thrown away, while the stack can be also used temporarily to transmit the result of the call. A tape of Turing machine can serve as a stack.) $\square$

See the slides for an animated example of the work of this algorithm.

8.2.2 Application of STCON: Savitch's Theorem

The reason why STCON is so important for space computation is that one can apply it to graphs of configurations in order to determine whether a (nondeterministic) Turing machine can reach the accepting configuration from the starting configuration.

Theorem 8.1. STCON is NL-complete.

Proof. STCON $\in$ NL. In order to check STCON nondeterministically using logarithmic space we can simply guess a path from s to t . We guess subsequent nodes of this path one by one and check that the corresponding edge is in the graph; we do not need to keep the portion of the path that we have already checked, thus only the last node is kept (which uses $O(\log n)$ space only).

Here is a pseudocode for this algorithm:

Algorithm 2 NL algorithm for STCON (inputs: $(V, E), s, t$)

```
1: Let  $current \leftarrow s$ 
2: Let  $counter \leftarrow 0$ 
3: Let  $n \leftarrow |V|$ 
4: while  $current \neq t$  and  $counter < n - 1$  do
5:   Nondeterministically guess a vertex  $v \in V$ 
6:   if  $(current, v) \in E$  then
7:      $current \leftarrow v$ 
8:      $counter \leftarrow counter + 1$ 
9:   else
10:    Reject ▷ Bad guess: this computation path fails
11:   end if
12: end while
13: if  $current = t$  then
14:   Accept
15: else
16:   Reject
17: end if
```

STCON is NL-hard (under logspace reductions) Consider $L \in \text{NL}$. There exists a nondeterministic Turing machine M that decides L using $O(\log n)$ space. For an input x of length n , we construct the [configuration graph](#) of this machine $G_M(x) = (V, E)$ as follows:

- V = set of all possible configurations of M on input x .
- $E = \{(c, c') \mid M \text{ can transition from } c \text{ to } c' \text{ in one step}\}$.

A configuration includes:

- The state of M (from its finite set of states),
- The contents of the working tapes (this is $O(\log n)$ in total),
- The position of the heads on the working tapes.
- The position of the head on the input tape (but not the contents of this tape!).

In the previous lecture we have already computed the total number of configurations:

$$|Q| \cdot n \cdot 2^{O(\log n)} \cdot O(\log n) = \text{poly}(n),$$

where $|Q|$ is the number of states. Let c_{start} be the starting configuration and c_{accept} a unique accepting configuration (recall that one can modify M so that it has only one accepting configuration).

To check $x \in L$, we need to check that M accepts x , that is, that there is a directed path from c_{start} to c_{accept} in $G_M(x)$.

Given x , our reduction outputs the graph and these two configurations: $f(x) = (G_M(x), c_{\text{start}}, c_{\text{accept}})$. Then $x \in L \iff f(x) \in \text{STCON}$.

Logspace computability: We must show f is computable by a deterministic Turing machine using $O(\log n)$ work space (with a write-only output tape). Note that the size of a configuration is $O(\log n)$. The machine enumerates all pairs of configurations (c, c') (using $O(\log n)$ -bit counters) and outputs the adjacency matrix (or edge list) of $G_M(x)$ in a write-only mode (that is, it prints them and never comes back to see what it has printed). For each pair (c, c') , we check whether there is an edge (c, c') by looking at the transition function of M . Actually, we are building a reduction for a specific language, thus for a specific machine, so we can even hardwire this transition function into our reduction. Anyway, to check that this transition $c \rightarrow c'$ is legal for M one needs to compare its tapes in c and c' bit by bit making sure that they are identical everywhere except for the place where the head is. The symbol under the head and the new state are looked up in the transition function of M ; one also needs to check that the positions of all the heads changed accordingly to the transition that M performs. (We noted earlier in the course that one can always assume that a nondeterministic machine has only two possible transitions from each configuration, so both are easy to check.)

Therefore, our reduction f can be computed using logarithmic space, that is, we have built a logspace reduction from L to **STCON**. $\square$

Corollary 8.1. $\text{NL} \subseteq \text{DSpace}[\log^2 n]$.

Proof. By the **NL**-completeness of **STCON** (Theorem 8.1), the $\text{DSpace}[\log^2 n]$ -algorithm for **STCON**, and the fact that $\text{DSpace}[\log^2 n]$ is closed under logspace reductions (follows from our construction that combines two logspace-computable functions, Theorem 7.1):

Since **STCON** is **NL-complete**:

1. Any language $L \in \text{NL}$ can be reduced to **STCON** via a logspace reduction.
2. Composing the reduction with our **PATH** algorithm gives: space $O(\log n) + O(\log^2 n) = O(\log^2 n)$.
3. Therefore, $L \in \text{DSpace}[\log^2 n]$.

$\square$

One can easily generalize this corollary:

Theorem 8.2 (Savitch). Let $s(n) \geq \log n$ be a “nice” complexity function (we will later formalize “nice” as “space-computable”, in particular, $s(n)$ itself must be computable within space $O(s(n))$). Then

$$\text{NSpace}[s(n)] \subseteq \text{DSpace}[s(n)^2]$$

Proof (Sketch). Replace the $\log n$ -space NTM in Theorem 8.1 and Corollary 8.1 by a $s(n)$ -space machine. Then the number of configurations will be $2^{O(s(n))}$, the length of the desired path will be $2^{O(s(n))}$, that is, it will be described by $s(n)$ bits. All the counters in the construction of the reduction will be of bit-size $O(s(n))$.

A bit more detailed sketch:

1. We showed that **STCON** (directed s - t connectivity) can be decided by a deterministic Turing machine using $O(\log^2 n)$ space, where n is the number of vertices (or edges — it does not matter under $O(\log \dots)$). This was done via the recursive **PATH** algorithm.
2. For any language $L \in \mathbf{NL} = \mathbf{NSpace}[\log n]$, we reduced L to **STCON** via a logspace reduction that builds the configuration graph of the nondeterministic machine for L . The configuration graph has $\text{poly}(n)$ vertices, so the reduction combined with the $O(\log^2 n)$ algorithm for **STCON** places L in $\mathbf{DSpace}[\log^2 n]$. Thus $\mathbf{NL} \subseteq \mathbf{DSpace}[\log^2 n]$.
3. To generalize to $\mathbf{NSpace}[s(n)]$, consider any $L \in \mathbf{NSpace}[s(n)]$ decided by an NTM M using $O(s(n))$ space. For an input x of length n , the configuration graph $G_M(x)$ has at most $2^{O(s(n))}$ vertices. Running the same recursive path-checking algorithm on this graph requires

$$O\left(\log^2\left(2^{O(s(n))}\right)\right) = O(s(n)^2)$$

space, because storing a vertex (configuration) needs $O(s(n))$ bits and the recursion depth is $O(\log(2^{O(s(n))})) = O(s(n))$. Hence $L \in \mathbf{DSpace}[s(n)^2]$. $\square$

Exercise 8.1. Complete this proof after learning the definition of a space-computable function in the last part of this lecture. You can either follow the sketch or try using the padding technique introduced at the very end of the lecture (but be very accurate with your intermediate data — do not violate the space bound).

Remark 8.1. We have already seen that $\mathbf{PSPACE} = \mathbf{NPSPACE}$ as a corollary of the $\mathbf{NPSPACE}$ -completeness of $\mathbf{QBF} \in \mathbf{PSPACE}$. Savitch's Theorem also implies $\mathbf{PSPACE} = \mathbf{NPSPACE}$. However, note that while $\mathbf{PSPACE} = \mathbf{NPSPACE}$ yields $\mathbf{NPSPACE} = \mathbf{co-NPSPACE}$ because of the symmetry of $\mathbf{PSPACE}$ ($\mathbf{PSPACE} = \mathbf{co-PSPACE}$) and space-bounded classes in general, it does not prove that $\mathbf{NSpace}[s(n)] = \mathbf{co-NSpace}[s(n)]$, as Savitch's theorem has a quadratic overhead: Savitch's theorem: $\mathbf{NSpace}[s(n)] \subseteq \mathbf{DSpace}[s(n)^2]$ implies $\mathbf{co-NSpace}[s(n)] \subseteq \mathbf{NSpace}[s(n)^2]$ because deterministic space classes are closed under complement, but the square is still there. In fact, the equality $\mathbf{NSpace}[s(n)] = \mathbf{co-NSpace}[s(n)]$ is true, but this is the Immerman–Szelepcsényi theorem, which is a nontrivial result we will prove next.

8.3 The Immerman–Szelepcsényi theorem

The Immerman–Szelepcsényi theorem states that $\mathbf{NSpace}(f) = \mathbf{co-NSpace}(f)$ (without squaring the space bound, unlike Savitch's theorem). Since **STCON** is $\mathbf{NL}$ -complete, it suffices to show that $\overline{\mathbf{STCON}} \in \mathbf{NL}$; note that we have already shown $\mathbf{STCON} \in \mathbf{DSpace}[\log^2 n]$ via Savitch's algorithm.

Let us add self-loops to our graph to avoid talking about paths of length $\leq i$ vs $= i$. It does not change the reachability property.

Lemma 8.2. Given a directed graph $G = (V, E)$ with all self-loops (i.e., $(u, u) \in E$ for every $u \in V$), and a vertex $s \in V$, the set of vertices reachable from s can be visited (in particular, counted) by a nondeterministic Turing machine using space $O(\log n)$, where $n = |V|$. In particular, we can determine that a given vertex t is *not* reachable from s .

Proof. Let $n = |V|$. Define:

$$S(k) = \{v \in V \mid \text{there is a path } s \rightsquigarrow v \text{ of length } \leq k\}.$$

Our algorithm will work in stages. At stage k , we will have the number $|S(k-1)|$ computed beforehand. During that stage will visit every node of $S(k)$. We will also count them, and the number $|S(k)|$ (of size $O(\log n)$) will be the only information saved for the next stage $k+1$. (Initially we know $|S(0)| = 1$).

Algorithm 3 Nondeterministic logspace algorithm for $\overline{\text{STCON}}$

```
1:  $|S(0)| \leftarrow 1$  ▷ Only  $s$  itself is reachable in 0 steps
2: for  $k \leftarrow 1$  to  $n - 1$  do
3:   counter_new  $\leftarrow 0$ 
4:   for all  $u \in V$  do
5:     counter_old  $\leftarrow 0$ 
6:     for all  $v \in V$  do ▷ Trying to find a path  $s \rightsquigarrow v \rightarrow u$ 
7:       Nondeterministically guess and verify a path  $s \rightsquigarrow v$  of length  $\leq k - 1$ 
8:       if the guessed path is correct then
9:         counter_old  $\leftarrow$  counter_old + 1 ▷ Found one more vertex in  $S(k - 1)$ 
10:        if  $(v, u) \in E$  then
11:          counter_new  $\leftarrow$  counter_new + 1 ▷ Found one more vertex in  $S(k)$ 
12:          if  $u = t$  then
13:            reject ▷ Found a path to  $t$ 
14:          end if
15:          break the “For  $v$ ” cycle and go to next  $u$ 
16:          ▷  $v$  is found, no need to verify the checksum
17:        end if
18:      end if
19:    end for( $v$ ) ▷ Now  $v$  is not found, we need to verify the checksum: whether  $u$  is
    unreachable or some  $v \in S(k - 1)$  has not been inspected because of a wrong guess?
20:    if counter_old  $\neq |S(k - 1)|$  then ▷ Checksum verification failure
21:      reject ▷ Failed nondeterministic branch
22:    end if
23:  end for( $u$ )
24:   $|S(k)| \leftarrow$  counter_new
25: end for( $k$ )
26: accept ▷ Everything successfully inspected, but no path to  $t$  was found in any  $S(k)$ 
```

During a stage we need to keep only a constant number of counters and vertices, each of size $O(\log n)$. Between the stages only a single number $|S(k)|$ of size $O(\log n)$ is kept. The path is guessed and verified “on the fly”, that is, when guessing and verifying it we keep only the last vertex of the path and forget all previous nodes of the path, thus again only $O(\log n)$ space is used. $\square$

Corollary 8.2. $\text{NL} = \text{co-NL}$.

Proof. Since STCON is NL -complete, $\overline{\text{STCON}}$ is co-NL -complete, and we have proved that $\overline{\text{STCON}} \in \text{NL}$. $\square$

Corollary 8.3. For any space-computable $s(n) \geq \log n$, $\text{NSpace}[s(n)] = \text{co-NSpace}[s(n)]$.

Proof. Exercise — do it after you learn what is space-computable (a couple of subsections below). It is analogous to a similar conclusion (transition from $\log n$ to arbitrary $s(n)$) after Savitch's theorem. $\square$

8.4 Circuits as a computational model and Introduction to parallel computations

8.4.1 Depth is Time

Consider a circuit. If for each gate we have a separate processor that computes the result of the gate, and these processors can work in parallel, then

$$\text{time} = \text{depth}.$$

(The only thing we need to make sure is that the next gate gets the information from the previous gates — by layering the circuit inductively from the input to the output — inputs, gates computed from the inputs, gates computed from these gates (and the inputs), etc — we maintain this condition, and the time will correspond to the longest path from an input to an output, that is, the depth of the circuit.)

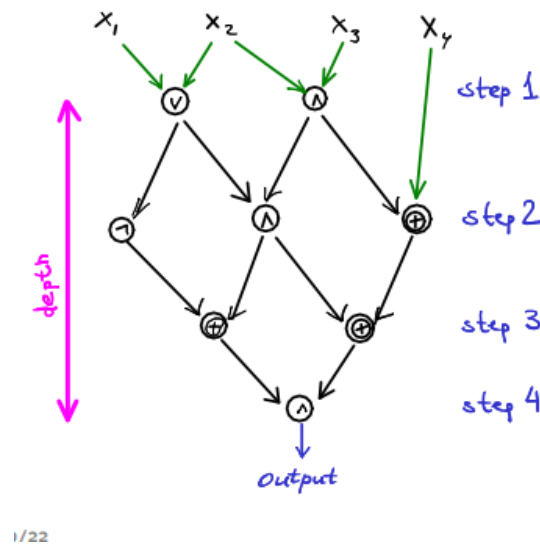


Figure 20: Example of a circuit

The ultimate goal of parallel computing is to parallelize the task as much as possible (if one gives us less parallel processors, we can always spread the load over them accordingly; however, if one gives us more than we expected, there is no standard way to parallelize).

Ideally, we would like to parallelize all polynomial-time computations to achieve $O(\log^i n)$ parallel time!

This motivates the definition of circuit complexity classes corresponding to parallel computation (**NB!!!**: “N” in **NC** is not for nondeterminism — it stands for “Nick’s class” in honor of Nick Pippenger). These classes allow computation by circuits of depth $O(\log^i n)$. However, a (non-uniform) circuit family is not a computer program. To maintain scalability, we need a single algorithm that generates them. Therefore, several notions of uniform circuit families appear — we will come to them after considering some examples.

8.4.2 Examples of parallel algorithms

It is obvious that one can consider a big conjunction, a big disjunction, or a big $\oplus$ in parallel: organize computing $\bigvee_{i=1}^n x_i$ as a binary tree of depth $\lceil \log n \rceil$.

One can use this idea to multiply Boolean matrices (actually, not only Boolean) in parallel. The Boolean product D of two $n \times n$ Boolean matrices B and C has entries:

$$D_{ij} = \bigvee_{k=1}^n (B_{ik} \wedge C_{kj}). \quad (6)$$

One can use Boolean matrix multiplication in order to find paths in directed graphs. Let $G = (V, E)$ be a directed graph with $n = |V|$ vertices. Its adjacency matrix A is defined by:

$$A_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Let us assume that $A_{ii} = 1$, that is, let us add a loop in each vertex — it does not change the reachability property. Then the entry A_{ij} of the Boolean square A^2 (defined as in (6) for $A = B = C$) says whether there is a directed path from i to j of length at most 2. The m -th power A^m of the adjacency matrix corresponds to paths of length $\leq m$:

$$(A^m)_{ij} = 1 \quad \text{if and only if there is a path from } i \text{ to } j \text{ of length at most } m.$$

Since the longest simple path has at most $n - 1$ edges, to determine reachability between all pairs of vertices in G it suffices to compute A^{n-1} or any large power of A .

We can compute high powers efficiently via repeated squaring, which is highly parallelizable:

$$A \rightarrow A^2 \rightarrow A^4 \rightarrow \dots \rightarrow A^{2^{\lceil \log_2(n-1) \rceil}}.$$

After $k = \lceil \log_2(n-1) \rceil = O(\log n)$ squarings, we obtain a matrix that contains all reachability information (because $2^k \geq n-1$).

Each squaring is done as a Boolean matrix multiplication, which can be performed in parallel within depth $O(\log n)$, thus the total depth of the circuit is:

$$O(\log n) \text{ (number of squarings)} \times O(\log n) \text{ (depth per multiplication)} = O(\log^2 n).$$

Hence, the reachability problem (STCON) can be solved in parallel time $O(\log^2 n)$.

It can be easily seen that circuits for this problem are polynomial-size and can be easily printed given n . The definitions given in the next section formalize the notion of such parallel algorithms.

8.4.3 Polynomial-time uniform and logspace-uniform families of circuits

Consider a family of circuits $\{C_n\}_{n \in \mathbb{N}}$. By default circuits C_i and C_j can have nothing in common. We call such families **non-uniform**: they come without an algorithm that would print all these circuits. In particular, such an infinite family can decide even an undecidable language such as the halting problem.

To describe realistic computations, we need uniform families.

Definition 8.1 (polynomial-time uniform circuit family). A family of Boolean circuits $\{C_n\}_{n \in \mathbb{N}}$ is called a **polynomial-time uniform family** if there is a polynomial p and a DTM T that prints each circuit C_n (i.e., $T(1^n) = C_n$) in time $p(n)$.

Theorem 8.3.

$L \in \mathbf{P} \iff$ there exists a polynomial-time uniform circuit family for L .

Proof. $\Rightarrow$ is Lemma 3.1. $\Leftarrow$: to use this family to decide L , read the input x , determine its size n , use $T(1^n)$ to print C_n , then evaluate C_n on x — all this takes polynomial time. $\square$

For parallel computation we will need a more strict condition of logspace-uniformness (we will see then why: parallel computation classes are tightly related to logspace-bounded classes).

Definition 8.2 (logspace-uniform circuit family). A family of Boolean circuits $\{C_n\}_{n \in \mathbb{N}}$ is called a **logspace-uniform family** if there is a DTM T that prints each circuit C_n (i.e., $T(1^n) = C_n$) using space $O(\log n)$ and polynomial time.

Definition 8.3.

$$\begin{aligned} \mathbf{NC}^i &= \{L : \text{there exists a logspace-uniform family of } O(\log^i n)\text{-depth circuits for } L\}. \\ \mathbf{NC} &= \bigcup_{i \in \mathbb{N}} \mathbf{NC}^i. \end{aligned}$$

Note that this definition implies that $\mathbf{NC} \subseteq \mathbf{P}$: one can generate a circuit in logarithmic space and polynomial time and then evaluate this polynomial-size circuit on the input. It is unknown whether these two classes coincide: it would mean that we could parallelize all the polynomial-time computations!

8.4.4 Relations between parallel computation classes and other classes

Theorem 8.4. $\mathbf{NC}^1 \subseteq \mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{NC}^2$.

Proof. The second containment $\mathbf{L} \subseteq \mathbf{NL}$ is trivial, the last containment $\mathbf{NL} \subseteq \mathbf{NC}^2$ follows from the example in Section 8.4.2: To solve STCON (which is $\mathbf{NL}$ -complete), we can repeatedly square the adjacency matrix of the input graph (using Boolean matrix multiplication), the depth of this circuit is $O(\log^2 n)$, and these circuits can be easily generated given the value of n (only a constant number of counters are needed).

The first containment is left as an [Exercise](#).

Sketch: We will combine (compose) three logspace-computable functions according to Theorem 7.1. Let $L \in \mathbf{NC}^1$. There exists a logspace-uniform family of circuits $\{C_n\}$ of depth $O(\log n)$ that decides L . Thus the first function will read the input x and generate C_n for $n = |x|$.

Now observe that if one expands a log-depth circuit (dag) into a formula (tree), the blowup won't be exponential (because of the depth). Thus our second logspace-computable function will read the circuit and output a log-depth formula (details are left as an exercise).

Now the third logspace-computable function will evaluate this formula on x . □

Theorem 8.5. $\mathbf{NC}$ is closed under logspace reductions. In particular, if S is $\mathbf{P}$ -complete, then $S \in \mathbf{NC} \iff \mathbf{P} = \mathbf{NC}$.

Proof. Let R be a logspace reduction that reduces A to $B \in \mathbf{NC}$. There is a DTM T that prints C_n for B using space $O(\log n)$, these circuits C_n are of depth $O(\log^i n)$ for some specific i .

Since R is computable in logspace, we can compute R by a logspace-uniform family of multiple-output circuits D_n of depth $O(\log^2 n)$ (it follows from the last inclusion in Theorem 8.4 by separating output bits of R into languages $\{\langle x, i \rangle : (R(x))_i = 1\}$ and then generating a separate circuit for each $i \leq |R(x)|$).

It remains to connect the output bits of D_n to the input bits of C_n . This “composition” $D_n \circ C_n$ can be printed by a logspace machine (printing D_n , C_n , and the edges between them), and the total depth is the sum of the depth of D_n and the depth of C_n (in any case it is still polylogarithmic).

The second statement follows from the first one. Indeed, let S be $\mathbf{P}$ -complete.

($\Rightarrow$) Suppose $S \in \mathbf{NC}$. Since every $A \in \mathbf{P}$ logspace-reduces to S , $\mathbf{NC}$ is closed under logspace reductions, we obtain $A \in \mathbf{NC}$.

($\Leftarrow$) If $\mathbf{P} = \mathbf{NC}$, then trivially $S \in \mathbf{P} = \mathbf{NC}$. □

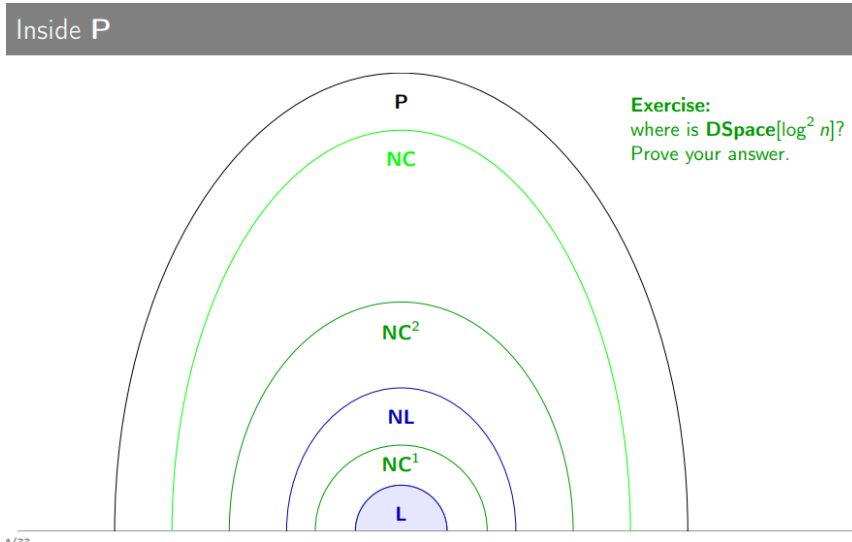


Figure 21: Inside P

8.5 The space hierarchy theorem (and other hierarchy theorems)

At last we are going to prove that some complexity classes differ. We will do it first for space-bounded classes that use a different amount of space and then will talk about other similar settings.

These proofs use diagonalization, that is, we will run a machine on itself and invert the answer. In order to run an arbitrary machine we use a universal Turing machine. Let us talk about fine details of its construction (or rather of the overhead compared to the original machine).

8.5.1 A few words about universal Turing machines

Theorem 8.6 (Universal DTM). There is a DTM U such that $\forall x \forall \text{DTM } M \quad U(\langle M, x \rangle) = M(x)$ and

1. U uses $O(\text{space used by } M(x))$,
2. U uses time $O(T(|x|) \log T(|x|))$, where $T(n) \geq n$ is the running time of $M(x)$.

Proof. 1. Recall the construction of a universal Turing machine. It keeps the memory of the simulated machine on its tape, but it does not need much space in addition: just the machine itself and several counters.

2. We will not prove it in this course. (But our time hierarchy theorem will use this formulation; one could use a slightly weaker formulation and obtain a weaker time hierarchy.) $\square$

8.5.2 The space hierarchy theorem

We will use the classes $\mathbf{DSpace}[f(n)]$, $\mathbf{DTime}[f(n)]$, etc only for “nice” functions $f(n)$. If the class cannot compute the function $f(n)$ itself, it is a very strange class (essentially, it cannot use this amount of resources, because it does not know when to stop). It leads to the following definition.

Definition 8.4. A function $f : \mathbb{N} \rightarrow \mathbb{N}$ is called **space-constructible** if:

- $f(n) \geq \log n$,
- $f(n)$ is non-decreasing,
- $f(n)$ is computable by a DTM within space $O(f(n))$ (i.e., there is a DTM that on input $|x|$ outputs $f(|x|)$ and uses space $O(f(n))$ only).

We consider $\mathbf{DSpace}[f(n)]$ only for space-constructible f .

NB!!! The space and time hierarchy theorems have nothing to do with the polynomial hierarchy! Space and time hierarchies are known to be strict and they have nothing to do with the number of quantifiers — they talk about specific bounds on the resource (space or time).

Theorem 8.7 (The space hierarchy theorem). Let $s(n)$ and $S(n)$ be space-constructible functions such that $s(n) = o(S(n))$ and $S(n) \geq \log n$. Then

$$\mathbf{DSpace}[s(n)] \subsetneq \mathbf{DSpace}[S(n)].$$

Proof. We use a diagonalization argument. Define

$$L = \left\{ x = \langle M \rangle 01^k \mid \begin{array}{l} |\langle M \rangle| < S(|x|), \\ M \text{ is a DTM that stops and rejects } x \text{ using space } \leq S(|x|) \end{array} \right\}.$$

Note that the description of M and the natural number k can be easily decoded from $\langle M \rangle 01^k$ (read this string right-to-left until you see the first zero), and that if k is large enough, then the space restriction and the technical requirement on the length of $\langle M \rangle$ space restriction are satisfied.

1. Let us prove that $L \in \mathbf{DSpace}[S(n)]$.

We design a DTM that decides L using $O(S(n))$ space:

1. Compute $S(|x|)$ — we can do it using space $O(S(n))$ by space-constructibility.
2. Decode M .
3. Check the condition $|\langle M \rangle| < S(|x|)$.

4. Simulate M on input x using at most $S(|x|)$ space using a universal Turing machine (Theorem 8.6): the space can be controlled, for example, by putting a “barrier” (special symbol) on the working tape at the position $S(|x|) - 1$ or by maintaining a counter for the head position.
5. If the simulation uses more than $S(|x|)$ space, reject.
6. Otherwise, if M rejects x , accept; if M accepts x , reject.

2. Let us prove that $L \notin \mathbf{DSpace}[S(n)]$.

Assume for the sake of contradiction that $L \in \mathbf{DSpace}[s(n)]$. It means that there exists a function $s_*(n) = O(s(n))$ ($= o(s(n))$) and a DTM M_* that decides L using space at most $s_*(n)$. Let us choose k (that is, essentially the length of the input) large enough. Since s_* is asymptotically inferior to S , there exists a number N_1 such that for all $n > N_1$, $s_*(n) < S(n)$. Let $N_* > \max\{N_1, 2^{\langle M_* \rangle}\}$ and consider the corresponding input of this length:

$$x_* = \langle M_* \rangle 01^{N_* - \langle M_* \rangle - 1}.$$

By construction the condition $\langle M_* \rangle < S(N_*)$ is satisfied (recall that $S(n) \geq \log n$). The condition that M_* stops on x_* without using space more than $S_*(|x_*|)$ is also satisfied by the choice of N_* . Thus the question whether x_* satisfies the conditions of our language L is exactly the question whether M_* rejects x_* or not.

If M_* rejects it, the condition is satisfied, and $x_* \in L$, but by our assumption M_* decides L , so if it rejects x_* , then $x_* \notin L$ — a contradiction.

Similarly, if M_* accepts x_* , the condition is falsified, and $x_* \notin L$, but by our assumption M_* decides L , so if it accepts x_* , then $x_* \in L$ — a contradiction.

Thus both possibilities lead to a contradiction, and our assumption (the existence of an efficient DTM M_* for L) was wrong. $\square$

Remark 8.2. In fact, one can prove this theorem for $S(n) = \lceil \log \log n \rceil$ and $s(n) = O(1)$. However, $\mathbf{DSpace}[(\log \log n)^{1-\varepsilon}] = \mathbf{DSpace}[O(1)] = \mathbf{REG}$. We will not prove it in this course.

8.5.3 The time hierarchy theorem

Although this chapter on *space* complexity may be not the right place for the *time* hierarchy theorem, it is now straightforward. The only difference from the space hierarchy theorem is in the parameters, it is due to the fact that UTM is less efficient w.r.t. time.

“Nice” functions bounding the time are called time-constructible:

Definition 8.5. A function $f(n)$ is called **time-constructible** if:

- $f(n) \geq n$,

- $f(n)$ is non-decreasing,
- $f(n)$ is computable by a DTM within time $O(f(n))$.

We consider the classes $\mathbf{DTime}[f(n)]$ only for time-constructible f .

Theorem 8.8 (The time hierarchy theorem). Let $t(n)$, $T(n)$ be time-constructible. Then

$$\mathbf{DTime}[t(n)] \neq \mathbf{DTime}[T(n)], \quad \text{where } \frac{t(n) \log t(n)}{T(n)} \rightarrow 0.$$

Proof. Similarly to the space hierarchy theorem proof, define the language

$$L = \left\{ x = \langle M \rangle 01^k \mid \begin{array}{l} |\langle M \rangle| < T(|x|), \\ \mathbf{UTM} \text{ rejects } \langle M, x \rangle \text{ using time } \leq T(|x|) \end{array} \right\}.$$

First, note that $L \in \mathbf{DTime}[T(n)]$ because we have bounded the UTM (not M 's) time by $T(|x|)$ and the computation of $T(|x|)$ can be done in time $O(T(|x|))$ because of the time-constructibility of T .

Assume for the sake of contradiction that $L \in \mathbf{DTime}[t(n)]$, i.e., there exists a DTM M_* that decides L in time $t_*(n) = O(t(n))$.

Let us denote a specific constant in the time complexity of UTM (Theorem 8.6) simulating M_* by c , that is, our UTM simulates M_* in time $ct_*(n) \log t_*(n)$. Since $\frac{t(n) \log t(n)}{T(n)} \rightarrow 0$, we also have $\frac{t_*(n) \log t_*(n)}{T(n)} \rightarrow 0$. Hence there exists N_1 such that for all $n > N_1$,

$$ct_*(n) \log t_*(n) < T(n).$$

Choose $N_* > \max\{N_1, |\langle M_* \rangle|\}$ and construct the input

$$x_* = \langle M_* \rangle 01^{N_* - |\langle M_* \rangle| - 1},$$

so that $|x_*| = N_*$. By construction, $|\langle M_* \rangle| < T(N_*)$. Since UTM is efficient enough (Theorem 8.6), our choice of N_* (and N_1) ensure that time $T(n)$ suffices to simulate M_* until it stops. Therefore, the condition in the language L essentially says that M_* rejects x_* — all other conditions are satisfied.

Now consider the computation of M_* on x_* :

- If M_* accepts x_* , then by definition of L , $x_* \notin L$. But since M_* decides L , this implies that M_* must reject x_* — a contradiction.
- If M_* rejects x_* , then by definition of L , $x_* \in L$. But since M_* decides L , this implies that M_* must accept x_* — a contradiction.

Therefore no such M_* exists, and $L \notin \mathbf{DTime}[t(n)]$ while $L \in \mathbf{DTime}[T(n)]$. This proves the strict separation. $\square$

8.5.4 The nondeterministic time hierarchy theorem

Theorem 8.9. Let t be time-constructible. Then

$$\mathbf{NTime}[t(n)] \neq \mathbf{NTime}[T(n)],$$

where $t(n+1) = o(T(n))$ and both $t(n)$ and $T(n)$ are time-constructible.

(Without proof in this course.)

8.5.5 Corollaries

From the space and time hierarchy theorems we obtain:

- $\mathbf{L} \neq \mathbf{PSPACE}$.
- $\mathbf{P} \neq \mathbf{EXP}$.
- $\mathbf{NP} \neq \mathbf{NEXP}$.

8.6 Padding

Now as we have started to talk about such classes as $\mathbf{EXP}$ and $\mathbf{NEXP}$, let us think about the following two open questions:

$$\mathbf{P} \stackrel{?}{=} \mathbf{NP} \quad \text{vs} \quad \mathbf{EXP} \stackrel{?}{=} \mathbf{NEXP}$$

How are they related?

This and similar questions can be resolved using the “padding” technique that transfers languages solvable using a certain amount of resources to smaller classes (that use fewer resources) by artificially enlarging the input with “padding” (i.e., $x01^k$ instead of x).

Theorem 8.10. $\mathbf{P} = \mathbf{NP} \Rightarrow \mathbf{EXP} = \mathbf{NEXP}$.

Proof. Assume $\mathbf{P} = \mathbf{NP}$. Take any language $L \in \mathbf{NEXP}$. Then there exists a nondeterministic Turing machine N that decides L in time $2^{p(n)}$ for some polynomial p .

Define the padded version of L :

$$L_{\text{pad}} = \left\{ x 01^{2^{p(|x|)} - |x| - 1} \mid x \in L \right\}.$$

The string $x 01^{2^{p(|x|)} - |x| - 1}$ has length exactly $2^{p(|x|)}$. On input $y = x 01^{2^{p(|x|)} - |x| - 1}$ one can:

1. Check that y has the correct form (i.e., ends with the appropriate number of 1’s after a 0).

2. Simulate N on x for $2^{p(|x|)}$ steps.

Because the simulation takes deterministic time $2^{O(p(|x|))}$ and the input length is $2^{p(|x|)}$, the overall time is polynomial in the input length of y . Hence $L_{\text{pad}} \in \mathbf{NP}$.

By the assumption $\mathbf{P} = \mathbf{NP}$, we have $L_{\text{pad}} \in \mathbf{P}$; let M be a deterministic polynomial-time machine deciding L_{pad} with running time $q(m)$, where m is the input length.

Now we can decide L in deterministic exponential time as follows:

- On input x , construct $y = x 01^{2^{p(|x|)} - |x| - 1}$.
- Run $M(y)$ and output the result.

The construction of y takes time $O(2^{p(|x|)})$. Running $M(y)$ takes time $q(|y|) = q(2^{p(|x|)})$, which is exponential in $|x|$ because q is a polynomial. Hence $L \in \mathbf{EXP}$.

Thus $\mathbf{NEXP} \subseteq \mathbf{EXP}$, and since $\mathbf{EXP} \subseteq \mathbf{NEXP}$ trivially, we obtain $\mathbf{EXP} = \mathbf{NEXP}$. $\square$

Note that it means that $\mathbf{EXP} \neq \mathbf{NEXP} \implies \mathbf{P} \neq \mathbf{NP}$, i.e., padding translates equalities upwards and non-equalities downwards.

Exercise 8.2. Try using the padding technique for inferring statements about an arbitrary complexity function $s(n) \geq \log n$ from statements about $s(n) = \log n$ (we had several statements like that earlier in this lecture). Observe the necessity of computing $s(n)$ within space $O(s(n))$ for doing padding.

9 Lecture 9. Scribe: Yohann Pardes

This lecture is devoted to randomized computation, but before that we start with a small appendix to the previous lecture — the topic of $\mathbf{NC}^i$ (Sect. 8.4).

9.1 A postscriptum to the previous lecture: Arithmetic circuits

We studied $\mathbf{NC}^i$ — logspace-uniform circuits of depth $O(\log^i n)$, which are a model of parallel computations. There is a related notion:

Definition 9.1. An **arithmetic** circuit is, like a standard Boolean circuit, a directed acyclic graph. The difference is that the operations are $\wedge$ and $\vee$ of arbitrary arity; therefore, the indegree of a node may be arbitrary (not just binary or unary). One can also use the unary negation operation $\neg$.

A language belongs to the class $\mathbf{AC}^i$ if there is a family of arithmetic circuits of depth $O(\log^i n)$ deciding it.

$$\mathbf{AC} = \bigcup_{i \in \mathbb{N}} \mathbf{AC}^i.$$

Note that this name is a bit misleading: circuits that really use arithmetic operations like $+$ and $\cdot$ are usually called algebraic.

NB!!! In the literature, $\mathbf{NC}^i$ typically refers to logspace-uniform families, while $\mathbf{AC}^i$ usually refers to non-uniform families. However, both notations can be used in a different way, so it is advised to check what is assumed by default in a specific article or book.

Let us compare $\mathbf{NC}^i$ and $\mathbf{AC}^i$ classes. Of course, to compare them one needs to consider the same kind of families (uniform or non-uniform).

Theorem 9.1. Let us consider, for clarity, non-uniform families of circuits. For every $i \geq 0$,

$$\mathbf{NC}^i \subseteq \mathbf{AC}^i \subseteq \mathbf{NC}^{i+1}.$$

In particular, $\mathbf{NC} = \mathbf{AC}$.

Proof. $\mathbf{NC}^i \subseteq \mathbf{AC}^i$ is almost obvious: while $\mathbf{NC}^i$ circuits can use all binary operations (and not just $\wedge$ and $\vee$), they can be easily converted to the de Morgan basis with a linear increase in depth and size (for example, replace $x \oplus y$ by $(x \wedge \neg y) \vee (\neg x \wedge y)$: since we are talking about circuits and not formulas, one can simply reconnect the incoming and outgoing wires of the former $\oplus$ gate without any other change).

$\mathbf{AC}^i \subseteq \mathbf{NC}^{i+1}$ is clear from the example of big- $\wedge$ and big- $\vee$ that we considered in the previous lecture: one can compute a k -ary symmetric operation like that as a binary tree of depth $\lceil \log_2 k \rceil$; thus, the depth will be at most multiplied by another $\log n$, and depth- $\log^i n$ circuits will become depth- $\log^{i+1} n$. $\square$

Remark 9.1. It is important that $\mathbf{AC}$ -circuits are defined over the de Morgan basis $\{\wedge, \vee, \neg\}$. One cannot encode a **big- $\oplus$** gate using $\wedge$, $\vee$, and $\neg$ without a drastic increase in size or depth. In fact, Håstad's theorem says that the parity function $\bigoplus_{i=1}^n x_i$ has no constant-depth arithmetic circuits (using $\wedge, \vee, \neg$) of polynomial size.

9.1.1 Randomized (probabilistic) computation

In the lecture we will dive into the following classes:

- **ZPP**: Randomized polynomial time with zero error,
- **RP**: Randomized polynomial time with one-sided error, for “yes” instances.
- **BPP**: Randomized polynomial time with two-sided error.

9.1.2 Introduction to randomized algorithms

First let us discuss the notion of a randomized algorithm.

- Let us talk about randomized algorithms for decision problems (thus the answer will be yes or no — yet sometimes it will be incorrect).

Note: We are not talking about approximation algorithms! The answer is exact... but may be wrong.

- We have access to a source of uniformly random independent bits (aka “random string”). We can consider it as a dedicated readonly tape containing random bits.
- A good randomized algorithm can give a wrong answer with a “small” probability.
- There are multiple types of errors:
 - “False negative”: the correct answer is YES, but we say NO. We will later define such algorithms that make only this type of error (thus error is one-sided) as **RP**-algorithms.
 - “False positive”: the correct answer is NO, but we say YES. (Thus **co-RP**.)
 - Both false negative and false positive answers. Thus error is two-sided. We will define another class, **BPP**, for them.
 - Zero error: unlike the previous classes, the randomness here influences the running time of the algorithm, not the correctness of the algorithm’s answer. We will define the corresponding class **ZPP**.

9.2 One-sided bounded error

Note: Our algorithms are probabilistic Turing machines with access to an infinite random tape. Algorithms that stop in polynomial time, of course, use only a polynomial portion of that tape. We will later re-define these classes rigorously using polynomial-time verifiable relations like we did with **NP**.

Definition 9.2. A **one-sided bounded-error** algorithm A for a language L must satisfy the following conditions:

- $\forall x \in L \quad \Pr\{A(x) = 1\} \geq \frac{1}{2} \quad // \text{A false negative can occur with prob. } \leq \frac{1}{2}$
- $\forall x \notin L \quad A(x) = 0 \quad // \text{No false positives}$

So, in other words, the machine may reject some strings in L (false negatives), but it never accepts strings not in L (no false positives).

Definition 9.3 (**RTime** $[t(n)]$). $L \in \text{RTime}[t(n)]$ if there is a $O(t(n))$ -time one-sided bounded-error algorithm for it (with no false positives).

NB!!! The running time bound is guaranteed, it does not depend on random bits! The algorithm must always stop within this time bound.

Definition 9.4 (RP).

$$\mathbf{RP} = \bigcup_{k \in \mathbb{N}} \mathbf{RTime}[n^k]$$

9.2.1 Example: Verifying matrix multiplication

Matrix multiplication of $n \times n$ -matrices can be done by definition using $O(n^3)$ arithmetic operations

$$(AB)_{ij} = \sum_{k=1}^n A_{ik}B_{kj} \quad n \text{ multiplications and } n-1 \text{ additions for each of the } n^2 \text{ entries}$$

or slightly better (the currently best algorithm due to Vassilevska-Williams et al uses $O(n^{2.3715\dots})$ arithmetic operations.

Can one *verify* faster the result of matrix multiplication that someone has performed for us? Certainly not faster than $O(n^2)$ because we need to read the matrices, but can we do it in time $O(n^2)$? It is unknown how to do it deterministically (of course, one can just recalculate the result and compare, but it will be too slow). Can we do it probabilistically?

Yes!

The following randomized (**co-RTime**) algorithm (false positives only) is due to R. Freivalds. It uses $O(n^2)$ arithmetic operations in order to check the correctness of matrix multiplication.

Input: $n \times n$ matrices A, B, C .

Output: ‘‘Yes’’ if $A \cdot B = C$, otherwise ‘‘No’’.

Overview: We will “hash” matrices using a product by a random vector. In other words, we will compute the sum of a random subset of columns and check that these vectors coincide. Because of associativity we won’t need to multiply matrices — just a matrix and a vector.

-
1. Sample a random vector $r \in \{0, 1\}^n$.
 2. Compute vectors $e_1 = A(Br)$ and $e_2 = Cr$ $//O(n^2)$ operations
 3. If $e_1 = e_2$, then accept; else reject.
-

In order to prove that the algorithm belongs to **co-RTime** we need to show:

1. The algorithm always accept if $x \in L$: obvious, if $AB = C$, then $e_1 = e_2$.
2. The algorithm might accept $x \notin L$ only with a small probability.

To prove the second item, assume that $A \cdot B \neq C$ and estimate the probability that nevertheless $e_1 = e_2$.

Let $D := AB - C$. This is a nonzero matrix, that is, a matrix with nonzero entry, say, $D_{p,s}$. The probability of false positive is:

$$\begin{aligned} \Pr_r\{e_1 = e_2\} &= \Pr_r\{A(Br) = Cr\} = \Pr_r\{(AB)r = Cr\} = \Pr_r\{(AB - C)r = 0\} \\ &= \Pr_r\{Dr = 0\} = \Pr_r\left\{\sum_{i=1}^n D_{p,i}r_i = 0\right\} = \Pr_r\left\{-\sum_{i \neq s} D_{p,i}r_i = D_{p,s}r_s\right\} \leq \frac{1}{2}. \end{aligned}$$

To see the last inequality, recall that r_i 's are independent, thus we can think that $r_s \in \{0, 1\}$ is chosen after $\sum_{i \neq s} D_{p,i}r_i$ is computed; since r_s has two possible values, the probability that it will match $D_{p,s}^{-1} \sum_{i \neq s} D_{p,i}r_i$ is at most $1/2$.

Intuition:

What is the probability that there is some error in the matrix multiplication (hence a nonzero spot in D) and we are still getting 0 when computing Dr . There are two options:

1. $\sum_{i \neq s} D_{p,i}r_i = 0$, then r_s must be 0.
2. $\sum_{i \neq s} D_{p,i}r_i = c \neq 0$, then r_s must be 1 (and even this will help only if $D_{p,s} = -c$).

Since r_s is selected uniformly at random, the upper bound of the probability is at most $\frac{1}{2}$.

9.2.2 Probability amplification: the case of one-sided error

Our algorithm error is bounded just by $\frac{1}{2}$. It looks unacceptable in practice, how can we improve on this?

Answer: We can perform probability amplification (amplify the success probability, i.e., lower the error probability).

Let A be a **RTime** algorithm for L with error probability $1/2$. Let us repeat it (using independent random bits for each attempt):

Algorithm B .

Input: x .

```

For i:=1 to k
--- if A(x) = 1 then accept.
Reject.                                     //If all attempts failed.
```

If $s \in L$ then

$$\Pr\{B(x) = 0\} \leq \Pr\{A(x) \text{ has made an error } k \text{ times}\} \leq \frac{1}{2^k}.$$

Of course, if $s \notin L$, $B(x) = A(x)$.

If A runs in polynomial time and k is polynomial, the running time of B is still polynomial!

We can further generalize it even if we have an error probability $1 - \frac{1}{T}$. If we make $k = T$ trials, the overall error probability of B turns into $(1 - \frac{1}{T})^T < \frac{1}{e}$.

It gives rise to the following paradigm: if we want to amplify the success probability from very small $\varepsilon(n)$ by repetition, $O(1/\varepsilon(n))$ iterations will suffice, thus the running time will be $O(t(n)/\varepsilon(n))$ if the original algorithm A runs in time $O(t(n))$.

In particular, if A is polynomial-time and $\varepsilon(n) = n^{-const}$, then we can lower the error probability to an exponentially small one while staying in polynomial time.

9.2.3 Example: Polynomial Identity Testing

An [algebraic](#) circuit operates over integers (or another ring or field) and uses the addition and multiplication operations. Its source nodes are inputs (each one now having a meaning of a number and not a single bit) and (integer) constants. **Let us assume for simplicity that these constants are 0, 1, and -1.** Such a circuit C naturally computes a polynomial $p_C \in \mathbb{Z}[x_1, \dots, x_n]$.

The Polynomial Identity Testing ([PIT](#)) is the problem of checking whether two algebraic circuits C and D compute the same polynomial, i.e., the coefficients of p_C and p_D are exactly the same.

This problem can be easily reducible to another “easier” problem, **ZeroP**, which asks whether the input circuit computes the zero polynomial (all its coefficients are 0). PIT easily reduces to **ZeroP**: given two circuits C and D , consider the circuit $E(\vec{x})$ computing $C(\vec{x}) - D(\vec{x})$: take the output of the first circuit and add to it the output of the second circuit multiplied by -1 . The resulting circuit computes the zero polynomial iff the two circuits computed the same polynomial.

It is an important open problem to check PIT deterministically in polynomial time. Let us consider the following randomized algorithm for **ZeroP**:

Input: an algebraic circuit C .

1. Select random values $r_1, \dots, r_n$ for $x_1, \dots, x_n$ (the input variables of C).
 2. Compute $C(r_1, \dots, r_n)$.
 3. If $C(r_1, \dots, r_n) = 0$, then answer yes; otherwise answer no.
-

This algorithm obviously cannot have any false negatives. It remains to decide from what set to pick the values of r_i 's and what will be the resulting probability of error.

For a univariate polynomial we know that its number of roots is at most its degree. The following lemma generalizes this statement to the case of multiple variables:

Lemma 9.1 (Schwartz–Zippel). Let $\mathbb{F}$ be a field (it works also for $\mathbb{Z}$ even though it is not a field). Let $0 \neq p \in \mathbb{F}[x_1, \dots, x_n]$, let the total degree²³ $\deg p \leq d$. Let $S \subseteq \mathbb{F}$. Then

$$\Pr[p(r_1, \dots, r_n) = 0] \leq \frac{d}{|S|}.$$

It is not difficult to prove this lemma by induction, but we have not proved it during the lecture.

In other words, if we pick random values from a large enough set S , then our risk to hit a root, i.e., to evaluate a nonzero polynomial p to zero, is bounded by $\frac{\deg p}{|S|}$.

Therefore if we choose $0 \leq r_i \leq s \cdot \deg p_C$, then the probability of error $\leq 1/s$ (we can choose s as we want in order to control the probability of error — of course, if our field is large enough; for $\mathbb{Z}$ there is no problem with that).

If we can bound $\deg p_C$, we are lucky. But in general we only know it is bounded by $2^{|C|}$ (where $|C|$ is the number of gates: think about a circuit performing squaring at each gate, like Fig. 22).

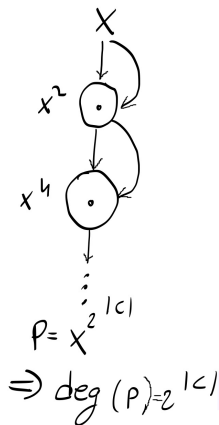


Figure 22: A circuit performing squaring at each gate

Then r_i 's are large, and the results of operations may reach $2^{2^{|C|}}$, which is a problem: we will spend too much time evaluating the circuit. Even if we need only $\log(2^{2^{|C|}})$ bits to represent the numbers, it is still exponential.

So, in order to mitigate this problem and keep the numbers polynomial-size and the algorithm polynomial-time, let us choose $\mathbb{F} = \mathbb{F}_m$, the finite field module some prime number $m \in \mathbb{P}$. If m is small enough, the operations will return small numbers, and modulo

²³The degree w.r.t. all variables altogether. For example, the total degree of $x_1^4 x_2^5 + x_1^2 x_2 x_3^7$ is 15, because the first monomial has degree $4 + 5 = 9$.

arithmetic can be implemented in polynomial time. However, if original polynomial was over $\mathbb{Z}$ and not over our $\mathbb{F}_m$, there may be now two sources of error:

1. $p \neq 0$, but its coefficients are zeroed modulo m ;
2. $p \neq 0$ even over $\mathbb{F}_m$, but we hit a root with our random r_i 's.

The second option is handled by Lemma 9.1. To overcome the first option, we will select m also at random.

Namely, let us select m at random in the interval $2^{2|C|} \leq m \leq 2^{4|C|}$. Due to the Prime Number Theorem there are $\Omega(2^{4|C|}/|C|)$ prime numbers in this interval²⁴. On the other hand, if $p \neq 0$, look at some its nonzero coefficient b . It cannot have more than $\log_2 b$ different prime divisors. Thus the probability that we hit one by selecting m at random is $O(\frac{\log_2 b}{2^{4|C|}/|C|}) = O(\frac{2^{|C|}}{2^{4|C|}/|C|})$, which is exponentially small.

In the meantime, the probability of error due to the lemma is at most $\frac{\deg p}{m} \leq \frac{2^{|C|}}{2^{2|C|}}$, also exponentially small.

The overall probability of error is thus exponentially small.

9.2.4 Two-sided bounded error

Now that we explored one-sided error, either false positive or false negative we will dive into two-sided error algorithms that make both types of errors.

Definition 9.5. A **two-sided bounded-error** algorithm A for a language L must satisfy the following two conditions:

- $\forall x \in L \quad \Pr\{A(x) = 1\} \geq \frac{3}{4}. \quad // \text{False negative with probability} \leq 1/4.$
- $\forall x \notin L \quad \Pr\{A(x) = 1\} \leq \frac{1}{4}. \quad // \text{False positive with probability} \leq 1/4.$

Note: As we will see, the probabilities here are arbitrary, and their sum does not need to sum up to 1. It could have been even ≥ 0.1 vs ≤ 0.05 (such a bizarre situation is left as an **exercise** — do it after we show how to use Chernoff's bounds).

Definition 9.6. The class **BPTIME** $[t(n)]$ consists of languages decided by two-sided bounded-error $O(t(n))$ -time algorithms.

$$\mathbf{BPP} = \bigcup_{k \in \mathbb{N}} \mathbf{BPTIME}[n^k].$$

²⁴So how do we pick a random one? Repeatedly take such a number at random and test it for primality. If, say, $|C|^2$ attempts fail, that's an error, but its probability is very small.

9.2.5 Amplification of success: the case of two-sided errors

We previously saw how to reduce the probability of error when our algorithm has a one-sided error. This strategy will now work for two-sided error, because we cannot stop at the moment when the algorithm returns yes (or no? the error is two-sided!). We will still make several attempts to run the algorithm, but we will count the number of yes and no answers and see which one comes more frequently.

Let A be a **BPTIME** algorithm for L with error probability $\frac{1}{4}$.

Amplified algorithm B .

Input: x .

```

For i:=1 to k
--- if  $A(x) = 1$  then  $a_i := 1$  else  $a_i := 0$ . //All attempts use independent randomness!
If  $\sum_{j=1}^k a_i > k/2$  then accept else reject.

```

To estimate the probability of error, recall Chernoff's inequality

$$\Pr\{Z > (1 + \varepsilon)pk\} < \left(\frac{e^\varepsilon}{(1 + \varepsilon)^{1+\varepsilon}} \right)^{pk} \leq e^{-\frac{pk\varepsilon^2}{4}},$$

where $Z = \sum_{i=1}^k z_i$ and every z_i is an independent random variable that equals 1 with probability p and 0 with probability $1 - p$.

(Note that $\mathbf{E}Z = pk$, so the probability here is $\Pr\{Z > (1 + \varepsilon)\mathbf{E}Z\}$.)

For us z_i is an error ($a_i \neq L(x)$), in the i -th attempt; let us set $p = \frac{1}{4}$, $\varepsilon = 1$. That is, $(1 + \varepsilon)pk = \frac{k}{2}$, and the probability in this inequality is the probability of the even that the majority of the outcomes is wrong. Thus, the error probability of Algorithm B is

$$\Pr\{\text{more than } k/2 \text{ errors}\} \leq 2^{-\Omega(k)}.$$

9.2.6 RP and BPP via witnesses and computation trees

One-sided error. Recall that $L \in \mathbf{NP}$ iff there is an “**NP**-relation”²⁵

$R \subseteq \bigcup_{n \in \mathbb{N}} \{0, 1\}^n \times \{0, 1\}^{p(n)}$ such that $\forall x \in \{0, 1\}^*$

$$x \notin L \Rightarrow \forall w (x, w) \notin R.$$

$$x \in L \Rightarrow \exists w (x, w) \in R.$$

(Recall that we can assume that the length of a witness is exactly $p(n)$ for a polynomial p .)

One can define **RP** via such a relation in a similar manner:

²⁵Meant there: polynomially bounded (balanced), polynomial-time computable

$$\begin{aligned}
x \notin L &\Rightarrow \forall w (x, w) \notin R. \\
x \in L &\Rightarrow \frac{|\{w : (x, w) \in R\}|}{|\{\text{all } w\}|} \geq \frac{1}{2}.
\end{aligned}$$

That is, if in **NP** we require that for $x \in L$ there is at least one correct solution (a witness accepted by R), for **RP** we require that at least half the possible witnesses are correct solutions.

One can readily see the equivalence to the previous definition of **RP**: The witnesses of length $p(n)$ are random bit strings of length $p(n)$.

As we previously learned, the probability $\frac{1}{2}$ in this definition can be replaced by any other constant strictly between 0 and 1.

Example 9.1. To bring some intuition to this representation, think about a computation tree for a nondeterministic machine deciding the satisfiability of a *Boolean formula (or a circuit)* such that *more than a half of all the assignments are satisfying it*.

If your problem can be reduced to such formulas (i.e., either a formula is unsatisfiable or it has that many solutions), it has a one-sided bounded-error algorithm.

Two-sided error. Similarly, one can define **BPP** via such a relation by requiring

$$\begin{aligned}
x \notin L &\Rightarrow \frac{|\{w : (x, w) \in R\}|}{|\{\text{all } w\}|} \leq \frac{1}{4} \\
x \in L &\Rightarrow \frac{|\{w : (x, w) \in R\}|}{|\{\text{all } w\}|} \geq \frac{3}{4}
\end{aligned}$$

(As we previously learned — partially as an exercise — the probabilities $1/4$ and $3/4$ here can be replaced by any other pair of constants c_1 and c_2 such that $0 < c_1 < c_2 < 1$.)

That is, the portion of “correct solution” for inputs in the language is a strictly greater constant compared to the portion of “correct solutions” for inputs outside the language.

9.3 Errorless algorithms

Let us give two definitions to errorless algorithms and then prove their equivalence.

Definition 9.7. $L \in \mathbf{ZPP}$ if there is an errorless expected-polynomial-time algorithm for L , that is, a probabilistic machine A such that $\forall x A(x) = L(x)$ and $\max_{x \in \{0,1\}^n} \mathbf{E}\text{time}_A(x) \leq \text{poly}(n)$.

Definition 9.8. $\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{co-RP}$.

Proof. $\boxed{1 \Rightarrow 2}$ Let A have $\mathbb{E} \text{time}_A(x) \leq p(|x|)$.

Interrupt A after $2p(|x|)$ steps and answer “no” (thus we introduce a false negative).

$\Pr\{\text{time} \geq 2 \cdot \mathbb{E} \text{time}\} \leq \frac{1}{2}$ by Markov’s inequality.

Thus we get an **RP** algorithm.

To get a **co-RP** algorithm instead, answer “yes” (instead of “no”) when A does not stop in $2p(|x|)$ steps: this way we introduce false positives only.

$\boxed{2 \Rightarrow 1}$ Let B be an **RP** and C be a **co-RP** algorithm for L , both with a $1/2$ error rate.

Run $B(x)$ and $C(x)$ repeatedly until one of them produces a provably correct answer for x . (That is, either B answers “yes” or C answers “no”.)

Since there is some specific correct answer, the respective algorithm will give it with probability $p \geq 1/2$ after the first run, with probability $(1-p)p$ after the second run, $(1-p)^2p$ after the third run, etc. It is easy to see that

$$\mathbb{E}[\text{attempts}] \leq p \cdot 1 + (1-p)p \cdot 2 + (1-p)^2p \cdot 3 + \dots \leq \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \dots = \sum_{k=1}^{\infty} \frac{k}{2^k} = 2.$$

□

9.4 Closure properties and relation of ZPP, RP, and BPP to other classes

- Trivially **RP** $\subseteq$ **NP**.
- Thus **ZPP** = **RP** $\cap$ **co-RP** $\subseteq$ **NP** $\cap$ **co-NP**.
- Both questions **BPP** $\stackrel{?}{\subseteq}$ **NP** and **NP** $\stackrel{?}{\subseteq}$ **BPP** are open. Next time we will see where **BPP** is.

Several observations about inserting further probabilistic procedures inside probabilistic algorithms:

- **P** = **P^P** (deterministic, instead of using the oracle just simulate the oracle’s machine inside the lower machine — already learned, a polynomial of a polynomial is a polynomial).
- **BPP** = **BPP^{BPP}** (before simulating the oracle decrease the success probability of the oracle’s machine to exponentially small; then the chances that the lower **BPP** machine will behave not as intended would be exponentially small as it makes at most a polynomial number of oracle queries; the overall error probability is $\varepsilon(n) + p(n)/2^{k(n)}$, where $\varepsilon(n)$ is the probability of the lower machine, $p(n)$ is its running time, and $k(n)$ is the number of attempts to make the probability of the upper machine small enough, e.g., $k(n) = p(n)$).

- $\mathbf{ZPP} = \mathbf{ZPP}^{\mathbf{ZPP}}$ (Exercise — use the definition with expected polynomial time to prove it).
- We cannot say anything meaningful about $\mathbf{RP}^{\mathbf{RP}}$ (except that it is obviously inside $\mathbf{BPP}$), because using a one-sided error ($\mathbf{RP}$) language as an oracle essentially uses both types of errors; this way one can build a hierarchy $\mathbf{RP}^{\mathbf{RP}^{\mathbf{RP}}}$ inside $\mathbf{BPP}$ like we did with $\mathbf{PH}$!

9.4.1 Example: Errorless algorithm

Consider the problem of finding a pattern in a string.

Input: $p, s \in \{0, 1\}^*$, strings.

Output: “yes” if $\exists x, y$ s.t. $s = xpy$.

- it is obviously in $\mathbf{NP}$,
- a brute-force deterministic algorithm makes $(|s| - |p| + 1)|p|$ comparisons: just go over the string and check for the substring at every position,
- deterministic $O(|s|)$ time: Knuth–Morrison–Pratt algorithm.

As an example let us consider a very easy errorless $O(|s|)$ -time algorithm for this problem. We will start with a one-side error algorithm (the one without the blue-color additional check below) and then make it errorless at the cost of switching from worst-case to expected running time.

We identify $|p|$ -bit strings with numbers.

-
1. Choose a random number $m \in \mathbb{P}$ at most M (we’ll determine M later).
 2. Compute $r := p \bmod m$.
 3. For $i := 1$ to $|s| - |p| + 1$
 - (a) Compute $r' := s[i..i + |p| - 1] \bmod m$.
 - (b) If $r = r'$ then we check the substring, if equal then, accept.
 4. Reject.
-

Note that r' can be recomputed very fast from its previous value (take into account one new bit and one old bit).

Idea. We compare fingerprints of substrings instead of comparing the substrings directly. Fix a modulus m (chosen at random) and interpret a string as an integer under modulo encoding. We then compare values modulo m . If p occurs in s , their fingerprints match at the correct position. If fingerprints match at some position, we either err (if we prefer one-sided error and guaranteed running time) or verify by direct comparison to avoid errors getting an errorless version of this algorithm.

Correctness (co-RTime, if considered without the verification step). If the pattern p occurs in s , the algorithm accepts.

If the pattern p does not occur, there may be false positive with probability $\leq \frac{(|s|-|p|+1)|p|}{M}$ (note that a k -bit number can have at most k prime divisors; then $(|s| - |p| + 1) |p|$ -bit number can have $\leq (|s| - |p| + 1)|p|$ prime divisors).

So in order to mitigate the error we choose $M \sim 2|p||s|$.

The running time of the errorless version (so-to-say ZPTIME). If we add the blue part of the algorithm (checking the actual substring) we fall into **ZPP** with the expected time $\mathbb{E}[\text{operations}] \leq O(|p|) + c|s| + \frac{|s||p|}{M} \cdot |s| \cdot |p| = c|s| + O(|p|)$ if $M \sim |p|^2|s|^2$. (Here c is the small constant denoting the number of operations needed to recalculate r' from its previous value and $O(|p|)$ is for the initialization.)

10 Lecture 10. No scribe, essentially just reformatted slides

10.1 BPP, continued

10.1.1 Reminder: A PPTM, what is it?

Last lesson we studied randomized algorithms. Like we identify deterministic algorithms with DTMs, we can identify randomized algorithms with probabilistic Turing machines.

One can define them in several possible ways: as an algorithm that is capable of tossing a coin; as a Turing machine with a dedicated readonly random tape (that is read left-to-right only — if we are talking about space-bounded algorithms), this tape is filled with random bits before the run of the algorithm; as a NTM (either with witnesses or with choice — yet with a different sense of a choice, it is not nondeterministic).

Let us assume that when we have a $p(n)$ -time machine, then its witnesses or random strings have this exact length $p(n)$ (unless we are interested in shortening them). If we talk about computation trees, all the branches have the same length (for a given input).

Thus a **probabilistic Turing machine (PTM)** can be viewed as

- a DTM that has a dedicated readonly random tape (this is the default view),
it allows to use the probability theory and give nicer proofs if you remember it!

- a witness-type NTM where witness is the random tape,
useful if we can **count** the witnesses in some easy combinatorial way,
- a NTM making decisions – just they are now randomized, not nondeterministic,
visualized as a computation tree, we are interested in the proportion of accepting /
rejecting branches.

Thus a PPTM (probabilistic polynomial-time Turing machine) is a particular case of a polynomial-time NTM. **So what's the difference beyond the definition of deciding a language?**

- **Every polynomial-time NTM** defines *some* language in **NP**.
- **Not every PPTM** defines a language in, say, **RP**, because the machine needs to satisfy the definition, i.e., the probability of error must be bounded.

This situation prevents from diagonalizing against PPTMs (like we did against DTMs for proving the deterministic time hierarchy). Because of that, the question of the existence of complete problems for **RP**, **ZPP**, **BPP** is unknown; also time hierarchies as close as we had for **DTime** classes are also unknown.

For example, the following machine is **not** a PPTM:

```
Stupid Turing machine (input  $x$ ):
--- a:=random({0,1});      //toss a random coin
--- return a;
```

This one defines no language in **RP** or **BPP**. It is simply not a bounded-error TM. For this particular machine it is straightforward, but in general there is no obvious (algorithmic) way to check the bounded-error condition for all possible inputs at once if we are given the description of a PPTM.

10.1.2 Reminder: Two-sided bounded error

We continue studying two-sided bounded-error algorithms. By default, a **two-sided bounded error** algorithm A for language L must satisfy the following conditions:

- $\forall x \in L \quad \Pr\{A(x) = 1\} \geq \frac{3}{4}. \quad // \text{False negative occurs with probability } \leq 1/4$
- $\forall x \notin L \quad \Pr\{A(x) = 1\} \leq \frac{1}{4}. \quad // \text{False positive occurs with probability } \leq 1/4$

In total:

$$\Pr\{A(x) \neq L(x)\} \leq \frac{1}{4},$$

thus we say A has error $1/4$.

The class **BPTIME** $[t(n)]$ consists of all languages that have such $O(t(n))$ -time algorithms.

We defined

$$\mathbf{BPP} = \bigcup_{k \in \mathbb{N}} \mathbf{BPTIME}[n^k]$$

We noticed it before that by repeating a bounded-error algorithm a **polynomial** number of times, we can decrease the probability of error from $\frac{1}{4}$ and even from $\frac{1}{2} - \frac{1}{\text{poly}(n)}$ to $\frac{1}{2^{\text{poly}(n)}}$.

In particular, $\ell(n)$ iterations decrease it from $\frac{1}{4}$ to $\frac{1}{2^{\Omega(\ell(n))}}$.

Note that if we talk about specific time $t(n)$ and not just polynomial time, we need to count: repeating an algorithm, of course, increases time. Only **BPP** is robust wrt changing the probability of error, not **BPTIME**.

10.1.3 Non-uniform advice: another view of **P/poly**

Before (spoiler!) putting **BPP** into **P/poly** let us consider another equivalent definition of **P/poly**.

1. Recall our old definition: $L \in \mathbf{P/poly}$ if there is a family $\{C_n\}_{n \in \mathbb{N}}$ of poly-size circuits such that for every input x ,

$$x \in L \iff C_{|x|}(x) = 1.$$

For each input length n there is a circuit C_n .

2. Now a new definition: $L \in \mathbf{P/poly}$ if there is a sequence $\{a_n\}_{n \in \mathbb{N}}$ of poly-size strings and a poly-time DTM M such that for every input x ,

$$x \in L \iff M(\langle x, a_{|x|} \rangle) = 1.$$

For each input length n there is an advice string a_n .

Lemma 10.1. These two definitions of **P/poly** are equivalent.

Proof. old $\Rightarrow$ new $a_n = C_n$ and $M(x, C) = C(x)$.

new $\Rightarrow$ old C_n simulates M on $\langle x, a_n \rangle$ for (known) polynomial number of steps, see Lemma 3.1; a_n is hardwired into C_n . $\square$

NB!!! String a_n is a non-uniform advice and not a witness:

- We do not need to verify it, we believe it and just use it.
- We get the same string a_n for every input of the same length n .

10.1.4 BPP $\subset$ P/poly

Theorem 10.1. $\text{BPP} \subset \text{P/poly}$.

Proof. Let A be a **BPP** algorithm. We can assume that it has error probability $\leq \frac{1}{4^n}$. We write $A(x, w)$ to denote its run (and answer) on a particular input and a particular random string (particular choice of random bits): x is the input and w is the random string.

Let us call a random string w “good” for x if it leads to no error on input x , i.e., $A(x, w) = L(x)$. We can assume the share of such “good” random strings $\geq 1 - \frac{1}{4^n}$ (the share of “bad” strings that lead to errors is at most $\frac{1}{4^n}$).

Note that there exists a random string that is good for every input (see Fig. 23):

$$\text{“share of bad strings for one input”} \times \text{“number of possible inputs”} \leq \frac{1}{4^n} \times 2^n < 1.$$

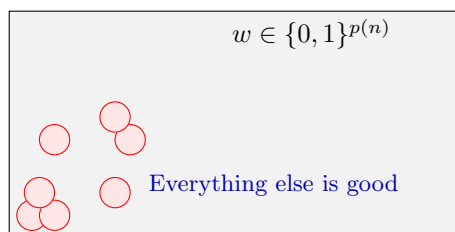


Figure 23: Sets of “bad” strings (for all length- n inputs) of a **BPP** algorithm with a small probability of error do not cover the whole space, thus there is at least one string that is “good” for every input in $\{0, 1\}^n$.

Let a_n be the random string that is good for all $x \in \{0, 1\}^n$. To satisfy the new definition of **P/poly** we simply run A on x and a_n (instead of its random string). $\square$

Therefore all our classes of randomized algorithms are inside **P/poly**: if a language can be decided in polynomial time with bounded error, then there is a family of (non-uniform) polynomial-size Boolean circuits for it — see Fig. 24.

Remark 10.1. One can consider also families of probabilistic polynomial-size circuits (circuits that have variables for the input bits and for the random bits). However, one can similarly prove that they decide the same class **P/poly** — [exercise](#).

Remark 10.2. Note that the containment in Theorem 10.1 is strict, because **P/poly** contains undecidable languages (note, for example, that *all* unary languages are in **P/poly**, whether decidable or undecidable, because each circuit of a family is designed for specific

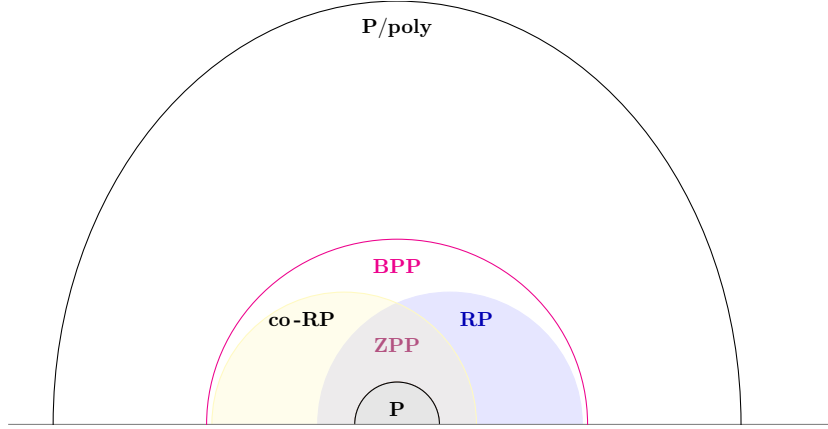


Figure 24: Randomized algorithms vs **P/poly**.

length n of the input, and in the case of a unary language there is nothing left to do for different strings of this length — except maybe checking that all the bits of the input are 1's), and **BPP** contains only decidable languages (to simulate a **BPP** machine in deterministic exponential time one can simply enumerate all possible random strings deterministically, run the algorithm on each of them, and count the number of “yes” and “no” answer).

10.1.5 $\mathbf{BPP} \subseteq \Sigma_2^P \cap \Pi_2^P$

We know that $\mathbf{RP} \subseteq \mathbf{NP}$, but where is **BPP** relative to the classes of the polynomial hierarchy? We will prove that it is in Σ_2^P (thus by symmetry also in Π_2^P).

Theorem 10.2. $\mathbf{BPP} \subseteq \Sigma_2^P$.

Proof. Consider a probabilistic algorithm $A(x, w)$, where w is its random string (of length $p(n)$).

Let $Y_x := \{w \in \{0, 1\}^{p(n)} \mid A(x, w) = 1\}$ be the set of random strings leading to the acceptance. (*This is not the set of “good” random strings! They are good only for the answer “yes”.*)

We can assume that the error probability of A is at most $\frac{1}{2^n}$. Then if $x \in L$, then $|Y_x| \geq (1 - \frac{1}{2^n}) \cdot 2^{p(n)}$.

For $x \in L$, our intention is to cover the whole set $U = \{0, 1\}^{p(n)}$ of random strings using a small amount d of “xor-copies” of Y_x . An “xor-copy” of a set $Y \subseteq U$ is a set of the same cardinality that is obtained by “moving” (“translating”) the set by bitwise xor with some string $r \in U$. Formally, $Y \oplus r = \{w \oplus r : w \in Y\}$, where each i -th bit of the resulting string is computed as $(w \oplus r)_i = w_i \oplus r_i$.

Let us estimate what d is enough for covering U : we want the following two-quantifier sentence to be true for $x \in L$:

$$\exists \{t_i\}_{i=1}^d \forall r \in U \bigvee_{i=1}^d (r \in Y_x \oplus t_i).$$

Let us prove that $d = p(n)$ copies suffice for it. For the sake of proving it (and only for that!) let us consider a random collection $T = \{t_1, \dots, t_d\}$ of strings in U and prove that the probability that it satisfies the corresponding statement in (10.1.5) is positive (thus it indeed *exists*). Put another way let us prove that the probability that it does not satisfy it (the sets $Y_x \oplus t_i$ do not cover U) is strictly less than 1:

$$\begin{aligned} & \Pr_T \{ \neg (\forall r \in U \bigvee_{i=1}^d (r \in Y_x \oplus t_i)) \} \xrightarrow{\text{de Morgan}} \Pr_T \{ \exists r \in U \bigwedge_{i=1}^d (r \notin Y_x \oplus t_i) \} \xrightarrow{\text{union bound}} \\ & \sum_{r \in U} \Pr_T \{ \bigwedge_{i=1}^d (r \notin Y_x \oplus t_i) \} \xrightarrow{t_i \text{'s independent}} \sum_{r \in U} \prod_{i=1}^d \Pr_T \{ r \notin Y_x \oplus t_i \} \stackrel{|Y_x \oplus t_i| = |Y_x| \text{ is big!}}{\leq} \frac{1}{2^{nd}} 2^{p(n)} \stackrel{d=p(n)}{<} 1. \end{aligned}$$

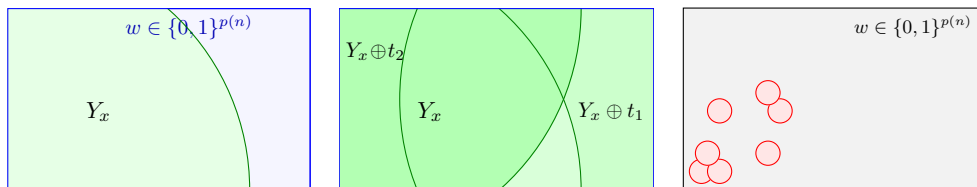


Figure 25: Covering the set of all random strings: (1) a big set Y_x , (2) polynomially many copies of a big Y_x cover U , (3) that many copies of a small Y_x are unable to cover U .

Thus, if $x \in L$, then (10.1.5) holds for it. This is a two quantifier $\exists\forall$ condition, like in Σ_2^p ! It remains to show that what is written after the quantifiers can be checked in polynomial time. Indeed, $r \in Y_x \oplus t_i$ if and only if $r \oplus t_i \in Y_x$ if and only if $A(x, r \oplus t_i)$ accepts. That is, to check this expression for given x , r , and t_i it suffices to run A just once. Now the big $\bigvee$ can be checked using a “for” cycle for $p(n)$ many possibilities ($i := 1$ to d), and at each iteration we run A just once.

What if $x \notin L$? In this case one cannot cover it just because of the cardinality! The share of Y_x in this case is at most $\frac{1}{2^n}$, and the cardinality of the union of $d = p(n)$ copies is at most $\frac{p(n)}{2^n} < 1$, they cannot fully cover U . Thus in this case (10.1.5) is false, so the two-quantifier description (10.1.5) indeed defines the language L . $\square$

Corollary 10.1. $\text{BPP} \subseteq \Sigma_2^p \cap \Pi_2^p$.

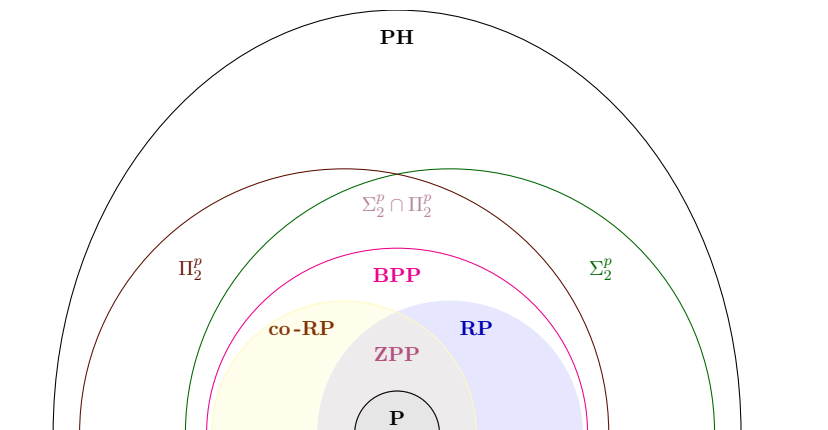


Figure 26: **BPP** vs **PH**.

Proof. By definition **BPP** = **co-BPP**. Theorem 10.2 proves **BPP** $\subseteq \Sigma_2^P$, thus **BPP** = **co-BPP** $\subseteq \text{co-}\Sigma_2^P = \Pi_2^P$. $\square$

Thus all classes for randomized polynomial-time algorithms with bounded error are contained in the second level of the polynomial hierarchy — see Fig. 26.

10.1.6 Probabilistic reductions: Solution isolation by a randomized reduction

Consider the “Unique-SAT” problem: Find a satisfying assignment if it is promised there is at most one. Is it easier to find a satisfying assignment for F in this case? Formally and actually — yes. (For example, for every variable there must be a clause satisfied by this variable only: $(\overline{x_1} \vee x_2 \vee \overline{x_3})$. This observation can indeed improve (exponential) running time — we won’t prove it in this course; see *Boolean SAT course*.) It turns out, however, that w.r.t. polynomial time there is no gain from the promise that there is no other satisfying assignment. We will prove it.

Our strategy of reducing the general SAT problem to “Unique-SAT” is:

- If the input formula has many satisfying assignments, blindly pick one of them using [hashing](#).
- Namely, change $F(x)$ into $F(x) \wedge h(x) = z$ for randomly chosen h and z .
- If we are lucky enough, the new formula will have just a single satisfying assignment, and then we can call an oracle for “Unique-SAT” thus solving the general problem for the original formula F using this oracle.

That is, we are going to prove that if there is an algorithm running in deterministic polynomial time that solves correctly SAT for formulas that have at most one satisfying assignment, then $\mathbf{NP} = \mathbf{RP}$.

A family of pairwise-independent hash-functions. We will need the following useful combinatorial notion.

Definition 10.1. Let $k, n \in \mathbb{N}$. $H_{n,k}$ is a **family of pairwise-independent hash-functions** if

- it contains functions $h : \{0, 1\}^n \rightarrow \{0, 1\}^k$,
- for every $x \neq x'$, for every y, y' , $\Pr_{h \leftarrow H_{n,k}} \{h(x) = y \wedge h(x') = y'\} = 2^{-2k}$.

(In particular, $\Pr_h \{h(x) = y\} = 2^{-k}$.)

Such families exist. For example, the following one.

Lemma 10.2 (construction of a family of pairwise-independent hash-functions, Theorem 8.15 in [AB09]). Identify $\{0, 1\}^n$ with $GF(2^n)$ ²⁶, then one can take $H_{n,n}$ composed of functions

$$h_{a,b}(x) = ax + b \quad (a, b \in GF(2^n)).$$

Proof. Fix four elements y, y' , and $x \neq x'$. If $h_{a,b}(x) = y$ and $h_{a,b}(x') = y'$, then a, b is unambiguously defined by

$$\begin{aligned} a \cdot x + b &= y \\ a \cdot x' + b &= y'. \end{aligned}$$

Thus for mapping $x \mapsto y$ and $x' \mapsto y'$ there is just one function out of $\frac{1}{2^{2n}}$ possible functions. Therefore, we are done for $n = k$.

To complete the proof for the case of $n \neq k$, take this construction and fill the remaining details as an **exercise**: For the case of $k < n$, truncate outputs of $H_{n,n}$ to k bits. For the case of $k > n$, extend inputs of $H_{k,k}$ to k bits. $\square$

The Valiant–Vazirani Lemma. Using a family of pairwise-independent hash-functions we can isolate a single solution with *high enough*²⁷ probability. This is called the **Valiant–Vazirani lemma**.

Lemma 10.3 (Valiant, Vazirani). There is a polynomial-time computable (with randomness) function f mapping every F to G such that

²⁶The field with 2^n elements. If you don't know, one think about some other field.

²⁷Not very high, but we know how to boost the probability of success from a polynomially small one to a very good one.

- $F \notin \text{SAT} \Rightarrow G \notin \text{SAT}$,
- $F \in \text{SAT} \Rightarrow \Pr\{\text{there exists a unique } y \text{ s.t. } G(y) = 1\} \geq \frac{1}{8n}$.

Proof. We will take a randomly selected $h \in H_{n,k}$ and append the constraint $h(x) = 0$ to the input formula in the hope that only a single satisfying assignment will satisfy it. The new formula $G(x, x')$ will have more variables than the original formula $F(x)$, because we need to express $h(x)$ in CNF using auxiliary variables (since h is easily computable, one can use Lemmas 3.1 and 2.1 for that — note that these constructions do not change the number of satisfying assignments, as values of the auxiliary variables in Lemma 2.1 are determined by the values of the original variables of the circuit). So we will have $\exists x' G(x, x') = 1 \iff F(x) = 1 \wedge h(x) = 0$, and G has a unique satisfying assignment if and only if exactly one satisfying assignment of F satisfies the condition $h(x) = 0$.

It remains to choose k and prove that with high probability exactly one satisfying assignment of F satisfies the condition $h(x) = 0$ when h is chosen at random from $H_{n,k}$. Informally, we need to guess the size of the set of satisfying assignments (the larger the value of k is, the stronger is hashing; ideally k should be around $\log_2$ of the number of satisfying assignments of F).

Let F be satisfiable, and let S be the set of all satisfying assignments of F , and n be the number of variables in F . Now our algorithm is:

1. Guess k between 1 and n at random.
2. Take a hash-function h from $H_{n,k}$ at random.
3. Output $G(x, x')$ equivalent to $F(x) = 1 \wedge h(x) = 0$ (as described above).

We want to select k such that $2^{k-2} < |S| \leq 2^{k-1}$. Note that there are only n possible values for k , thus we will select such k with probability $\frac{1}{n}$.

Let us prove that $\Pr_{h \in H_{n,k}} \{|S \cap \{x : h(x) = 0\}| = 1\} \geq \frac{1}{8}$.

Let $N = |\{x \in S : h(x) = 0\}|$. (We want $N = 1$.)

$$\Pr\{N \geq 1\} \geq \sum_{x \in S} \Pr\{h(x) = 0\} - \sum_{x, x' \in S} \Pr\{h(x) = 0, h(x') = 0\} = |S|2^{-k} - \binom{|S|}{2}2^{-2k}.$$

$$\Pr\{N \geq 2\} = \text{the number of pairs} \times \text{probability to keep both assignments} \leq \binom{|S|}{2}2^{-2k}.$$

$$\Pr\{N = 1\} = \Pr\{N \geq 1\} - \Pr\{N \geq 2\} \geq |S|2^{-k} - |S|^2 2^{-2k} = |S|2^{-k}(1 - |S|2^{-k}) \geq \frac{1}{8}.$$

(The last inequality is due to $\frac{1}{4} < |S|2^{-k} \leq \frac{1}{2}$ by our choice of k — do this small estimation as an easy **exercise** in calculus.) $\square$

Theorem 10.3. If there is an algorithm working in deterministic polynomial time that solves correctly SAT for formulas that have at most one satisfying assignment, then **NP** = **RP**.

Proof. Repeat the procedure in Lemma 10.3 $8n$ times (trying to apply the imaginary Unique-SAT algorithm to every G obtained).

If the input formula was unsatisfiable, we have no chance to succeed (all tries for formula G will be unsatisfiable and the subprocedure for Unique-SAT will return “no”).

If it was satisfiable, the chances that we will be unlucky in all the $8n$ tries are at most

$$\left(1 - \frac{1}{8n}\right)^{8n} < \frac{1}{e}.$$

Note that it does not matter what the subprocedure answers for non-uniquely satisfiable formulas: we are relying on its behaviour on uniquely satisfiable and unsatisfiable formulas only. $\square$

10.2 Relativization: How do oracles transform classes (and proofs)

10.2.1 Relativization

One can think of a situation where we are given an oracle for a particular function as a “world” where this function is given for free as a physical device. Thus facts of our ordinary world (without such a “free lunch”) may or may not still happen in such an augmented world.

For different oracles it can lead to different situations:

Theorem 10.4 (Baker–Gill–Solovay). There is an oracle A such that $\mathbf{P}^A = \mathbf{NP}^A$. There is an oracle B such that $\mathbf{P}^B \neq \mathbf{NP}^B$.

This theorem means, in particular, that it is impossible to resolve the $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$ question using a “relativizable” proof, that is, a proof that remains valid in every world where someone has access to some oracle. Constructive proofs that are just algorithms using other algorithms as subprocedures are typically relativizable (just add an oracle to every step of such a construction). There are very few non-relativizable proofs (counting arguments are one of them, we will hopefully see another one later in the course).

The construction of such an oracle A is quite simple: Take $A = \mathbf{QBF}$ and observe $\mathbf{P}^{\mathbf{QBF}} = \mathbf{NP}^{\mathbf{QBF}} = \mathbf{PSPACE}$ (complete this proof as an **exercise**: prove these two equivalences). Another possibility is to consider $A' = \mathbf{EXPCOMP} := \{\langle M, x \rangle 01^n : \text{DTM } M \text{ accepts } x \text{ in } \leq 2^n \text{ steps}\}$ (complete the proof for A' as an **exercise** as well).

The construction of an appropriate oracle B is more difficult and is not included in this course.

10.2.2 Exercises: How to work correctly with the oracle notation

Let us play a “**What if...**” game.

1. What consequences one could extract if someone proves $\mathbf{NP} \subseteq \mathbf{BPP}$?

Obviously then $\mathbf{NP}^{\mathbf{NP}} \subseteq \mathbf{NP}^{\mathbf{BPP}}$. One could also want to replace the lower $\mathbf{NP}$ class by $\mathbf{BPP}$, but **DON'T DO IT!** One cannot simply replace the lower class, because no one said that $\mathbf{NP}^O \subseteq \mathbf{BPP}^O$ for every oracle O (this is what would be formally needed for the replacement of the lower class: recall that in the $\mathbf{NP}^O$ or $\mathbf{NP}^{\mathbf{BPP}}$ notation the lower $\mathbf{NP}$ is not the class of *languages*, it is the class of *oracle machines*).

Moral: If you are trying to replace one class by another one, start it from the oracle.

Still in this case one can prove (in a different way) that this assumption implies $\mathbf{PH} = \mathbf{BPP}$ (and, in particular, it implies the collapse of the polynomial hierarchy).

Lemma 10.4. If $\mathbf{NP} \subseteq \mathbf{BPP}$, then $\mathbf{PH} = \mathbf{BPP}$.

Proof Sketch. Recall the proof of $\mathbf{BPP}^{\mathbf{BPP}} = \mathbf{BPP}$ and show that it works also for computing a function (not just a yes/no answer). (Complete this proof as an exercise. We will come back to it later, but please try doing it yourself before that.) $\square$

2. What consequences one could extract if someone proves $\mathbf{BPP} \subseteq \mathbf{NP}$?

Then by symmetry $\mathbf{BPP} \subseteq \mathbf{NP} \cap \mathbf{co-NP}$. That's it, unfortunately, we cannot add any catastrophic consequences. Some people believe even in $\mathbf{P} = \mathbf{BPP}$, so it is unlikely we can extract unexpected consequences from $\mathbf{BPP} \subseteq \mathbf{NP}$.

10.3 Introduction to Interactive Protocols (Interactive Proofs)

Consider a language L . Let x be an input string. In an interactive proof two agents, Prover (P) and Verifier (V) communicate (exchange multiple messages) and P tries to persuade V that $x \in L$. See Fig. 27.

A very simple example of an interactive proof is the definition of $\mathbf{NP}$: if $L \in \mathbf{NP}$, the prover can send a witness (a correct solution) for x to the verifier, and the verifier verifies it in deterministic polynomial time. There is only one round (prover-to-verifier) in this protocol.

In general, there may be many rounds and also the parties can use randomness (this is to be defined — there are at least two versions of that). In any case, the prover is assumed to be as powerful as possible (that is, P is a function that looks at the transcript of the protocol and outputs the next message — there is no need to compute anything algorithmically). The verifier is assumed to be polynomial time. The number of rounds is typically polynomial or even smaller (constant), during each round a message (a bit string) is sent from P to V or from V to P.

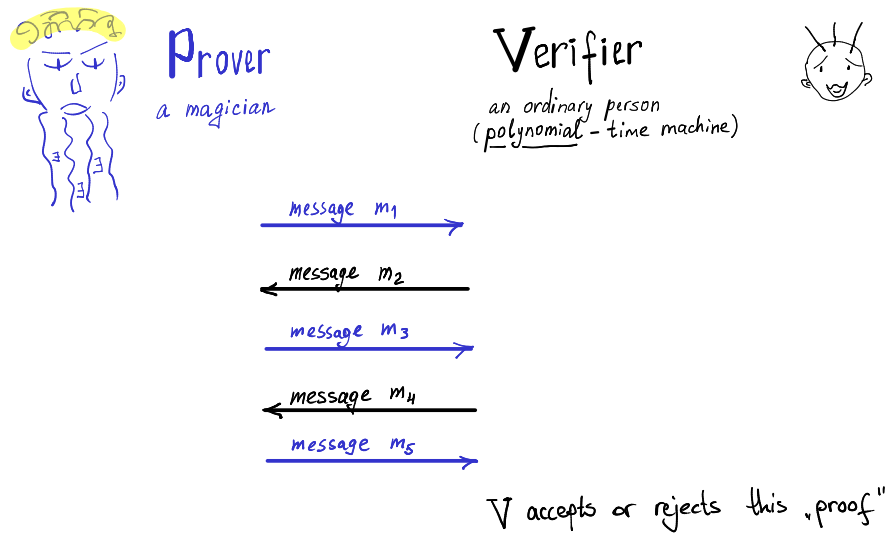


Figure 27: A general view of an interactive proof.

10.3.1 Deterministic protocols

It turns out that if P and V do not use randomness, then their protocol is always equivalent to the one-round protocol for $L \in \mathbf{NP}$ mentioned above.

Indeed, since everything is deterministic, all messages are functions of the input and previously sent messages. Therefore, P can compute all its messages without any interaction with V (because the all-powerful prover can, of course, perform V 's computations). Because of that, V can ask P to send all its messages as well as all V 's messages in a single round. Then V performs its computation like if it would communicate with P , verifying that P indeed included correct V 's messages in what it sent.

(If anything is unclear in this argument, please complete it as an [exercise](#).)

Remark 10.3. The class of languages that have deterministic interactive proofs is denoted in the book [AB09] as **dIP**. The above argument proves that **dIP** = **NP**, so it is not an interesting class.

10.3.2 Protocols with private coin

Let us consider protocols where the verifier is randomized, that is, it can toss a random coin in private (the verifier is not required to let the prover know the outcome, i.e., the actual values of these random bits). See Fig. 28.

Let us introduce notation and define formally this type of protocols.

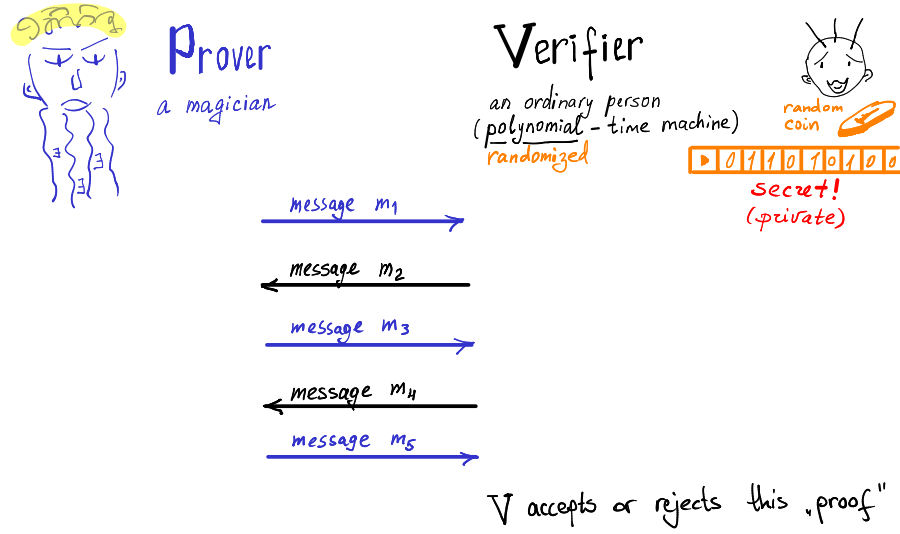


Figure 28: A private-coin protocol.

Denote by $\text{Out}\langle P, V \rangle(x)$ = the final answer ("yes" or "no") that V gives after interacting with P on x . (Note that this is a random variable!)

Definition 10.2 (Private-coin protocols, $\mathbf{IP}$). $L \in \mathbf{IP}[k] \iff$ there is a k -round protocol with randomized poly-time algorithm V s.t. $\forall x$

(Completeness) $x \in L \Rightarrow \exists P$ s.t. $\Pr\{\text{Out}\langle P, V \rangle(x) = 1\} \geq \frac{3}{4}$ (some P convinces V).

(Soundness) $x \notin L \Rightarrow \forall P'$ only $\Pr\{\text{Out}\langle P', V \rangle(x) = 1\} \leq \frac{1}{4}$ (nobody convinces V).

$$\mathbf{IP} = \bigcup_{c \in \mathbb{N}} \mathbf{IP}[O(n^c)]$$

Note that if we write $\mathbf{IP}[2]$, we do not mean an implicit $O(\dots)$, we mean the actual number of rounds! This will be more important in another type of protocols that we will consider (they usually consider polynomially-many-round $\mathbf{IP}$ protocols), but still.

To make it absolutely formal, one could treat P and V as functions, where the arguments are

- For P — the input and the history of communication,
- For V — the input, the history of communication, and the values of random bits used so far.

The value of P is the next message it sends to V . The value of V is either the next message it sends to P or the final answer (“yes” or “no”).

Example 10.1 (**IP** protocol for **GNI**). Let us consider an interactive protocol for the Graph Non-Isomorphism problem (**GNI**). In this problem the input contains two graphs $G_1 = (U, E_1)$ and $G_2 = (U, E_2)$, and this pair $(G_1, G_2) \in \text{GNI}$ iff there is no isomorphism between these graphs. (Recall that an isomorphism is a permutation π of vertices that makes these graphs identical: $\{u, v\} \in E_1 \iff \{\pi(u), \pi(v)\} \in E_2$.)

Why do we concentrate on the negative answer? This is because while we will see in what follows that **IP** is symmetric (**IP** = **co-IP**), this is not at all clear from the definition. The Graph Isomorphism problem (**GI**) has a trivial protocol, because **GI** $\in$ **NP**. Can a prover prove that there is no isomorphism?

An **IP**-protocol for **GNI**.

Input: Graphs $G_0 = (U, E_1)$, $G_1 = (U, E_2)$.

Round 1. Verifier

--- samples $\pi :=$ a random permutation of U .
 --- samples $b :=$ a random bit.
 --- computes the graph $H := \pi(G_b)$.
 --- sends H to the prover (note that this is just a graph, π and b are hidden from the prover).

Round 2. Prover

--- sends a bit b' .

Decision. Verifier accepts if $b = b'$, otherwise it rejects.

If $G_0 \not\approx G_1$, the all-powerful prover can determine b (because either $G_0 \approx H$ or $G_1 \approx H$, but not both of them). Thus the protocol is complete.

Let us prove its soundness. What happens if $G_0 \approx G_1$? A crucial observation is that the probability distribution of H is in both cases $b = 0$ and $b = 1$. Indeed, let ψ be an isomorphism of G_0 and G_1 , i.e., $G_1 = \psi(G_0)$. Then if $b = 1$, $H = \pi(\psi(G_0))$ is sent, and if $b = 0$, $H = \pi(G_0)$ is sent. But since π is a random permutation (of course, uniformly distributed over all the $n!$ permutations) the distributions of π and $\pi \circ \psi$ are identical: each permutation has probability $1/n!$ in each of these two distributions.

Formally, for every graph H' , $\Pr\{H = H' | b = 0\} = \Pr\{H = H' | b = 1\}$. Therefore, the prover has no chance to compute b correctly, and the probability of $b = b'$ is $1/2$.

Exercise: If you are not convinced, compute the probability of $b = b'$ yourself, for example, as $\frac{1}{2}(\Pr\{b = b' | b = 0\} + \Pr\{b = b' | b = 1\})$. Note that prover's answer b' is a function of the received message (H), not of b or π .

While the probability of error that we obtained is $1/2$ and $1/4$, it can be improved by repeating the protocol (even in parallel!).

11 Lecture 11. Scribes: Yohann Pardes & Hillel Ohayon

11.1 Interactive Protocols (continued)

11.1.1 Protocols with private coin

An Arthur–Merlin protocol is an interactive proof system where the Verifier’s coins are “public”. In this framework, the two parties are:

- **Merlin (The Prover):** An all-powerful “magician” (computationally unbounded) who tries to convince Arthur that an input belongs to a language L .
- **Arthur (The Verifier):** An “ordinary person” represented by a polynomial-time machine.

The random source is public (visible to both parties as coins are tossed), thus one can view Arthur as a deterministic machine, but still the protocol result is probabilistic.

Public-coin protocols at a glance.

1. **Public Randomness:** Unlike general interactive proofs ($\mathbf{IP}[k]$) where the Verifier might keep their random bits secret, in an AM protocol, Arthur uses public randomness. Every random bit Arthur generates is visible to Merlin. That is, the random tape is visible to both parties (but new bits are disclosed only when Arthur needs them — Merlin cannot dive into the future before sending his message).
2. **Communication Flow:** The parties exchange messages $m_1, m_2, \dots, m_k$.
3. **Arthur’s Role:** One can think of Arthur’s messages as containing *solely* his random bits. Since Merlin is computationally unbounded, he can calculate any function Arthur would have computed provided that all the arguments are accessible by Merlin — and (contrary to private-coin protocols) they are (the input and the random bits).
4. **The Decision:** After the exchange, Arthur (the Verifier V) performs a polynomial-time computation on the transcript to either **accept** or **reject** the proof.

Public coins make the Verifier seemingly less powerful, because Arthur is a particular case of a private-coin Verifier (the latter could send his random bits if he wanted it).

Formal definition. Consider the problem $x \stackrel{?}{\in} L$, so x is the input. Merlin claims $x \in L$. Denote the random variable $\text{Out}\langle M, A \rangle(x)$ = the answer A gives after interacting with M on x . Merlin is formally a function of the history (without any requirement on computability or complexity of that function): the arguments are the input, all previous messages and disclosed randomness, the output is the next message. Arthur is also a function of

the history, but this function is computed by a polynomial-time Turing machine. Since we won't compute more than a polynomial number of rounds, the history length is also bounded by a polynomial in the input length.

Definition 11.1 ($\mathbf{AM}[k]$).

- $L \in \mathbf{AM}[k] \iff$ there is a k -round public-coin protocol (the first message is Arthur's) with a poly-time deterministic algorithm A such that $\forall x$:
- **(Completeness)** $x \in L \implies \exists M$ s.t. $\Pr\{\text{Out}\langle M, A \rangle(x) = 1\} \geq \frac{3}{4}$ (some M convinces A).
- **(Soundness)** $x \notin L \implies \forall M'$ only $\Pr\{\text{Out}\langle M', A \rangle(x) = 1\} \leq \frac{1}{4}$ (nobody convinces A).
- The probability is taken over a random string jointly viewed by M and A .

One typically consider public-coin protocols for a constant number of rounds. It can be visualized by considering classes like $\mathbf{AM}$, $\mathbf{AMA}$, $\mathbf{AMAM}$, etc, where each letter means a message of the corresponding participant.

Contrary to $\mathbf{IP}$, the default number of rounds for $\mathbf{AM}$ is two:

$$\mathbf{AM} = \mathbf{AM}[2]$$

In fact, $\mathbf{AM}[k] = \mathbf{AM}$ for any constant k (we won't prove it in this course).

In spite of the fact that a public coin is a particular case of a random coin, $\mathbf{IP}[O(1)] = \mathbf{AM}[O(1)]$ (in fact, two extra rounds are sufficient to convert private randomness into public randomness). (We will give a sketch of this proof for two rounds.) Also $\mathbf{IP}[\text{poly}(n)] = \mathbf{AM}[\text{poly}(n)]$. (We will give a sketch of the proof that implies it.)

So essentially the convention is that *unless otherwise stated* when they talk about a *constant number of rounds*, they mean $\mathbf{AM}$ and public coin; when they talk about a *polynomial number* of rounds, we mean $\mathbf{IP}$ and private coin.

Fact 11.1. There is an equivalent definition of $\mathbf{AM}$ with perfect completeness (that is, one can always redesign a protocol so that, for every $x \in L$, Merlin would be *always*, i.e., with probability 1, capable of persuading Arthur to accept).

We won't prove it (but will later prove a similar fact for a smaller class $\mathbf{MA}$ and for a larger class $\mathbf{IP}$).

Example 11.1 (Set Size Lower Bound Protocol). The following protocol is a seminal result that is due to Shafi Goldwasser and Michael Sipser. (We will see one of its important consequences.)

Consider a specific input length m , and a set $S \subseteq \{0,1\}^m$ such that if $x \in S$, it can be "certified". (It mean formally that there is a polynomial-time procedure R such that

$S = \{x : \exists w R(\langle x, w \rangle) = 1\}$, where R is a polynomial-time DTM. Although this definition looks like an **NP** set definition, we will not concentrate on it and will not consider the whole set for all input lengths: it is enough for us that on this input length m Arthur is capable of checking that a certificate w indeed matches x , that is, $R(x, w) = 1$.)

Consider a specific input length m . We are interested in estimating the size of the set S . Namely, given $m, K \in \mathbb{N}$, with probability close to 1 Merlin will persuade Arthur to accept if $|S| \geq K$ and will not persuade him to accept if $|S| \leq K/2$. The running time will be polynomial in m .

To build this protocol we will use a family of pairwise-independent hash-functions (Lemma 10.2). The idea is to hash the elements of S into a set of size approximately K . Then if one chooses a hash value at random, if the size was large, then with high enough probability it will be a hash of an element of S ; if the size was small, then chances to hit such an element will be small.

Goldwasser-Sipser protocol for set lower bound.

Arthur:

- Chooses an integer k so that $2^{k-2} < K \leq 2^{k-1}$.
- Picks a hash function $h \in H_{m,k}$ at random.
- Picks $y \in \{0,1\}^k$ at random.
- Sends h and y .

Merlin:

- Computes x such that $h(x) = y$.
- Computes w such that $R(\langle x, w \rangle) = 1$.
- Sends x and w .

Arthur:

- Verifies that $h(x) = y$ and $R(\langle x, w \rangle) = 1$.
Accepts iff both conditions are satisfied.

We will prove that if the size of our set is $\geq K$, then the probability of acceptance will be large than some constant, and if it is $\leq K/2$, then it will be smaller than another (smaller) constant. By repeating the protocol and counting how many times it accepts and using Chernoff's bound (like we did for **BPP** amplification) one can decrease the probability of error to any desirable constant.

Note that this protocol accepts if and only if Merlin can indeed compute x and w as stated, that is, if $y \in h(S)$.

Denote $p := K/2^k$. Note that this constant between $\frac{1}{4}$ and $\frac{1}{2}$ is known to Arthur, because he computed k himself.

Soundness: Let us prove that if S is small, then with high probability $y \notin h(S)$ and thus Merlin has no chance to succeed (he will have either to give an incorrect x or an incorrect w , and thus will be caught).

We will use the union bound for that (the inequality holds even for a fixed h , not to say for random h):

$$\Pr_{h,y}\{h^{-1}(y) \cap S \neq \emptyset\} = \Pr_{h,y}\{y \in h(S)\} \leq \max_h \Pr_y\{y \in h(S)\} \leq |S| \cdot \frac{1}{2^k}.$$

Thus if $|S| \leq K/2$, then Arthur will accept Merlin's proof with probability $\leq \frac{|S|}{2^k} \leq \frac{K}{2 \cdot 2^k} \leq \frac{p}{2}$.

Completeness: Let us prove that if S is big, then Merlin will succeed with probability high enough. Since it is clear that the larger S is, the larger $h(S)$ is, and the larger are the chances that a randomly chosen y hits $h(S)$, it suffices to consider the case $K \leq |S| \leq 2^k$.

$$\Pr_{h,y}\{h^{-1}(y) \cap S \neq \emptyset\} = \Pr_{h,y}\{\exists x \in S : h(x) = y\} \quad (7)$$

We can estimate it even for every y (not to say for a random one)! By the inclusion formula we get that (7) is at least

$$\begin{aligned} &\geq \sum_{x \in S} \Pr_h\{h(x) = y\} - \frac{1}{2} \sum_{x \neq x' \in S} \Pr_h\{h(x) = y \wedge h(x') = y\} = \frac{|S|}{2^k} - \frac{1}{2} \cdot \frac{|S|^2}{2^{2k}} \\ &= \frac{|S|}{2^k} \left(1 - \frac{|S|}{2^{k+1}}\right) \geq \frac{3}{4} \cdot \frac{|S|}{2^k} \geq \frac{3}{4}p. \end{aligned}$$

The final protocol: We proved that

- If $|S| \geq K$, then $\Pr\{\text{Merlin succeeds}\} \geq \frac{3}{4}p$.
- If $|S| \leq K/2$, then $\Pr\{\text{Merlin succeeds}\} \leq \frac{p}{2}$.

By repeating this protocol (one can do it in parallel: Arthur chooses several y 's and h 's independently, Merlin answers all the challenges altogether) and using Chernoff's bound (like in **BPP** success amplification we have seen before) we get good probabilities. Namely, we count the number of successful Merlin's answers and set the threshold of the share of "yes" answers to, say, $\frac{5}{8}p$ (because $\frac{p}{2} < \frac{5}{8}p < \frac{3}{4}p$). Recall that p is a known constant bounded from below (it is at least $\frac{1}{4}$), thus the difference $\frac{3}{4}p - \frac{p}{2}$ is also bounded from below by a constant, thus Chernoff's bound will work.

Corollary 11.1 (Goldwasser–Sipser). $\mathbf{IP}[O(1)] = \mathbf{AM}[O(1)]$.

Proof sketch for two rounds. We will illustrate the technique for two-round protocols (like the **GNI** protocol we considered before as an example): Verifier chooses a private random string and sends some message to Prover, then Prover sends his response to Verifier, then Verifier accepts or rejects.

Merlin and Arthur will consider this private-coin protocol between Prover and Verifier without running it; Merlin will persuade Arthur that if they would engage in this protocol, then Arthur would accept with high probability.

Let $x \in \{0, 1\}^n$ be an input, and let S_x be the set of (private) random strings r that cause the Verifier to accept x . A big probability of acceptance means literally that this set S_x is large.

Merlin will use the Set Lower Bound Protocol for S_x in order to persuade Arthur. Namely, this (public-coin) protocol will distinguish between the situations that $|S_x| \geq \frac{2}{3} \cdot 2^{\rho(n)}$ and $|S_x| \leq \frac{1}{3} \cdot 2^{\rho(n)}$, where $\rho(n)$ is the length of Verifier's random string. It exactly corresponds to the situations where in the original private-coin protocol Verifier would accept or reject with probability as stated in the definition.

Note that S_x is indeed certifiable (as needed in the Set Lower Bound Protocol): the certificate for r is w that Prover would send in the private-coin protocol; the verification of this certificate (R) is done by the polynomial-time procedure of Verifier (that is given x, r, w on its input). $\square$

AM via probabilistic reductions. Recall that we have already seen an example of a probabilistic reduction when we reduced **SAT** to its uniquely-satisfiable instances (the Valiant–Vazirani Lemma 10.3). It was an oracle reduction. Let us look at **AM** as a class of languages reducible to **NP** using a many-one probabilistic reduction.

Namely, let us say that a language L is **many-one reducible to a language S via a two-sided error probabilistic reduction** (notation: $L = \mathbf{BP} \cdot S$) if $S \subseteq \bigcup_{n \in \mathbb{N}} \{0, 1\}^n \times \{0, 1\}^{p(n)}$

and

$$\begin{aligned} x \in L &\Rightarrow \Pr_{r \in \{0, 1\}^{p(|x|)}} \{(x, r) \in S\} \geq \frac{3}{4} \\ x \notin L &\Rightarrow \Pr_{r \in \{0, 1\}^{p(|x|)}} \{(x, r) \in S\} \leq \frac{1}{4}. \end{aligned}$$

Then $\mathbf{AM} = \mathbf{BP} \cdot \mathbf{NP}$, that is, $\mathbf{AM}$ is the class of languages that are many-one reducible to $\mathbf{NP}$ via a two-sided error probabilistic reduction:

Lemma 11.1. $\mathbf{AM} = \mathbf{BP} \cdot \mathbf{NP}$. In particular, $\mathbf{AM} \subseteq \mathbf{BPP}^{\mathbf{NP}}$.

Proof. Note that $L \in \mathbf{AM}$ iff there is a polynomial-time deterministic algorithm A such that $x \in L \Rightarrow \Pr_r\{\exists w : A(x, w, r) = 1\} \geq \frac{3}{4}$,
 $x \notin L \Rightarrow \Pr_r\{\exists w : A(x, w, r) = 1\} \leq \frac{1}{4}$.

(We simply rephrased the definition so that the random string became a part of the input of A .)

Define $S := \{(x, r) : \exists w A(x, w, r) = 1\}$. This set is in $\mathbf{NP}$, because this w is a witness and it can be verified by A . For every x ,

$$\Pr_r\{(x, r) \in S\} = \Pr_r\{\exists w A(x, w, r) = 1\},$$

thus the definition of $\mathbf{AM}$ above becomes literally the definition of $\mathbf{BP} \cdot \mathbf{NP}$ (namely, $L = \mathbf{BP} \cdot S$). $\square$

Exercise 11.1. Show that $\mathbf{BPP}^{\mathbf{NP}} \subseteq \Sigma_3^P \cap \Pi_3^P$, thus $\mathbf{AM}$ is in the third level of the polynomial hierarchy. In other words, check that our proof of Theorem 10.2 relativizes.

11.1.2 Yet simpler protocols: MA (Merlin speaks first)

Recall that in the definition of $\mathbf{AM}$ we assumed that Arthur speaks first. Thus if we consider protocols where Merlin speaks first, if Merlin then has another chance to speak, it will fall under the $\mathbf{AM}$ definition and (as we know that $\mathbf{AM} = \mathbf{AM}[k]$ will be just $\mathbf{AM}$). Therefore, the only uncovered case is when Merlin speaks first and just once; then Arthur decides using a randomized polynomial-time procedure (Merlin can see these random bits, but already has no chance to answer, so it does not matter really whether they are public or private).

Definition 11.2 (Class $\mathbf{MA}$). A language $L \in \mathbf{MA}$ if there is a randomized polynomial-time algorithm A such that for all x :

Completeness: $x \in L \implies \exists w \Pr\{A(x, w) = 1\} \geq \frac{3}{4}$ (Some Merlin can convince A).

Soundness: $x \notin L \implies \forall w \Pr\{A(x, w) = 1\} \leq \frac{1}{4}$ (Nobody convinces A).

Note that this is similar to $\mathbf{NP}$, except the verification step is randomized.

Example 11.2 (Circuit Optimization). Given a circuit C , determine whether there exists an equivalent circuit of size $\leq k$.

Protocol:

- Merlin sends a circuit D of size $\leq k$.
- Arthur verifies the equivalence using Polynomial Identity Testing (PIT).

MA is contained in AM. By what we have already discussed, it is clear that **MA** should be a particular case of **AM**. However, we did not give a proof for $\mathbf{AM} = \mathbf{AM}[k]$, and actually in order to prove it one needs to prove that $\mathbf{MA} \subseteq \mathbf{AM}$ (that is, two rounds A-then-M are sufficient to simulate M-then-A). Let us prove at least this (it is easy counting):

Lemma 11.2. $\mathbf{MA} \subseteq \mathbf{AM}$.

Proof. Look at the two definitions showing the random string r explicitly as a part of Arthur's input (thus Arthur is formally deterministic):

- $L \in \mathbf{MA}$:

$$\begin{aligned} & \exists \text{ polynomial-time } A \text{ such that } \forall x \\ x \in L & \implies \exists w \Pr_r \{A(x, w, r) = 1\} \geq \frac{3}{4} \\ x \notin L & \implies \forall w \Pr_r \{A(x, w, r) = 1\} \leq \frac{1}{4} \end{aligned}$$

- $L \in \mathbf{AM}$:

$$\begin{aligned} & \exists \text{ polynomial-time } A \text{ such that } \forall x \\ x \in L & \implies \Pr_r \{\exists w : A(x, w, r) = 1\} \geq \frac{3}{4} \\ x \notin L & \implies \Pr_r \{\exists w : A(x, w, r) = 1\} \leq \frac{1}{4} \end{aligned}$$

The two definitions differ in the order of “quantifiers”: $\exists \Pr$ vs $\Pr \exists$. The proof idea is to swap these “quantifiers”, that is, simply to let Arthur (from **MA**) send his message before Merlin's. It cannot be done literally this way, but we will see that if the probability of error of the original protocol is small enough (which we can reach by repeating it), then the new protocol will be correct (as an **AM** protocol).

Let us do it formally. Let input $x \in \{0, 1\}^n$, and let $|w| = p(n)$ for $|x| = n$ in the original MA protocol.

1. In the original MA protocol Arthur can repeat the verification $O(np(n))$ times on the same w , thus reducing the error probability to $\leq 1/2^{np(n)}$ (recall **BPP** success amplification, it is exactly the same procedure, but the input consists of both x and w). Let us consider this protocol instead, so without loss of generality A will refer to this low-error MA protocol. Note that $|w|$ has not changed.
2. Now let Arthur disclose his random bits before Merlin sends w (effectively turning MA into AM). Thus we use exactly the same procedure $A(x, w, r)$. We will prove that there are so few “bad” random strings for the original protocol that most of the good strings work for every message Merlin might send, not just one.

Completeness: If $x \in L$, this order change can only improve the chances that Merlin succeed.

Soundness: If $x \notin L$, then in the original protocol

$$\forall w \in \{0, 1\}^{p(n)} \quad \Pr_r \{A(x, w, r) = 1\} \leq \frac{1}{2^{np(n)}},$$

thus by the union bound:

$$\Pr_r \{\exists w : A(x, w, r) = 1\} \leq 2^{p(n)} \text{ (the number of } w\text{'s)} \times \frac{1}{2^{np(n)}} \text{ (the prob. to accept for a fixed } w) = \frac{1}{2^n}.$$

□

Perfect Completeness for MA. In fact, one can assume that in the case of the positive answer ($x \in L$) Merlin can always persuade Arthur. This observation will also find a place for **MA** in the second level of the polynomial hierarchy thus strengthening our previous result that **BPP** $\subseteq \Sigma_2^p$ (Theorem 10.2).

Lemma 11.3 (Perfect completeness for **MA**). If we replace $\frac{3}{4}$ by 1 in the definition of **MA**, it will yield an equivalent definition. In particular, **MA** $\subseteq \Sigma_2^p$.

Sketch. It is clear that this is a particular case of the previous definition, so the question is to prove that we do not restrict it too much by this requirement.

Recall the xor-cover method that we used to show **BPP** $\subseteq \Sigma_2^p$.

- Let $|w| = p(n)$ for $|x| = n$.
- Arthur repeats the protocol many times on the same w , reducing error to $\leq 1/2^{n^2 p^2(n)}$.
- Merlin can send w along with polynomially many strings $t_1, \dots, t_d$, these strings will show how to cover all random strings U by xor-copies of the set $Y_{x,w} = \{r \subseteq \{0, 1\}^{q(n)} : A(x, w, r) = 1\}$. The proof of their existence is literally the same as in Theorem 10.2.
- Arthur verifies $\bigcup_i Y_{x,w} \oplus t_i = \{0, 1\}^{q(n)}$ ($= U$) probabilistically (instead of the universal quantifier in our earlier proof). For $x \in L$, Arthur accepts. Otherwise, this union $\bigcup \dots$ is tiny!

In more detail, let us review our proof of Theorem 10.2: in the case where $x \in L$, the covering is complete (thus probability to hit a covered random string is 1); in the case where $x \notin L$, the union of these sets is so small that the probability to hit a covered random string is negligible. (Only that in Theorem 10.2 the question was about $x \in L$ vs $x \notin L$, and in our new setting the question is about a pair of (x, w) such that $\Pr_r A(x, w, r) \geq 1 - 1/2^{n^2 p^2(n)}$ or vs $\Pr_r A(x, w, r) \leq 1/2^{n^2 p^2(n)}$ for the original A).

The corollary about Σ_2^P follows directly from definitions. $\square$

AM and AMA are the same. Similarly to Lemma 11.2, one can prove that when Arthur speaks first, he does not need to use additional randomness at the end; in symbols:

Lemma 11.4. $\mathbf{AMA} = \mathbf{AM}$.

Proof sketch. It suffices to prove $\mathbf{AMA} \subseteq \mathbf{AM}$. Let $L \in \mathbf{AMA}$. By definition, there exists a probabilistic polynomial-time verifier A and polynomials p, q such that the protocol proceeds in three rounds:

1. **Arthur** chooses a random string r_1 .
2. **Merlin** sends a message w .
3. **Arthur** chooses a random string r_2 and decides as $A(x, r_1, w, r_2)$.

Let us fix the input x and Arthur's first message r_1 . The remaining interaction (rounds 2 and 3) essentially constitutes an **MA** protocol with respect to the input (x, r_1) .

Recall our proof of $\mathbf{MA} \subseteq \mathbf{AM}$: we learned that if the probability of the original protocol is made small, then we can exchange the rounds. Thus let Arthur perform his final check with r_2 many times so that the error probability becomes exponentially small. Then the new (longer) r_2 can be sent before w , that is, along with r_1 .

Now, the protocol indeed looks like:

1. **Arthur** chooses random string r_1 .
2. **Arthur** chooses random string r'_2 (from the transformed **AM** part).
3. **Merlin** sends message w (yes, he has to send w that would be good for every sample of the former r_2 : they are now substrings of r'_2).

$\square$

The containments between **MA**, **AM**, and (some) related classes are depicted in Fig. 29.

11.1.3 Polynomially many rounds: $\mathbf{IP} = \mathbf{PSPACE}$

Theorem 11.1 (Adi Shamir). $\mathbf{IP} = \mathbf{PSPACE}$.

Proof. $\mathbf{IP} \subseteq \mathbf{PSPACE}$ A language L is defined by a protocol (that is, by a polynomial-time machine V). We are given an input x and we need to compute ourselves deterministically (without any prover and without any randomness, but using polynomial space and potentially exponential time) whether the probability that the protocol accepts is more than $\frac{3}{4}$ (or less than $\frac{1}{4}$).

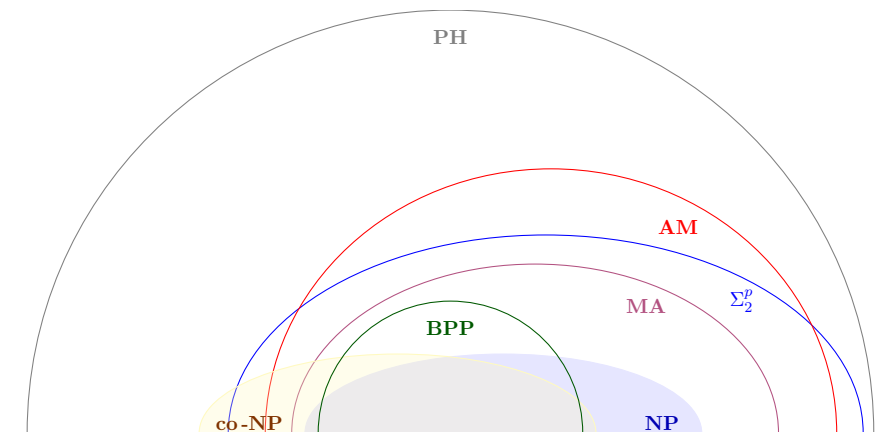


Figure 29: Containments between **MA**, **AM**, and related classes. **IP** is not shown as (spoiler!) it is above all that: **IP = PSPACE**.

Let us compute this probability using a recursive procedure. We will start in an incorrect way, but at the end we will fix it. One could think of the protocol as a tree where Verifier's nodes are represented by $\exists$ quantifiers (a good solution) and Prover's nodes as Pr meaning the next bit is selected at random. (See Fig. 30.) Since the number of rounds is polynomial anyway, we can assume that each message contains a single bit. The problem is that this tree actually corresponds to a public-coin protocol and not to the private-coin protocol, because lower nodes of Prover somehow “know” the random bits used by Verifier.

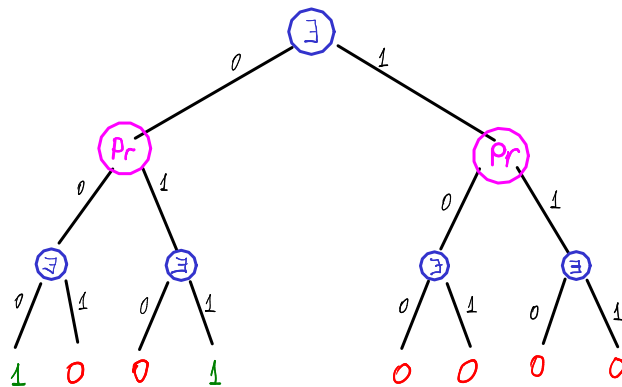


Figure 30: Public-coin protocol tree

To represent a private-coin protocol correctly, the tree should branch on verifier mes-

sages, not (secret) random bits. Splitting in a $\exists$ -node corresponds to two possible messages of Prover; splitting in a Pr-node corresponds to (how many? one can think that at most two) possible messages of Verifier. Each Verifier's message comes with a certain probability. (In the case of uniform random bits these probabilities are 0, 1/2, and 1, but for the sake of understanding think about a range of messages sent after sampling several random bits.) Note that even though the tree is binary, Verifier's message itself does not need to contain a single bit: we can have two (or one) branches corresponding to two (or one) multiple-bit messages; these messages depend on the whole history (the bits selected by the verifier earlier — higher in this branch).

Thus we split over the messages (not coins) and count the probabilities. (See Fig. 31.)

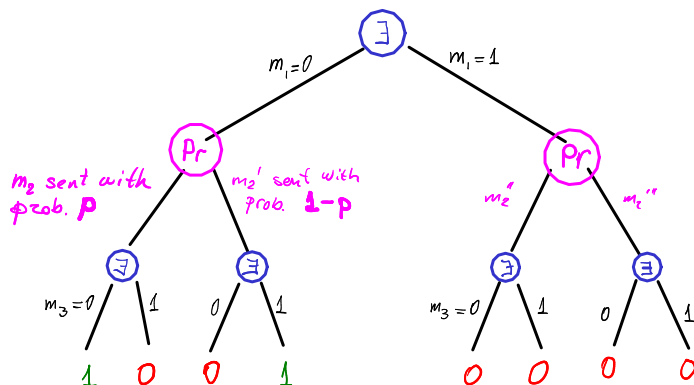


Figure 31: Private-coin protocol tree

Now one can design a recursive (thus **PSPACE**) deterministic algorithm for computing the probability of acceptance. For a leaf, the probability of acceptance is either 0 or 1 depending on whether Verifier rejects or accepts given the history of communication.

Consider a tree with two branches. If the probabilities of acceptance in two its subtrees are p_1 and p_2 ,

- If the root of our tree is an “ $\exists$ ” node, the acceptance probability of the whole tree evaluates as $\max(p_1, p_2)$ (The prover selects the best option!).
- If the root of our tree is a “Pr” node, the tree evaluates as $p \cdot p_1 + (1 - p) \cdot p_2$ (The verifier selects a subtree at random, where p is the probability of one possible message and $1 - p$ is the probability of the alternative message.)

Thus we can evaluate the whole tree by a recursive procedure (thus by depth-first search, thus using polynomial space — the depth of this tree is the total amount of communication, thus it is polynomially bounded). Note that all probabilities can be computed as base-2^k

where k is the number of random bits used, so we can save these values using a polynomial number of bits.

PSPACE $\subseteq$ **IP** (*Sketch*)

Update: See a more detailed version in the next lecture write-up. However, a “birds-eye” view may be better here in this lecture. Also the next lecture contains a couple of easier protocols (sumcheck and permanent) that may be easier to study before this one.

In order to prove that **PSPACE** $\subseteq$ **IP**, we design an interactive protocol for a **PSPACE**-complete language, **QBF**. (It is easy to see that **IP** is closed under polynomial-time many-one reductions: if we would like to design an interactive protocol for another language $L \in \mathbf{PSPACE}$, Verifier applies the reduction to **QBF** and then runs the protocol for **QBF**.)

Recall that $\mathbf{QBF} = \{\exists x_1 \forall x_2 \dots \forall x_n \phi(\vec{x}) : \text{this formula is true}\}$, where $\vec{x} = (x_1, x_2, \dots)$ lists all the Boolean variables occurring in the formula ϕ (one can assume that ϕ is in 3-CNF; or in 3-DNF; choose whatever is better for you).

A birds view of the proof:

- We will represent a quantified formula as a polynomial and the polynomial for a closed formula F (without free variables) will be actually a certain constant iff F is true (that is, belongs to the language **QBF**).
- We will start with writing a polynomial for a quantifier-free formula $\phi(\vec{x})$, then we will transform this polynomial in a polynomial corresponding to formulas with more and more quantifiers. New polynomials will be defined implicitly (that is, without writing down all the coefficients — think about an arithmetic formula with parentheses that we do not want to open because otherwise there will be an exponential blowup of terms).
- In the protocol Prover will claim that the quantified (closed) formula is true, thus the corresponding polynomial is a specific constant; then at each round the parties will get rid of one quantifier and thus consider polynomials in one, two, etc, variables, until they come to a quantifier-free formula.
- Thus in one round the parties will use a self-reduction of the problem: instead of proving that $P(\vec{r}) = c$ for a certain implicitly defined polynomial P , point $\vec{r}$ (not necessarily in 0/1-variables), and a constant c , they will prove a similar statement for different $P', \vec{r}', c'$, and P will correspond to a formula that has one quantifier less.
- Verifier will use PIT in order to verify the equivalence of polynomials.
- After we define the protocol, we will notice that the degree of our polynomials may grow too much, and then we will give (a sketch of) modification that will avoid it.

There are many ways to “arithmetize” a Boolean formula. We start with converting a quantifier-free formula ϕ into a polynomial p (formally, in the same variables, but they can now take not only 0 and 1 values) such that $\phi(\vec{x}) = 1 \iff \forall \vec{x} \in \{0,1\}^n p(\vec{x}) = 0$.

Here is one way to do it (let us define the polynomial inductively) keeping the value 0 for true formulas and the value 1 for false formulas:

- A variable x_i is transformed into the polynomial $1 - x_i$ (it’s only root is 1).
- If a formula ψ is transformed into a polynomial q , then its negation $\overline{\psi}$ is transformed into $1 - q$.
- If ψ_1 and ψ_2 are transformed into q_1 and q_2 , then $\psi_1 \vee \psi_2$ is transformed into $q_1 q_2$.
- Then $\wedge$ can be translated using de Morgan rules (via $\vee$ and negation).

This arithmetization is not the same (and not as efficient) as in the book proof, but it may be easier because there is specific value both for true and for false formulas.

We also need to encode the quantifiers $\forall z, \exists z$ as operators A_z and E_z that take a polynomial in a (formerly Boolean) variable z and other variables $\vec{y}$ and return another polynomial with one variable less such that $\forall z F(\vec{y}, z)$ is true iff for every $z \in \{0,1\}$, $(A_z F)(\vec{y}) = 0$. Similarly, $\exists z F(\vec{y}, z)$ is true iff there is $z \in \{0,1\}$ such that $(E_z F)(\vec{y}) = 0$.

Note that $\exists$ over a two-element set is actually a disjunction, and $\forall$ is a conjunction. Thus one can define $E_z F(z) = F(0)F(1)$ (all other variables stay as they were; the variable z is substituted by two values, and these two polynomials are multiplied). One can define A_z similarly via de Morgan laws.

The protocol.

Initially Prover claims that

$$E_{x_1} A_{x_2} E_{x_3} \dots Q_{x_n} P_\phi = 0,$$

where P_ϕ is the polynomial we wrote for a quantifier-free formula $\phi(\vec{x})$.

The inductive step. At each step Prover claims something similar:

$$E_{x_i} A_{x_{i+1}} \dots Q_{x_n} P_\phi|_{x_1=r_1, \dots, x_{i-1}=r_{i-1}} = c, \quad (8)$$

where c is a certain constant (think about it as an integer; for the first stage it is 0) and $r_1, \dots, r_{i-1}$ are the values previously selected for the corresponding variables. The first operator may be either A or E .

Prover’s turn:

Note that if we remove one operator and consider

$$A_{x_{i+1}} \dots Q_{x_n} P_\phi|_{x_1=r_1, \dots, x_{i-1}=r_{i-1}} \quad (9)$$

it is a polynomial in a single variable x_i (all other variables are either substituted by values or removed by the operators); call it $q(x_i)$.

The prover sends the coefficients of this uni-variable polynomial.

Verifier's turn:

The verifier needs to check two things:

1. q is indeed the polynomial (9),
2. conditioned on that, the equation (8) is true,

Item 2 can be verified by Verifier easily: if the first operator is E_{x_i} , it needs just to verify $q(0)q(1) = c$. Given the coefficients of q and the constant c , this is a very easy task.

Now item 1 will be delegated to further steps of the protocol. Verifier chooses a random value r_i for x_i and computes the value of q at this random point:

$$c' := q(r_i).$$

Then the task of proving that the value of (9) at this point is indeed c' is delegated recursively to the next step of the protocol (note that this task is of the same form as the previous one, yet there is one operator less in the implicit representation of the polynomial): Prover must prove that

$$A_{x_{i+1}} \dots Q_{x_n} P_\phi|_{x_1=r_1, \dots, x_{i-1}=r_{i-1}, x_i=r_i} = c' \quad (10)$$

The case E_{x_i} is treated similarly.

The base.

We proceed like that until we eliminate all the operators (formerly, quantifiers). That is, there are values r_i for all the variables x_i , and Verifier simply substitutes them into the quantifier-free polynomial P_ϕ ; our arithmetization rules of transforming a formula into a polynomial show how to compute it. (One can treat them as a description of a circuit, or a straight-line program, for computing the value of P_ϕ given a specific point $x_1 = r_1, \dots, x_n = r_n$.)

We can now estimate the error probability of this algorithm. If Prover's claim is true, then he can simply provide Verifier with correct data, and Verifier will definitely accept.

If Prover's claim is false (thus the quantified formula is false), then at each step the following can happen: he sends wrong coefficients of the univariate polynomial (9), that is, the polynomials (9) and q are different; yet the random value $x_i = r_i$ chosen by Verifier is a root of their difference; that is, at this particular point their values are equal, and thus the equation delegated to the next round becomes true.

This is a particular (univariate!) case of Polynomial Identity Testing. If we choose r_i from a large enough set (or a field) $\mathbb{F}$, then the chances that we hit a root will be negligible (call it $\varepsilon(n)$), thus the overall probability of error will be

$$\text{the (polynomial) number of rounds} \times \varepsilon(n),$$

still small enough.

It remains to choose $\mathbb{F}$, because two questions remain — and here is where the proof becomes sketchy:

- We perform certain computations with numbers. If these numbers become large, it might be a problem. We will thus work in a finite field (say, modulo a certain prime number) large enough to keep $\varepsilon(n)$ small, but small enough to keep the number of bits in our numbers reasonable.
- To keep $\varepsilon(n)$ small, we need a field of at least, say, $\geq 2T \deg q$ elements, where T is the number of rounds. However, the degree of q can be large. Here is what can be done (but you can think about a different alternative!):
 - Note that we are initially interested in the values of P_ϕ on 0's and 1's. Such polynomials can be made *multilinear*, that is, the degree over each variable is at most 1 (thus the overall degree is at most the number of variables n). This is because for a 0/1-variable z we always have $z^2 = z$.
 - One can introduce a new “multilinearization” operator $L_z P$ that removes the degrees over variable z . Namely,

$$(L_z P)(z) = P(0)(1 - z) + P(1)z. \quad (11)$$

In other words, we take P modulo $z^2 - z$ as a polynomial modulo a polynomial. Then if we insert a sequence of such operators after every other operator, all the intermediate polynomials that we have to test for PIT will have degree at most n after this operator and slightly more ($2n$) before it; the only degree that might be larger will be of P_ϕ itself, but it also cannot be very big (well, if it would be, we could compute it in stages too).

- The overall task will thus look like

$$E_{x_1} L_{x_1} A_{x_2} L_{x_1} L_{x_2} E_{x_3} \dots Q_{x_n} L_{x_1} L_{x_2} \dots L_{x_n} P_\phi = 0.$$

- The additional steps to be checked will correspond to the operator L . Verifier will check them according to (11) and choose a *new* value for the variable (this will be the only difference from A and E -steps, because the operator L does not remove a variable).

Unfortunately, we do not have time to go into detail of this sketch in this lecture.

Update: next lecture we will repeat it and finish the issue with the linearization operator. □

Corollary 11.2.

1. $\mathbf{IP} = \mathbf{AM}[\text{poly}(n)]$.
2. $\mathbf{IP}$ can be equivalently formulated with perfect completeness (that is, if $x \in L$, then we can assume that the prover can *always* persuade the verifier to accept).
3. $\mathbf{IP} = \mathbf{co-IP}$.

Proof. Items 1 and 2 are corollaries of the (obvious) properties of our particular protocol for QBF: if we have a protocol that does not satisfy these requirements, we can convert it into a **PSPACE** machine (by the first part of the theorem) and then to a protocol with public coin and perfect completeness (by the second part of the theorem).

The last item 3 is due to the symmetry of **PSPACE**: $\mathbf{PSPACE} = \mathbf{co-PSPACE}$. $\square$

12 Lecture 12. No scribe, essentially just reformatted slides

12.1 More examples of interactive protocols (IP)

Even though we have already considered the most general protocol for QBF, we will consider several easier examples and then will come back to the QBF protocol.

12.1.1 Sumcheck protocol

Input: a polynomial g in n variables.

Prover claims: $K = \sum_{b_1=0}^1 \sum_{b_2=0}^1 \dots \sum_{b_n=0}^1 g(b_1, \dots, b_n)$

The protocol will proceed in rounds, in each round one $\sum$ from the claim will be removed and the task of verifying will be delegated to the next round.

Round for n variables:

Prover:
 --- send coefficients of the polynomial $h(x_1) := \sum_{b_2=0}^1 \dots \sum_{b_n=0}^1 g(x_1, b_2, \dots, b_n)$

Verifier:
 --- check $h(0) + h(1) = K$ (otherwise reject),
 --- pick and send random a ,

Then they recursively check $K' = \sum_{b_2=0}^1 \dots \sum_{b_n=0}^1 g'(b_2, \dots, b_n)$
 for $(n-1)$ -variable polynomial $g' = g|_{x_1=a}$ and $K' = h(a)$.

Last round ($n = 1$):
 --- check $g(0) + g(1) = K$, thus accept or reject.

If Prover's claim is correct, Verifier will accept (if Prover answers the questions correctly — but it can do it). If the claim is incorrect, we maintain the following invariant: if an error (with a small probability) does not happen, then the claim delegated to the next round is also incorrect.

The error happens only if Prover sends wrong coefficients, so the two polynomials are distinct, but their values at a randomly selected point coincide. That is, we accidentally hit a root of the nonzero polynomial that is a difference of the two polynomials. Since a degree- d univariate polynomial has at most d roots, the probability of error in one round is $\leq \frac{d}{\text{field size}} \leq \frac{1}{n^2}$ for a field of size $\geq dn^2$. In total (n rounds) the probability of error is $\leq n \cdot \frac{1}{n^2} \leq \frac{1}{n}$.

Exercise 12.1. Although we studied (and will repeat now) a stronger protocol for QBF, as an exercise conclude from the sumcheck protocol that $\mathbf{co-NP} \subseteq \mathbf{IP}$. (You will need to arithmetize the formula, but here you can do it in an easier way, for example, by arithmetize a 3-clause into a polynomial in a way such that Prover will have to prove that the sum of these polynomials is zero.)

12.1.2 Interactive protocol for computing the permanent of an integer matrix

The permanent is computed for a square matrix. It is defined like a determinant, but without signs. Computing it is very hard computationally. A formal definition:

$$\mathbf{perm} A = \sum_{\sigma \in S_n} \prod_{i=1}^n a_{i, \sigma(i)}$$

NB: Over the field of two elements the permanent equals the determinant (as $-1 = 1$) and is *easy*!

Although this is a function and not a predicate, we will present an interactive protocol for computing it. The prover will claim a certain value for this function and the verifier will verify it.

The protocol will proceed by downward self-reduction. Assume that Prover claims $\mathbf{perm} A = a$.

- Downward self-reduction: $\mathbf{perm} A = a \iff \sum_{j=1}^n a_{1j} \cdot \mathbf{perm} A_{1j} = a$.

The intention is to do the following:

- Prover sends $d_j = \mathbf{perm} A_{1j}$.
- Verifier checks $\sum_j a_{1j} d_j = a$ and recursively checks $\mathbf{perm} A_{1j} = d_j$.

There is no error during this communication exchange.

However, we have now n tasks of size $n - 1$ instead of a single task of size n . Too many of them: we need a single task of size $n - 1$ for the self-reduction. Thus let us merge all these tasks into one. We will do it by merging two tasks into one until just one task remains (and then the protocol will proceed with the next dimension reduction).

- We need to merge checking two problems: $\mathbf{perm} B = b$ and $\mathbf{perm} C = c$:
 - Prover sends the coefficients of $p(x) = \mathbf{perm}(Bx + C(1 - x))$.
 - Verifier checks $p(0) = c$ and $p(1) = d$ and checks recursively $\mathbf{perm}(Br + C(1 - r)) = p(r)$ for a random r .

There may be an error here if Prover lies about the coefficients of p . Since a univariate degree d polynomial cannot have more than d roots, the error probability is $\leq \frac{\deg(p(x) - \mathbf{perm}(Bx + C(1 - x)))}{\text{field size}}$.

Now if we do all the computations modulo a prime number $\geq n^4$ (it is easy to select it, n^4 is not a big number), the overall error probability in $O(n^2)$ merging events is $O(n^{-2})$.

12.1.3 An interactive protocol for QBF

Let us review in more detail the interactive protocol for QBF proving that $\mathbf{IP} = \mathbf{PSPACE}$. This theorem is due to Adi Shamir, this (a bit simplified) proof is due to Alexander Shen.

Caution: This section's exposition follows a slightly different arithmetization than before: true formulas correspond to the value 1 and not 0. It does not matter much, however.

Arithmetization. Let us arithmetize a quantifier-free Boolean formula (say, in CNF) as follows:

formula	$\mapsto$	polynomial
False	$\mapsto$	0
True	$\mapsto$	1
$\alpha \wedge \beta$	$\mapsto$	$\alpha \cdot \beta$
$\neg \alpha$	$\mapsto$	$1 - \alpha$
$\alpha \vee \beta$	$\mapsto$	$1 - (1 - \alpha)(1 - \beta) =: \alpha \odot \beta$

This way if the original formula was **True** for some 0-1 assignment, the polynomial will have the value 1 for this assignment; otherwise it will have the value 0.

To arithmetize quantified Boolean formulas, let us introduce operators (working on polynomials) instead of quantifiers:

$$\begin{aligned} (A_x P)(\dots) &= P(0, \dots) \cdot P(1, \dots), \\ (E_x P)(\dots) &= P(0, \dots) \odot P(1, \dots); \end{aligned}$$

We will need also one more operator (not for a quantifier): multilinearization

$$(L_x P)(x, \dots) = P \mod (x^2 - x).$$

This quantifier simply removes the degrees over the variable x ; note that for 0-1 variables the degree does not matter, $0^k = 0$ and $1^k = 1$. Still if the original formula was **True** for some 0-1 assignment of its free (not quantified) variables, the polynomial will have the value 1 for this assignment; otherwise it will have the value 0. Because of L_x the total degree of the intermediate polynomials (if we would compute them) would be never more than $\max(d, 2n)$ (even if A_x or E_x doubles the degree, subsequent L_y 's multilinearize it down to n).

Thus instead of proving that a quantified CNF is true, it suffices to prove that

$$q_{x_1}^{(1)} L_{x_1} q_{x_2}^{(2)} L_{x_1} L_{x_2} q^{(x_3)} \dots q_{x_n}^{(n)} L_{x_1} \dots L_{x_n} P(x_1 \dots x_n) = 1,$$

where each $q^{(i)}$ is the operator corresponding to the respective quantifier of the original formula (A_x to $\forall x$, E_x to $\exists x$). This is Prover's claim. (Note that the left-hand side is an implicit representation not just of a polynomial — of a constant! Because all the variables are removed by the operators A_x and E_x .)

The protocol. Let d be the input formula size. W.l.o.g. $d \geq n$, where n is the number of variables.

Our protocol is recursive. At its intermediate stage, Prover will be claiming that the following equality is true: Recursive protocol for proving that the formula is true:

$$q_{x_i} q'_{\dots} \dots q''_{\dots} P(x_1, \dots, x_{i-1}, x_i \dots x_n) |_{x_1=r_1, \dots, x_{i-1}=r_{i-1}} = c :$$

Denote

$$R(x_1, \dots, x_i) = q'_{\dots} \dots q''_{\dots} P(x_1, \dots, x_i).$$

Prover gives coefficients of this polynomial $s(x_i)$ of degree at most d .

$q = A$. If $s(0)s(1) \neq c$, Verifier rejects.

Otherwise checks recursively $R(r_1 \dots r_{i-1}, r_i) = s(r_i)$ for a random r_i .

$q = E$. Analogously, checking $s(0) \odot s(1) = c$.

$q = L$. Analogously, but we already have some value r_i for x_i !

Verifier checks $s(0) + (s(1) - s(0))r_i = c$ and checks recursively that $R(r_1, \dots, r_{i-1}, r'_i) = s(r'_i)$ for a new random r'_i .

The probability of error at each step is $\leq \frac{d}{\text{field size}}$. In total (d^2 steps) the probability of error is $\leq \frac{1}{d}$ for a field of size $\geq d^4$. (Yes, we can work modulo a prime number slightly more than d^4 .)

12.2 Probabilistically Checkable Proofs (PCP) and Hardness of Approximation

12.2.1 Probabilistically Checkable Proofs and the formulation of the PCP theorem

For $F \in \text{SAT}$, a proof is a satisfying assignment $w \in \{0, 1\}^n$. One needs to read all its bits to verify it. This is a typical situation for an **NP** language.

Can one read less than that? Ideally, we would prefer to read just $O(1)$ bits of the proof. Deterministically, of course, no (an easy **exercise** would show that then we would get $\mathbf{P} = \mathbf{NP}$). It turns out that if we use randomness and check a proof at random (but specially chosen) places, it would be possible. Of course, one needs to switch from satisfying assignments to another type of witness (again, **exercise**: show that with satisfying assignments we would get at least $\mathbf{NP} \subseteq \mathbf{BPP}$).

The difference between satisfying assignments and a new type of proof we will introduce is illustrated in Fig. 32.



Figure 32: Illustration of the difference between **NP** and **PCP**.

Thus we are looking for a proof π of the fact $x \in L$ in a format that we do not need to read, that is, such a proof that it would be enough to inspect $O(1)$ bits of such a proof in order to verify $x \in L$ with high probability.

Definition 12.1. Let us define the model of probabilistically checkable proofs.

Consider a language L and input x of length n .

Prover (actually, a $\exists$ quantifier): claims that $x \in L$, publishes a proof π for x and cannot modify it after that.

Verifier (notation: $V^\pi(x)$): accepts or rejects (x, π) , can use $\rho(n)$ random bits, polynomial time $p(n)$, can access $q(n)$ bits of π by index: that is, it has an “oracle” access to π : V can ask about the i -th bit and then V gets $\pi[i]$ instantly .

Let $\rho, q: \mathbb{N} \rightarrow \mathbb{N}$. We say that L has probabilistically checkable proofs with randomness $\rho(n)$ and $q(n)$ queries, $L \in \mathbf{PCP}(\rho(n), q(n))$, if there is a randomized polynomial-time algorithm V (verifier) such that $\forall x$

(**completeness**) $x \in L \Rightarrow \exists \pi \Pr\{V^\pi(x) = 1\} = 1$,

(**soundness**) $x \notin L \Rightarrow \forall \pi' \Pr\{V^{\pi'}(x) = 1\} < \frac{1}{2}$.

By definition, $\mathbf{NP} = \mathbf{PCP}(0, \text{poly}(n))$, but randomness can help as we know from the following seminal result:

Theorem 12.1 (PCP Theorem; Arora, Lund, Motwani, Sudan, and Szegedy 1992).
 $\mathbf{NP} = \mathbf{PCP}(O(\log n), O(1))$.

Remark 12.1. Note that this definition is stated with perfect completeness. In fact, just *three* bits of the proof are enough if we allow probabilities $1 - \delta$ vs $\frac{1}{2} + \delta$ instead.

Later we will prove an important part of this theorem: $\mathbf{NP} \subseteq \mathbf{PCP}(\text{poly}(n), O(1))$, that is, for every $\mathbf{NP}$ language there are probabilistically checkable proofs with polynomial randomness and $O(1)$ queries. Before that, we will demonstrate one of the many applications of the PCP Theorem.

12.2.2 Reminder: Optimization problems, Approximation algorithms

An **optimization problem** is a search problem R equipped with a **goal function** $f: \{0, 1\}^* \times \{0, 1\}^* \rightarrow \mathbb{Q}$ that characterizes the “quality of a solution. On input x , we are interested either in finding a solution w with the maximum possible $f(x, w)$ (a **maximization problem**) or the minimum possible $f(x, w)$ (a **minimization problem**). In this lecture we will talk mostly about maximization problems.

Example 12.1. MAX-SAT: Given CNF F , find assignment A (that is, $R(F, A) = 1$ for any assignment, not just a satisfying assignment) that maximizes the goal function $f(F, A) =$ the number of clauses of F satisfied by A .

Definition 12.2. **Exact algorithm** for maximization problem given x finds an optimal solution s_E , i.e.,

- $(x, s_E) \in R$,
- $f(x, s_E) = \max_{s: (x, s) \in R} f(x, s)$.

Definition 12.3. **ρ -approximation** algorithm finds a solution $s_\approx$ such that

- $(x, s_\approx) \in R$,
- $f(x, s_\approx) \geq \rho \cdot f(x, s_E)$, where s_E is the optimal solution.

Example 12.2. A very simple deterministic $\frac{1}{2}$ -approximation algorithm for MAX-SAT:
 --- choose the best of: the all-0 assignment and the all-1 assignment.

Assuming that the input formula has no empty clauses, this algorithm finds not just a $1/2$ -approximate solution, it finds a solution that satisfies at least half of the clauses (irrespective of the value of the optimal solution), because each clause contains either at least one positive or at least one negative literal, thus each clause is satisfied either by the all-0 or by the all-1 assignment (or by both).

Example 12.3. Random sampling gives a $\frac{7}{8}$ -approximation algorithm if each clause contains at least 3 literals (corresponding to distinct variables!). In particular, it gives a $\frac{7}{8}$ -approximation algorithm for MAX-3E-SAT, that is, the case where each clause contains exactly 3 literals.

Indeed, a 3-clause is satisfied by a random assignment with probability $7/8$. By the linearity of expectation, the expected number of satisfied clauses is $7m/8$, where m is the number of clauses. It already gives a $7/8$ -approximation algorithm w.r.t. expected value. Using Markov inequality, one can transform it into a polynomial-time bounded-error randomized algorithm (exercise, note that the number of clauses is an integer). In fact, one can derandomize this algorithm, but this is not that easy.

One can find an outdated but extensive list of other examples at <https://www.csc.kth.se/tcs/compendium/> (“Compendium” of optimization problems and their approximability, by Viggo Kann and Pierluigi Crescenzi).

12.2.3 PCP Theorem and Hardness of Approximation

Strong unsatisfiability using the PCP Theorem.

Corollary 12.1 (The PCP Theorem restated). $\exists \delta > 0 \exists k \in \mathbb{N}$ such that $\forall L \in \mathbf{NP}$ there is a polynomial-time computable many-one reduction $S: \{0, 1\}^* \rightarrow \{k\text{-CNF}\}$ such that

$$\begin{aligned} x \in L &\Rightarrow S(x) \in \mathbf{SAT}, \\ x \notin L &\Rightarrow \text{no assignment satisfies a share } \geq (1 - \delta) \text{ of clauses of } S(x). \end{aligned}$$

That is (thinking of $L = \mathbf{SAT}$) using the PCP Theorem one can reduce CNFs to CNFs so that satisfiable formulas remain satisfiable, while unsatisfiable formulas remain “strongly unsatisfiable”: every assignment falsified a constant share of clauses. This corollary strengthens the Cook–Levin Theorem.

Proof. Let us prove Corollary 12.1 assuming the PCP Theorem. We design the reduction.

First of all, we convert V (from the PCP Theorem) to a non-adaptive algorithm (that is, one that decides at the beginning which bits it wants to check without querying them one by one): if the old version makes c queries, the new version just makes all queries

it could make irrespectively of the answers (note that $c' := \sum_{j=0}^{c-1} 2^j = 2^c - 1$ is still a constant). From now on we assume that V is non-adaptive and makes c' queries.

That is, one can consider $V(x, r, w)$ as a function of input x , random string $r \in \{0, 1\}^{\rho(n)}$, and answers $w \in \{0, 1\}^{c'}$.

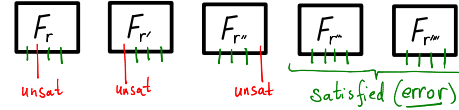
Consider a specific input x for our reduction.

Then for each r , we get a function $V_{x,r}: \{0, 1\}^{c'} \rightarrow \{0, 1\}$. Using the truth-table method we can write a constant-size c' -CNF F_r in variables $w_1, \dots, w_{c'}$ and auxiliary variables such that

$$F_r|_{w=w_*} \text{ is satisfiable iff } V_{x,r}(w_*) = 1.$$

The number of clauses in F_r is at most $2^{c'}$.

Our reduction S on input x outputs the conjunction of all these formulas F_r , that is, the formula $\bigwedge_{r \in \{0,1\}^{\rho(n)}} F_r$.



Now if $x \in L$ and π is a good proof, then all F_r are satisfied by π , because all tests go OK for any possible random string r . (Note that bits of w 's are bits of π .)

If $x \notin L$, then V rejects with probability at least $1/2$, that is, for every proof π , at least a half of r 's force $V^\pi(x, r) = 0$. Thus a half of F_r 's are unsatisfiable, every assignment falsifies at least one clause in each F_r . Thus a fraction of $\delta \geq 1/2^{c'+1}$ of the clauses is falsified. $\square$

Proving inapproximability of MAX-SAT using the PCP Theorem.

Corollary 12.2. If $\mathbf{P} \neq \mathbf{NP}$, then there are $k \in \mathbb{N}$ and $\alpha \in \mathbb{Q}$ such that there is no polynomial-time α -approximation algorithm for MAX- k -SAT.

Proof. One can choose any $\alpha > 1 - \delta$, where δ is the share of unsatisfied clauses guaranteed by Corollary 12.1; the number k for k -SAT is also from this corollary.

Assume that X is a δ -approximation polynomial-time algorithm. Let us design a polynomial-time algorithm for k -SAT, thus obtaining a contradicton.

Polynomial-time algorithm for SAT (input x):

- Produce CNF $F := S(x)$.
- If $X(F)$ satisfies a share of $> (1 - \delta)$ clauses of F , then accept.
- Otherwise reject.

Let us prove the correctness of this algorithm.

If $x \in \text{SAT}$, then $X(F)$ satisfies a share of $\geq \alpha \cdot 1 > 1 - \delta$.

If $x \notin \text{SAT}$, then $X(F)$ satisfies a share of at most $1 - \delta$ (strongly unsatisfiable). $\square$

See Fig. 33 for an informal explanation of this proof.

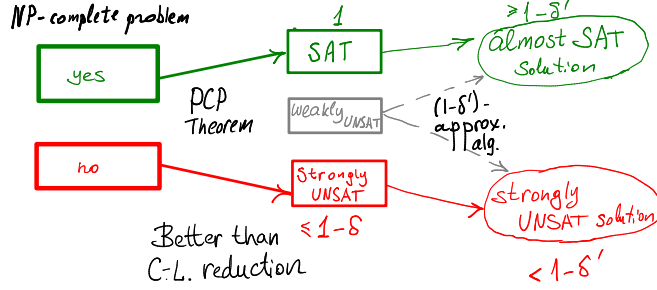


Figure 33: Proving inapproximability of MAX- k -SAT using strongly unsatisfiable formulas.

Remark 12.2. Note that Corollary 12.1 produces clauses of the same size c' , thus this result is valid for MAX- k -E-SAT as well.

Inapproximability of other problems. Given Corollary 12.2, one can prove inapproximability of other problems.

Example 12.4 (MAX-3E-SAT). Let us convert c' -CNF to 3-CNF (containing 3-clauses only), it extends this formula at most c' times: each c' -clause $(\ell_1 \vee \ell_2 \vee \dots \vee \ell_{c'})$ is converted into $(\ell_1 \vee \ell_2 \vee z_1) \wedge (\bar{z}_1 \vee \ell_2 \vee z_2) \vee \dots \vee (\bar{z}_{c'-1} \vee \ell_{c'-1} \vee \ell_{c'})$, where z_i 's are new variables introduced specifically for this clause.

Every assignment that falsified t clauses of the new formula will falsify at least t clauses of the original formula. Therefore, the share of falsified clauses will remain a constant (the share of falsified clauses will be worsened at most c' times, recall that c' was a constant).

One can illustrate this kind of reduction between optimization problems like in Fig. 34: one should check how the goal function is changed.

One can prove a stronger theorem (but not in this course):

Fact 12.1 (Håstad). If $\mathbf{P} \neq \mathbf{NP}$, then for MAX-3E-SAT (where all clauses have length exactly 3) there is no $(7/8 + \varepsilon)$ -approximation algorithm for any constant $\varepsilon > 0$.

Non-constant inapproximability. The next example shows that sometimes we can prove even [non-constant](#) inapproximability.

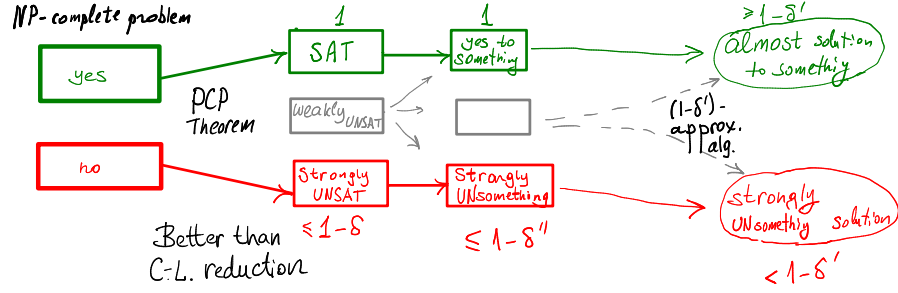


Figure 34: How to prove more inapproximabilities using reductions.

Example 12.5 (The maximum independent set). In the maximum independent set (**MAX-IS**) problem we are given an undirected graph $G = (V, E)$ and we are asked to find a subset of its vertices (independent set) $S \subseteq V$ such that $\forall u, v \in S$ such that $u \neq v$ we have $\{u, v\} \notin E$. The goal function is the number of vertices in S .

One can design a reduction from **MAX-3E-SAT** to **MAX-IS** so that the maximum number of satisfied clauses will be equal to the maximum independent set size; the number of vertices will be seven times larger than the number of clauses. We will not repeat this construction, see Figure 2.5 in the Arora–Barak book [AB09] for the details.

It implies that [there is](#) a certain constant α such that **MAX-IS** is not α -approximable unless $\mathbf{P} = \mathbf{NP}$. However, there is a stronger and more interesting result: it is not α -approximable [for any](#) constant $\alpha > 0$.

To do this, we design a reduction that would improve the inapproximability ratio (that is, it will improve the approximation ratio by a constant factor, thus if there is no approximation algorithm for a specific constant, there is no approximation algorithm for any other constant).

Note that by the nature of all the proof above, our inapproximability results can be viewed as impossibility (under the $\mathbf{P} \neq \mathbf{NP}$ hypothesis) of distinguishing an input that has a solution with the goal function value M from an input with the value ρM for a constant $\rho > 0$. Let ρ be the correspondent constant for **MAX-IS**, as explained above.

Given $G = (V, E)$ and a number $k \in \mathbb{N}$, consider a new graph $G_k = (V_k, E_k)$:

- $V_k = \{S \subseteq V : |S| = k\}$,
- $(S_1, S_2) \in E_k \iff S_1 \cup S_2$ is an independent set in G .

Note that $|\text{max. ind. set of } G_k| = \binom{|\text{max. ind. set of } G|}{k}$.

Thus to distinguish between the two situations when the input graph G has a max. ind. set of size M vs ρM (for a constant ρ) one could produce G_k , where these situations

would correspond to the max. ind. set size of $\binom{M}{k}$ vs $\binom{\rho M}{k}$. The ratio of these two values is asymptotically close to ρ^k , which can be made less than any desired constant ε . Thus an ε -approximation algorithm, if we apply it to G_k , would solve us the problem of distinguishing between M and ρM for G , providing a contradiction with $\mathbf{P} \neq \mathbf{NP}$.

Our final example is for a minimization problem:

Example 12.6 (Minimum Vertex Cover). The minimum vertex cover problem (**MIN-VC**) for a graph V asks for a subset $S \subseteq V$ (called a vertex cover) such that $\forall \{u, v\} \in E, u \in S \vee v \in S$.

Observe that the value of the maximum independent set equals $|V|$ minus the value of the minimum vertex cover. Moreover, independent sets are complements of vertex covers, and vice versa. Indeed, consider any independent set S : since each edge has at least one end outside S , the set $V \setminus S$ is a vertex cover. Then consider any vertex cover T : since each edge has at least one end in T , the set $V \setminus T$ is independent.

A feature of the reduction in Example 12.5 is that the input formula containing m had an assignment that satisfied M clauses if and only if the resulting graph $G = (V, E)$ with $|V| = 7m$ vertices had a maximum independent set of size M . Our goal is to distinguish **MAX-SAT** instances with m clauses that have solutions satisfying m clauses from those that have solutions satisfying at most $(1 - \delta)m$ clauses, i.e., to distinguish

- (a) graphs with $7m$ vertices that have maximum independent sets of size m from
- (b) those with maximum independent sets of size at most $(1 - \delta)m$.

Suppose that there is an algorithm that produces a solution S of size at most $\frac{C}{a}$ for **MIN-VC**, where C is the size of the smallest cover (and thus $|V| - C$ is the size of the largest independent set (the optimal solution for **MAX-IS**)). In particular, $V \setminus S$ is an independent set, and its value is $q := |V \setminus S| \geq |V| - \frac{C}{a} = 7m - \frac{C}{a}$.

For graphs of type (a) above $C = 7m - m = 6m$ and thus $q \geq (7 - \frac{6}{a})m$. To distinguish them from graphs of type (b) (for the latter graphs no algorithm can produce an independent set of size more than $(1 - \delta)m$, in particular, $q \leq (1 - \delta)m$), we set $a > 6/(6 + \delta)$ so that $(7 - \frac{6}{a})m > (1 - \delta)m$.

Bottom-line: if $\mathbf{P} \neq \mathbf{NP}$, no polynomial-time algorithm can produce a vertex cover of size less than C/a , where C is the size of the optimal solution and a is a constant derived above.

NB: In some sources for minimization problems the terminology may change from a -approximation to $\frac{1}{a}$ -approximation. If you see $a > 1$, then it is the former case, if you see $a < 1$, it is the latter case.

12.2.4 The proof of $\mathbf{NP} \subseteq \mathbf{PCP}(\text{poly}(n), O(1))$.

We will prove a weaker statement than the PCP Theorem: $\mathbf{NP} \subseteq \mathbf{PCP}(\text{poly}(n), O(1))$. Still it is interesting, because the number of queries is constant. Sometimes they call it the 1st part of the PCP Theorem.

(Why do we want the amount of random bits to be logarithmic? Otherwise the size of the proof is exponential, and we would not be able to produce a CNF of polynomial size for the inapproximability result.)

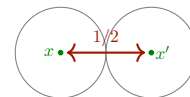
The Walsh–Hadamard code. Our proof format will use the Walsh–Hadamard code. Let us define it and prove its properties.

The function $\mathbf{WH}$ maps a string x of n bits into a codeword of 2^n bits, namely, the list of the 2^n values of the function $\mathbf{WH}(y) = \langle x, y \rangle = \sum_{i=1}^n x_i y_i \pmod 2$ (for every $y \in \{0, 1\}^n$). In other words, $\mathbf{WH}$ lists the values of the scalar product of x with every possible bit vector of the same size.

In what follows, all operations are modulo 2 (in particular, the subtraction and the addition are the same operation, XOR).

Let us prove that $\mathbf{WH}$ produces an error-correcting code of relative distance $1/2$ (you do not need to know what is an error-correcting code to understand the proof! it simply means that every two codewords must differ in at least half of their bits).

Lemma 12.1. $\forall x, x' \in \{0, 1\}^n \quad \Pr_{r \in \{0, 1\}^n} \{\mathbf{WH}(x)(r) \neq \mathbf{WH}(x')(r)\} \geq \frac{1}{2}.$



Proof. Note that $\mathbf{WH}(x - x') = \mathbf{WH}(x) - \mathbf{WH}(x')$, thus the question is about $\Pr_{r \in \{0, 1\}^n} \{\mathbf{WH}(y)(r) = 0\}$ for $y \neq 0$. We have seen such things before: let $y_i \neq 0$ (i.e., $y_i = 1$), then $\mathbf{WH}(y)(r) = 0 \iff r_i = \sum_{j \neq i} y_j r_j$; the probability to pick r_i that equals this value is exactly $1/2$. $\square$

Note that codewords of this code are exactly linear functions modulo 2!

Someone gave us access to $f: \{0, 1\}^n \rightarrow \{0, 1\}$, which is not necessary a codeword of anything. Our goal is to do the following using $O(1)$ queries:

1. Given access to some $f: \{0, 1\}^n \rightarrow \{0, 1\}$, check whether it is close to any codeword,
2. If it is close enough, restore the codeword.

Let us define “close” in the following way: two functions f, g are called δ -close (for some $\delta < 1/2$) if $\Pr_y \{f(y) \neq g(y)\} < \delta$.

Towards the first goal, consider the following procedure (the linearity test): Take random $y, z \in \{0, 1\}^n$ and check $f(y + z) = f(y) + f(z)$.

Lemma 12.2 (Linearity test). For $\delta < 1/3$, if f is not δ -close to a linear function, then

$$\Pr_{y,z}\{f(y+z) \neq f(y) + f(z)\} > \delta/2.$$

Proof. We will prove that for $\delta < 1/3$, if

$$\Pr_{y,z}\{f(y+z) = f(y) + f(z)\} \geq 1 - \delta/2,$$

then f is δ -close to a linear function.

Here is this linear function:

$$\tilde{f}(x) = \mathop{\text{maj}}_r (f(x+r) + f(r)).$$

It remains to prove that it is indeed linear and that f is close to it.

We start with improving the probability that $\tilde{f}$ and f coincide: by definition, they coincide in a half of the bits ($\tilde{f}$ is the majority bit!); we can prove more.

Improving the probability of the equality. Let $p_x = \Pr_r\{\tilde{f}(x) = f(x+r) + f(r)\}$. By construction $p_x \geq 1/2$. Let us prove $p_x \geq 1 - \delta$.

$$\begin{aligned} \Pr_{r,s}\{f(x+r) + f(s) \neq f(x+r+s)\} &\leq \delta/2, \\ \Pr_{r,s}\{f(r) + f(x+s) \neq f(x+r+s)\} &\leq \delta/2, \end{aligned}$$

i.e.,

$$\begin{aligned} 1 - \delta &\leq \Pr_{r,s}\{f(x+r) + f(s) = f(x+s) + f(r)\} = \Pr_{r,s}\{f(x+s) + f(s) = f(x+r) + f(r)\} = \\ &\sum_{b=0}^1 (\Pr_r\{f(x+r) + f(r) = b\})^2 = p_x^2 + (1 - p_x)^2 \leq p_x^2 + p_x(1 - p_x) = p_x. \end{aligned}$$

Overall,

$$\Pr_r\{\tilde{f}(x) \neq f(x+r) + f(r)\} < \delta.$$

Linearity. To prove that $\tilde{f}$ is linear, observe that

$$\begin{aligned} \Pr_r\{\tilde{f}(x) + \tilde{f}(y) + f(r) &\neq f(x+r) + \tilde{f}(y)\} < \delta, \\ \Pr_r\{f(x+r) + \tilde{f}(y) &\neq f(x+y+r)\} < \delta, \\ \Pr_r\{f(x+y+r) &\neq \tilde{f}(x+y) + f(r)\} < \delta, \end{aligned}$$

i.e. $\Pr_r\{\tilde{f}(x+y) = \tilde{f}(x) + \tilde{f}(y)\} > 0$. But this event does not depend on r , thus it just happens!

δ -closeness. Let us prove that f is δ -close to $\tilde{f}$: assume to the contrary that $\Pr_x\{f(x) \neq \tilde{f}(x)\} > \delta$. However, by construction $\Pr_r\{\tilde{f}(x) = f(x+r) + f(r)\} \geq 1/2$, thus $\Pr_{x,r}\{f(x) \neq f(x+r) + f(r)\} > \delta/2$, contradiction. $\square$

This lemma means that if we repeat the linearity test $O(1)$ times, then with a small (we can choose how small) probability of error we can conclude that f is 0.001-close to a linear function.

Towards the second goal, we introduce the following **self-correction procedure**: instead of f , the corrected function $\tilde{f}$ will be computed by taking a random y' and setting $\tilde{f}(y) = f(y') + f(y + y')$.

Lemma 12.3 (Self-correction). For $\delta < \frac{1}{4}$, if f is δ -close to a linear function $\tilde{f}$, then

$$\forall y \quad \Pr_{y'}\{\tilde{f}(y) = f(y') + f(y + y')\} \geq 1 - 2\delta.$$

Proof. By Lemma 12.2 both probabilities $\tilde{f}(y') \neq f(y')$ and $\tilde{f}(y + y') \neq f(y + y')$ are upper-bounded by δ . If none of these non-equalities happen, the event happens because $\tilde{f}(y) = \tilde{f}(y') + \tilde{f}(y + y')$ by the linearity of $\tilde{f}$. $\square$

Theorem 12.2. $\mathbf{NP} \subseteq \mathbf{PCP}(\text{poly}(n), O(1))$.

Proof. We consider the language of satisfiable (solvable) systems of quadratic equations mod 2.

Exercise #1: it suffices to show it for just one **NP**-complete language.

Exercise #2: the language of solvable systems of quadratic equations mod 2 is **NP**-complete (use equations like $z_{x=y} = xy + (1-x)(1-y)$ to define new variables and reduce **3-SAT** to it).

Consider a satisfiable instance of this problem (that is, a system of quadratic equations mod 2 of n variables that has a solution), and let $u = (u_1, \dots, u_n) \in \{0, 1\}^n$ be its satisfying assignment. The proof provided by the prover will contain $\mathbf{WH}(u)$ and $\mathbf{WH}(u \otimes u)$, where the vector $y \otimes z$ is defined as $(y \otimes z)_{ij} = y_i z_j$. Why do we need $u \otimes u$? Because in the quadratic system we have products of values of u and not just values themselves.

Thus we are given two functions f and g (purportedly $\mathbf{WH}(u)$ and $\mathbf{WH}(u \otimes u)$) for a satisfying assignment u). The idea of the verification algorithm is:

1. Test f and g for their 0.001-closeness to linear functions using the linearity test. If the test fails, we reject (the prover could give us truly linear functions, after all!). By Lemma 12.2 we can make the probability of error in this step arbitrarily small. Note that we make $O(1)$ queries to the bits of f and g .
2. In what follows use linear functions $\tilde{f}$ and $\tilde{g}$ obtained by self-correction. By Lemma 12.3 the probability that the self-correction procedure will produce incorrect values for $\tilde{f}$ or $\tilde{g}$ can be made arbitrarily small. Since in what follows we will query only $O(1)$ bits of $\tilde{f}$ and $\tilde{g}$, it will not require more than $O(1)$ bits of the original f and g .
3. Check that f and g are related as presumed (after all, they are supposed to be not just arbitrary functions!). If they are not related as needed (that is, $\tilde{g}$ is not produced as $u \otimes u$ for some u such that $\tilde{f} = \text{WH}(u)$), then

$$\Pr_{r,r'}\{\tilde{f}(r)\tilde{f}(r') \neq \tilde{g}(r \otimes r')\} \geq \frac{1}{4}. \quad (12)$$

Indeed, if the huge matrices $[\tilde{f}(r)\tilde{f}(r')]_{r,r'}$ and $[\tilde{g}(r \otimes r')]_{r,r'}$ are different, then

$$\Pr_{r'}\{[\tilde{f}(r)\tilde{f}(r')]_r = [\tilde{g}(r \otimes r')]_r\} \geq \frac{1}{2}$$

(that is, these two vectors are different). This is because taking a random r' produces a random subsum of their rows. Then for these two *different* vectors the probability that their random subsums are equal is again $\frac{1}{2}$ (and taking a random r is producing a subsum of a vector, note that the functions we are dealing with are linear, and for such a function selecting r is evaluating it at the point r).

4. It remains to verify that the assignment u that $\tilde{f}$ and $\tilde{g}$ are encoding indeed satisfies the original system.

The system is too big (many equations, even if we query a single bit for each equation, it would be too much). Thus to check the solution, we take a random subsum of equations of the original system obtaining a single quadratic equation, i.e., a quadratic form with certain coefficients a and right-hand side c . If the assignment does not satisfy the system, with probability $1/2$ it won't satisfy this quadratic equation as well. The latter we can check as $\tilde{g}(a) \neq c$ by querying g at a single point. (Note that evaluating $\tilde{g}(a) = \sum_{ij} a_{ij}u_iu_j \pmod 2$ we are actually multiplying the coefficients a_{ij} of the quadratic form by the values from the assignment, that is, we are evaluating the quadratic form a on u .)

Note that every time we query $\tilde{f}$ or $\tilde{g}$, we may err with small probability. By reducing the error probability as needed (by repeat), we obtain a good probability of error (less than $1/2$) by the union bound (that is, assuming that none of these unfortunate events will happen, our verification algorithm catches an incorrect proof, and we estimate the total probability of error as the sum of all these probabilities of errors). $\square$

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: A modern approach*. Cambridge University Press, 2009.